408 应试大题手册

考点×暴力解模板×真题年份溯源

数据结构 · 计算机组成原理 · 操作系统 · 计算机网络

2009–2026 统考真题大题全对齐

只写会考的。数据结构算法题给可直接默写的暴力解模板；其余三科每个考点都给可照抄的解题流水线，并标注该考点在哪一年的大题里出现过。

目 录

这本册子怎么用 . . . 4
分值结构与时间预算（必背） . . . 4
答题顺序建议 . . . 4
四科大题考点年份总表（速查） . . . 4

第一编 数据结构 . . . 9

答题规范与保底策略 . . . 10
顺序表暴力解模板 . . . 10
链表结点定义与建表 . . . 14
通用兜底套路 . . . 15
链表题五套路总览 . . . 16
双指针 · 倒数第 k 个 . . . 17
快慢指针 · 求中点 . . . 18
双指针 · 判环与入环点 . . . 18
头插法 · 整体逆置 . . . 19
头插法 · 局部逆置 . . . 19
长度差对齐 · 公共结点 . . . 20
前驱删除 · 绝对值去重 . . . 21
拆分 · 重排 · 合并 . . . 21
哨兵结点与 free 纪律 . . . 22
树 · 递归遍历四件套 . . . 23
树 · 带权路径长度 WPL . . . 24
树 · 中序遍历转中缀 . . . 24
树 · 二叉搜索树判定 . . . 25
树 · 层次遍历 . . . 26
树 · 线索二叉树 . . . 26
哈夫曼树构造流水线 . . . 27
哈夫曼编码与平均码长 . . . 29
Dijkstra 手算填表 . . . 30
Prim 与 Kruskal 手算 . . . 31
拓扑排序与 AOE 网 . . . 32
邻接矩阵幂与度计算 . . . 34
图的应用速查表 . . . 35
排序总表（必背） . . . 36
一趟结果的判定 . . . 37
快排划分模板 . . . 38
堆三动作与堆排序 . . . 39
归并与逆序对 . . . 39
比较计数排序 . . . 40
外部排序三件套 . . . 41
排序思想解算法题 . . . 41

第二编 计算机组成原理 . . . 43

补码十条性质速查 . . . 44
补码加减与三种判溢出 . . . 44

标志位 CF/ZF/OF/SF . . . 45
移位：算术逻辑循环 . . . 46
乘法与除法流水线 . . . 47
Cache 五角度总览 . . . 48
Cache 地址拆分模板 . . . 49
Cache 总容量与性能计算 . . . 50
Cache 写策略与脏位 . . . 51
定长扩展操作码切法 . . . 51
寻址方式全表与 EA . . . 52
相对寻址求转移目标 . . . 53
机器码与汇编互译 . . . 54
汇编循环三件套 . . . 55
单总线取指周期微操作 . . . 55
控制信号表怎么写 . . . 56
流水线五段与三类相关 . . . 57
流水线性能三公式 . . . 58
三种 I/O 方式总表 . . . 59
程序查询方式时间计算 . . . 59
中断响应全过程与传输时间 . . . 60
中断嵌套、优先级与屏蔽字 . . . 61
DMA 传送流程与争总线 . . . 61
DMA 传送时间与 CPU 占用率 . . . 62
磁盘参数计算 . . . 63
三种方式 CPU 开销对比模板 . . . 63

第三编　操作系统 . . . 65
PV 大题五步流程 . . . 66
信号量初值对照表 . . . 67
生产者—消费者模板 . . . 67
读者—写者与计数互斥 . . . 68
前驱图与先后同步 . . . 69
限流容量信号量 . . . 70
连续 k 次才释放 . . . 71
对称同步双重信箱 . . . 71
方向互斥独木桥 . . . 72
售票与流水线变体 . . . 73
分页拆位总模板 . . . 74
多级页表拆位流水线 . . . 75
页表项物理地址计算 . . . 76
多级页表占页数估算 . . . 76
访存次数统计模板 . . . 77
有效访问时间 EAT . . . 78
置换算法填表模板 . . . 79
请求分页完整访存流程 . . . 80
软硬件互斥代码模板 . . . 81
三种物理结构总表 . . . 82

连续分配插入删除次数 . . . 83
混合索引最大文件长度 . . . 84
偏移定位与读盘块数 . . . 85
FAT 表大小与路径查找 . . . 87
删除目录的元数据操作 . . . 87
磁盘调度手算模板 . . . 88
磁盘地址与簇号换算 . . . 90
位示图大小计算 . . . 91

第四编 计算机网络 . . . 92
分章分值与大题分布 . . . 93
时延四兄弟 · 计算模板 . . . 93
信道容量 · 奈氏与香农 . . . 94
编码与调制速查 . . . 95
停等 GBN SR 利用率 . . . 96
CSMA/CD 争用期与帧长 . . . 96
以太网帧逐字段解析 . . . 97
交换机自学习与作用域 . . . 98
802.11 地址四字段 . . . 99
PPP 与 HDLC 及 VLAN . . . 100
网络层大题题型地图 . . . 100
IPv4 首部逐字段 . . . 101
IP 分片计算模板 . . . 103
子网划分三步法与 VLSM . . . 103
路由聚合与最长前缀匹配 . . . 104
路由表转发决策流程 . . . 105
ARP 全过程与帧的变化 . . . 106
DHCP 四报文与租期 . . . 107
NAT 地址转换表填写 . . . 107
RIP / OSPF / BGP 对比 . . . 108
从 A 到 B 的完整答题模板 . . . 109
UDP 与 TCP 对比表 . . . 110
TCP 首部逐字段 . . . 110
TCP 三次握手流水线 . . . 111
四次挥手与 TIME_WAIT . . . 112
序号与确认号计算模板 . . . 112
流量控制与滑动窗口 . . . 113
拥塞控制四阶段 . . . 114
吞吐率与传输时间 . . . 115
DNS 递归与迭代查询 . . . 116
HTTP 请求响应与连接时间 . . . 116
从输入网址到页面显示 . . . 117

title: 408 应试大题手册

这本册子怎么用

它不是讲义，是考前能直接翻、直接背、直接抄上答题卡的作战手册。三条用法：

1. 算法题只背模板。数据结构的算法大题 15 分上下，评分按「设计思想 + 代码 + 复杂度」三段给分，写成暴力解也照样拿分段分。全书 120 多段可直接抄的代码模板与解题流水线，考场按题型对号入座即可。
2. 计算题只背流水线。计组、操作系统、计网的大题全是「按步骤数格子」，每个考点都给了编号步骤（step1 step2 …）和可直接代入的公式，照着走不会漏。
3. 年份标注是重点指示器。每个考点标题下面都有一条棕色「真题年份」栏，写的就是这个考点在哪一年的大题里出现过。年份密集的考点必复习；写着「大题未考」的，只需要应付选择题。

一句话：把年份最密的 12 个考点吃透，70 分的大题能拿到 45 分以上。

分值结构与时间预算（必背）

综合应用题第 41–47 题共 70 分，四科分配常年稳定：

科目	大题题数	大题分值	选择题	科目总分
数据结构	2 题（算法 + 应用）	21 ~ 25	约 11 题 22 分	45
计算机组成原理	2 题	21 ~ 24	约 11 题 22 分	45
操作系统	2 题	14 ~ 17	约 10 题 20 分	35
计算机网络	1 题（2014、2018 为 2 题）	7 ~ 9	约 8 题 16 分	25
合计	7 ~ 8 题	70	40 题 80 分	150

题号不是固定的：2009–2012 年算法题在第 42 题、应用题在第 41 题；2016 年算法题在第 43 题。进考场先扫题干认题，别背题号。本册只保证年份准确，不保证题号。

大题分值不等于科目总分——数据结构大题 23 分最高，计网只有 9 分。时间要按分值投：算法题 20–25 分钟，计组两题 25 分钟，OS 两题 20 分钟，计网 12–15 分钟。留 10 分钟复查。

答题顺序建议

1. 先扫一遍 **41–47** 的题干，30 秒判断哪两道最有把握。
2. 千万别把算法题放最后。算法大题是唯一「写代码就有分」的题，哪怕只写出思路和 20 行暴力解，也有 8–11 分。
3. 计网大题只有 7–9 分但通常要画表算好几层，性价比最低，放最后做，先拿第 **1** 小问的分。
4. 每道题的「为什么」「说明理由」小问都是白送分，宁可代码少写两行，也要把理由写满。

四科大题考点年份总表（速查）

下面五张表把 2009–2026 全部 18 年的大题按考点归了类。复习时按这张表倒推：年份密集的考点必考，隔年出现的说明轮考。表里每一行在本册正文都有对应一节，翻目录即可定位。

▶ 数据结构 · 算法题（每年必考，8　15 分）

年份	章节	考点
2009	线性表	单链表倒数第 k 个结点（双指针）
2010	线性表	数组循环左移 p 位（三次逆置）
2011	线性表	两有序序列中位数（二分/减治）
2012	线性表	单链表公共结点（双指针 + 长度差）
2013	线性表	主元素（摩尔投票）
2014	树	二叉树带权路径长度 WPL（递归）
2015	线性表	单链表按绝对值去重
2016	排序	数组划分（快排 Partition 应用）
2017	树	二叉树中序遍历应用（表达式树/转中缀）
2018	线性表	数组原地哈希/桶（最小未出现正整数）
2019	线性表	单链表拆分、逆置、合并
2020	线性表	有序数组多指针（三元组距离最小）
2021	图	基于邻接矩阵的图遍历（连通分量/度）
2022	树	二叉搜索树判定
2023	图	有向图出度/入度统计
2024	图	拓扑排序
2025	线性表	数组线性遍历与预处理
2026	树	二叉树遍历与线索二叉树

▶ 数据结构 · 应用题（每年一道，8 12 分）

年份	章节	考点
2009	图	Dijkstra 与贪心策略
2010	查找	散列表应用
2011	图	图的应用
2012	树	哈夫曼树的应用
2013	查找	查找算法及其优化
2014	图	邻接表 + 最短路径
2015	图	邻接矩阵幂运算的含义
2016	树	正则 k 叉树推导
2017	图	最小生成树 Prim
2018	图	最小生成树
2019	栈队列	队列存储结构设计
2020	树	哈夫曼编码
2021	排序	比较计数排序
2022	排序	堆排序
2023	排序	外部排序

年份	章节	考点
2024	查找	散列表应用
2025	图	AOE 网
2026	栈队列	卡特兰数、栈的应用

▶ 计算机组成原理（每年两题，共约 23 分）

年份	考点一	考点二
2009	I/O	中断与 DMA 的 CPU 时间开销；单总线指令执行节拍与控制信号
2010	指令系统 / 存储层次	指令格式各类寻址机器码；Cache 映射命中率空间局部性
2011	数值运算 / 存储层次	补码加减与溢出判断；页式虚拟存储 +TLB+Cache 地址划分
2012	CPU 性能 / 流水线	CPU 性能指标交叉存储器带宽；五级流水线数据相关阻塞
2013	CPU 性能 / 指令系统	Cache 缺失下总执行时间；条件转移相对寻址与标志位
2014	指令系统 + 流水线 / 存储层次	循环汇编分支相对寻址流水线冒险；Cache 缺失缺页 TLB 访问次数
2015	数据通路 / 指令系统	单总线 CPU 数据通路硬件组成；多地址指令编码与访存时钟
2016	I/O / 存储层次	异步串行 + 中断综合时钟开销；Cache 映射虚实地址分段
2017	数值运算 / 指令系统	IEEE754 精度溢出无符号循环；CISC 机器码进位标志浮点
2018	I/O / 存储层次	查询/中断/DMA 三种 I/O 占用率；多级页表 Cache 参数
2019	指令系统 + 数值运算 / 存储层次	递归调用寻址乘法溢出；Cache 地址分段
2020	数值运算 / 存储层次	移位乘法多位数乘积溢出；Cache 缺失次数
2021	指令系统 + 数值运算 / 存储层次	R/I/J 三类指令格式与溢出；TLB 映射 LRU 替换
2022	数值运算 + 数据通路 / I/O	ALU 符号溢出标志逻辑表达式；磁盘参数 DMA 总线请求
2023	存储层次 / 指令系统	数组遍历 Cache 命中率局部性；跳转变址寻址大小端
2024	指令系统 + 数值运算 / 指令系统 + 存储	RISC-V 指令 ALU 溢出；大小端字节重组数组寻址页计算
2025	存储层次 / 数值运算 + 数据通路	Cache 缺失率缺页统计；补码除法器除法异常
2026	指令系统 + 数值运算 / 数据通路	多格式定长指令编码与运算求值；CPU 通路扩展器控制信号

▶ 操作系统（每年两题，共约 15 分）

年份	考点一	考点二
2009	PV / 内存	生产者消费者三进程 PV；TLB+ 缺页访问时间与 LRU
2010	文件 / 内存	CSCAN 磁盘调度总耗时；FIFO/Clock 置换求物理地址
2011	PV / 文件	银行叫号 PV；三种文件物理分配方式对比
2012	内存 / 文件	自定义页面回收置换策略模拟；混合索引最大文件长度
2013	PV / 内存	博物馆 500 人 PV；一/二级页表地址拆分与页表项地址
2014	PV / 文件	环形缓冲区 + 消费者连续取 10 件 PV；连续/链接分配插入磁盘访问次数

年份	考点一	考点二
2015	PV / 内存	信箱辩论 PV；二级页表地址拆分与占用页表数量
2016	进程 / 文件	动态优先级调度解决饥饿；FAT 树形目录与路径访问簇号
2017	PV / 内存 + 系统调用	独木桥 PV；二级分页地址转换 + 系统调用状态切换
2018	内存 / 文件	二级页表十六进制地址构造；inode 多级混合索引容量
2019	PV / 文件	读者写者变体 PV；SSTF 磁盘调度与簇号磁盘地址转换
2020	PV / 内存	售票/同步 PV；二级页表二维数组地址与局部性
2021	PV / OS 概述 + 文件	流水线装配 PV；系统引导与磁盘格式化操作顺序
2022	PV / 文件	缓冲区同步 PV；inode 间接索引分级与目录读盘次数
2023	PV / 进程 +I/O	同步 PV；中断 I/O 下进程完整状态转换
2024	PV / 内存	同步 PV；大页分页缺页、页表项虚实地址换算
2025	PV / OS 概述 + 进程	同步 PV；程序变量对应进程内存分区
2026	进程 / 文件	优先级 + 时间片轮转调度时序推演；inode 索引偏移读盘次数与目录删除

操作系统是四科里最模板化的：PV 考了 13 次、内存/分页考了 9 次、文件与磁盘考了 10 次。这三块就是全部。

计网 18 年里有 **11** 年落在网络层（子网划分、路由转发、分片）。拿到题先看有没有 IP 地址，有就先算子网掩码和网络地址，这一问必得分。

▶ 计算机网络（每年一至两题，约 9 分）

年份	章节	考点
2009	网络层	子网划分、路由表与路由聚合
2010	数据链路层	CSMA/CD 冲突检测与以太网效率
2011	数据链路层	以太网帧解析、ARP、HTTP 与 IP 转发
2012	网络层 + 传输层	IP/TCP 首部解析、握手、填充与 TTL
2013	网络层	自治系统路由、最长匹配与路由聚合
2014(42 题)	网络层	OSPF 链路状态、Dijkstra 最短路径
2014(43 题)	网络层	路由表、IP 转发、TTL 与 LSI 更新
2015	网络层 + 链路层	DHCP、ARP、子网掩码与默认网关
2016	传输层	TCP 握手、流量/拥塞控制、吞吐率与连接释放
2017	数据链路层	GBN、捎带确认、重传与信道利用率
2018(42 题)	网络层	最小生成树组网与 TTL 可达性
2018(47 题)	网络层	子网划分、广播地址、IP 分片与片偏移
2019	网络层 + 链路层	交换机/路由器、NAT 与定向广播
2020	网络层	局域网互联、路由转发与 NAT 地址变化
2021	应用层 + 链路层	DNS/HTTP 封装、ARP 与交换表学习
2022	链路层 + 网络层	冲突域、DHCP、交换机与 802.11 地址
2023	应用层 + 传输层	FTP、TCP 序号/挥手、拥塞窗口与速率
2024	网络层 + 路由协议	RIP、OSPF、TTL、BGP 与路径选择
2025	网络层 + 传输层	卫星链路、GBN 窗口与 VLSM 子网划分
2026	传输层	TCP 三次握手、发送窗口、挥手与传输时间

第一编

数据结构

答题规范与保底策略

真题年份 | 2009–2025 每年一道（15 分） | 每年必有一道算法题，三段式给分结构从 2009 年至今没变过

必背结论

- 算法题固定三段式：①中文写设计思想②写核心函数③写时空复杂度。三问全答满，即使全用暴力解也保底 11 分；只写思路不写代码最多 5 分；空着 0 分。
- 只写一个核心函数，题干给的所有已知条件（数组名、长度 n、参数 k、值域 m）全部当函数参数传进来，不自己造全局变量。
- 不写头文件、不写 main、不写输入输出；唯一例外是题干明确要求「输出」时必须写 printf，此时函数返回值反而可以不写。
- 代码只许 C 或 C++，Java 从 2015 年起不给用。
- 动笔顺序：先在草稿上定下「用什么手段」（双指针/辅助数组/三次逆置）→直接把代码写完→再回头写第一问的设计思想。这样三问必然对应，不会自相矛盾。
- 代码控制在 10~20 行，注释只加在「每个循环的入口」和「关键判断语句」两处。判卷老师不会运行代码，基本只看注释和循环骨架。
- 版面纪律：每小问的答案写在最显眼的位置，过程能省就省。一个大题常常拆成三四个小问、每个小问一两分，判卷按点找答案，找不到就不给分。
- 第三问的复杂度按循环结构直接算：并列的多个循环取最大阶，嵌套的相乘；开了 k 个辅助单元就写 O(k)，写不出最优解就照实写，虚报比写大更扣分。
- swap、abs、取最大最小直接用现成的，自己实现不加分只加错误率。

万能开场句（第一问第一句直接照抄）：「本算法采用一前一后两个指针从两端向中间扫描」「采用一个与值域等大的辅助数组充当哈希表，统计每个值是否出现过」「采用头插法将原链表就地逆置」，随后补一句「时间复杂度 O(n)，空间复杂度 O(1)」。先把这句落在卷面上，再展开细节。

小问	考什么	分值	拿分写法
第一问	设计思想（中文描述）	4~5	一句话点明手段 + 关键指针含义 + 一趟还是两趟，绝对不要写伪代码
第二问	核心函数代码	7~8	只写一个函数，3~4 条注释，超长循环可拆成两段
第三问	时空复杂度	2~3	直接写 O(...)，每个补半句理由（几个循环、开了多大数组）

三条开数组纪律

- 纪律一：动态辅助数组一律写 `int *t = (int *)malloc(k * sizeof(int));`，不要写 `int t[k]`，变长数组在标准 C 编译器下直接报错。
- 纪律二：开完立刻手动清零，`for (i = 0; i < k; i++) t[i] = 0;`。不写这一步，内存里的脏数据会让后面全错，而且判卷看不出来你错在哪。
- 纪律三：大小一次定死，存的时候怎么偏移，取出来就怎么还原。开 N 个就要处处 `-1`，开 N+2 个就原封不动存——宁可多开两个，不要给自己留偏移量。

> 只写设计思想不写代码，15 分里一般只给 4~5 分 · `int t[p]` 这种变长数组是典型编译错误，判卷一眼看出不专业 · 找到答案用 break 而不是 return，break 之后那句兜底输出照样执行，答案变两个

顺序表暴力解模板

真题年份 | 2010/2013/2018/2020/2025 | 线性表 18 年共出 10 道大题，其中数组类 6 道，是链表之外的另一半

必背结论

- 顺序表题只有两个手段：双指针扫描（时间换空间）和辅助数组统计（空间换时间）。审题第一件事就是看题干给不给值域、给不给空间 O(1)。
- 判信号：要求空间 O(1) →左右夹逼或读写指针；值域已知且有限→开辅助数组当哈希表；要求保持相对次序→一律不许排序。
- 写不出最优解就先写通用解，本章五个模板里只有循环左移的三次逆置属于「技巧」，其余全是可套的骨架。

▶ 双指针 · 左右夹逼与原地逆置

真题年份 | 2010 | 逆置是数组题的母操作，2010 循环左移与 2019 拆分合并两道题里它都是直接得分点

必背结论

- 左指针 `i = 0`，右指针 `j = n-1`，交换后 `i++`、`j--`，`i < j` 一破就停——正中间那个元素不用管。
- 逆置是「原地」的，只借一个 `temp` 变量，空间 O(1)；新开数组逆置只能拿复杂度分，拿不到最优分。
- 快排一趟划分用的是同一套指针，只是多了三条性质：取枢轴、从右往左起步、每次交换改变移动方向。

```
/* 逆置 A[left..right]，闭区间 */
void Reverse(int A[], int left, int right) {
    int i = left, j = right, t;
    while (i < j) {
        t = A[i]; A[i] = A[j]; A[j] = t;   /* 交换左右指针所指元素 */
        i++; j--;                          /* 向中间夹逼，相遇即停 */
    }
}
/* 快排一趟划分：以 A[low] 为枢轴，小的甩左边大的甩右边，返回枢轴最终位置 */
int Partition(int A[], int low, int high) {
    int p = A[low];                        /* 枢轴取出来，位置就空出来了 */
    while (low < high) {
        while (low < high && A[high] >= p) high--;  /* 从右往左找比枢轴小的 */
        A[low] = A[high];
        while (low < high && A[low] <= p) low++;    /* 再从左往右找比枢轴大的 */
        A[high] = A[low];
    }
    A[low] = p;                            /* 两指针相遇处填回枢轴 */
    return low;
}
```

逆置写成 `i <= j` 会把中间那个元素和自己交换，虽不致命但白跑一趟 · 快排划分第一步必须从右往左扫，写反了枢轴位置必错 · `while` 内层两个循环都要带 `low < high` 越界判断

▶ 双指针 · 前后同向（读写指针）

真题年份 | 2025 | 2025 考的就是「一趟线性遍历 + 预处理」，这是它的标准答法

必背结论

- 读指针 `j` 扫全表、写指针 `i` 只写要保留的元素，一趟结束 `i` 就是新表长，空间 O(1)。
- 预处理 = 先花一趟把「到每个位置为止的信息」（前缀和、前缀最值、前缀计数）算进 `pre[]`，第二趟 O(1) 取用，总体仍是 O(n)。
- 后缀最值同理：从 `n-1` 往 `0` 倒着扫一遍就能得到「i 右边是否有更大值」这类信息。
- 题干出现「尽可能高效」「一遍扫描」「不能改变相对次序」这类措辞，答案基本就是单趟线性遍历；出现「每个位置左边/右边」就一定先预处理再回答。

```
/* 原地压缩：读指针 j 扫全表，写指针 i 只写保留下来的，返回新长度 */
int Compact(int A[], int n) {
```

```
    int i = 0, j;
    for (j = 0; j < n; j++)
        if (A[j] != 0)                  /* 换掉这个条件就是任意「保留」规则 */
            A[i++] = A[j];
    return i;
}
/* 预处理前缀和: pre[i] = A[0]+...+A[i-1], pre 要开 n+1 个 */
void PrefixSum(int A[], int n, int pre[]) {
    int i;
    pre[0] = 0;                         /* 空前缀, 下标从 1 对齐 */
    for (i = 1; i <= n; i++)
        pre[i] = pre[i - 1] + A[i - 1];/* 把 + 换成 max/min 就是前缀最值 */
}
/* 用 pre 回答任意区间和: A[l..r] 的和 = pre[r+1] - pre[l] */
```

写指针 i 与读指针 j 别用同一个变量，压完就找不回原位置 · 前缀数组必须开 n+1 个，开 n 个时 pre[n] 越界 · 预处理后仍然要遍历第二趟，别忘了复杂度写成 O(n) 而不是 O(1)

▶ 快慢指针 · 有序表删重复

真题年份 | 选择常考，大题未考 | 有序表的删重复与压缩，是 2015 链表绝对值去重题的数组预演

必背结论

- 有序表去重靠「比较相邻」: A[j] != A[i] 才保留，一趟 O(n)、空间 O(1)。
- 无序表去重不能这么写，必须先排序（O(nlogn)）或用辅助数组当哈希表（O(n) 时间 + O(n) 空间）。
- 快指针探路、慢指针驻守，是「原地删」这一类题的统一骨架；链表上的 2015 题是它的链式版本。

```
/* 有序顺序表去重: i 是慢指针（最后写入位置）, j 是快指针（探路） */
int Unique(int A[], int n) {
    int i = 0, j;
    if (n == 0) return 0;
    for (j = 1; j < n; j++)
        if (A[j] != A[i])               /* 与「已保留的最后一个」比，不是与 A[j-1] 比 */
            A[++i] = A[j];
    return i + 1;                       /* 新表长 = 最后下标 + 1 */
}
/* 删除所有值为 x 的元素（无序也能用）: 计数法 */
int DelX(int A[], int n, int x) {
    int i, k = 0;
    for (i = 0; i < n; i++)
        if (A[i] != x) A[k++] = A[i];  /* k 同时是写指针和新长度 */
    return k;
}
```

去重要和「已保留序列的末尾」比，写成与 A[j-1] 比在有三连重复时会漏删 · 返回 i 而不是 i+1 是最常见的一分之差 · n == 0 的边界判断不写，A[0] 直接越界

▶ 辅助数组当哈希表

真题年份 | 2013/2018 | 「数组当 map」是三大高频思想之一，统计类题目通吃

必背结论

- 把原数组的值当成辅助数组的下标，t[A[i]]++，一趟统计完再扫辅助数组，时间 O(n)、空间 O(值域)。
- 两个前提必须写进设计思想: 原数组的值必须是整数且非负（负数、小数不能当下标），且必须提前知道值域上界。
- 只关心「是否出现」时，t[A[i]] = 1 也行，但写 ++ 适用范围更广（次数为 0 即没出现），不亏。
- 值域上界不给时自己推: 长度为 n 的数组，未出现的最小正整数最大只可能是 n+1，所以开 n+2 个就够。

```
/* 统计 A[0..n-1] 中值域 0..m 各值出现次数, 返回第一个没出现过的值 */
int HashCount(int A[], int n, int m) {
    int i, *t = (int *)malloc((m + 1) * sizeof(int));
    for (i = 0; i <= m; i++) t[i] = 0;          /* 纪律二: 必须手动清零 */
```

```
    for (i = 0; i < n; i++)
        if (A[i] >= 0 && A[i] <= m) t[A[i]]++;  /* 越界必须挡掉，否则数组踩坏 */
    for (i = 0; i <= m; i++)
        if (t[i] == 0) { free(t); return i; }   /* 次数为 0 = 没出现过 */
    free(t);
    return -1;                                  /* 全出现过了 */
}
/* 未出现的最小正整数：辅助数组开 n+2 个，原封不动存，不减 1 */
int FirstMissing(int A[], int n) {
    int i, *t = (int *)malloc((n + 2) * sizeof(int));
    for (i = 0; i < n + 2; i++) t[i] = 0;
    for (i = 0; i < n; i++)
        if (A[i] >= 1 && A[i] <= n + 1) t[A[i]]++;   /* 只收 1..n+1，大的直接丢 */
    for (i = 1; i <= n + 1; i++)
        if (t[i] == 0) { free(t); return i; }
    free(t);
    return n + 2;                               /* 理论到不了，只为有返回值 */
}
```

开 n 个辅助数组要「减 1 存、加 1 取」，忘了加回来是头号扣分点，宁可开 n+2 个 · 不清零直接用，脏数据会让统计结果全错 · 输出的是值本身不是出现次数，`t[i]` 和 `i` 别写混

▶ 循环左移与循环右移

真题年份 | 2010 | 数组类算法题的基础母题，逆置思想在多道真题里复用

必背结论

- 循环左移 p 位 = 逆置前 p 个→逆置后 n−p 个→整体逆置；循环右移 p 位 = 整体逆置→逆置前 p 个→逆置后 n−p 个。左移是「先分后合」，右移是「先合后分」。
- 时间 O(n)（三次逆置加起来 2n 次交换）、空间 O(1)，这是「时间空间都尽可能高效」的标准答法。
- 想不出三次逆置就写通用解：开 p 个辅助数组暂存前 p 个，后面整体前移，再把暂存接回末尾，时间 O(n+p)、空间 O(p)，一样拿分。
- `p %= n` 必须写，题目只保证 p 是正整数、不一定给 p < n。

```
/* 逆置器：三道题都靠它，写成独立函数，别在每道题里重写一遍 */
void Reverse(int A[], int l, int r) {
    int i = l, j = r, t;
    while (i < j) { t = A[i]; A[i] = A[j]; A[j] = t; i++; j--; }
}
/* 循环左移 p 位：先逆置前 p 个，再逆置后 n-p 个，最后整体逆置 */
void LeftShift(int A[], int n, int p) {
    p = p % n;                          /* 防 p >= n */
    if (p == 0) return;                 /* 不移，直接返回 */
    Reverse(A, 0, p - 1);               /* ① 前 p 个逆置 */
    Reverse(A, p, n - 1);               /* ② 后 n-p 个逆置 */
    Reverse(A, 0, n - 1);               /* ③ 整体逆置 */
}
/* 循环右移 p 位：顺序反过来，整体逆置放第一步 */
void RightShift(int A[], int n, int p) {
    p = p % n;
    if (p == 0) return;
    Reverse(A, 0, n - 1);               /* ① 整体逆置 */
    Reverse(A, 0, p - 1);               /* ② 前 p 个逆置 */
    Reverse(A, p, n - 1);               /* ③ 后 n-p 个逆置 */
}
/* 通用解：开 p 个辅助数组暂存，谁都能想到，照样给分 */
void LeftShiftBuf(int A[], int n, int p) {
    int i, *t;
    p = p % n;
    if (p == 0) return;
    t = (int *)malloc(p * sizeof(int));
    for (i = 0; i < p; i++) t[i] = A[i];       /* 暂存前 p 个 */
    for (i = p; i < n; i++) A[i - p] = A[i];   /* 后面 n-p 个整体前移 */
    for (i = 0; i < p; i++) A[n - p + i] = t[i]; /* 暂存接回末尾 */
    free(t);
}
```

三次逆置的顺序是「先逆置两个分段、最后整体」，写反就全错 · `p %= n` 漏写在 p 大于 n 时数组越界 · 右移 p 位等价于左

移 n−p 位，只需写一个函数也能换算，但换算错了比不换算扣得更多

顺序表题的复杂度分最容易白丢：三重循环就说 $O(n^3)$，别硬写成 O(n)·边界条件（偏移量取模、下标上限）少一句，换组数据就崩·辅助数组开在栈上（`int t[1000]`）在 n 大时直接爆栈，一律用 malloc

链表结点定义与建表

真题年份 | 选择常考，大题未考 | 但它是 2009/2012/2015/2019 四道链表题的第一段必写内容

必背结论

- 结点定义必须能默写，`}` 后面那个分号不能丢；结构体内自引用指针在 C 里必须写 `struct LNode *next`，写 `LNode *next` 只有 C++ 能过。
- `ElemType` 只是书上用的万能占位符，题目数据是整数就直接写 `int`，是字符就写 `char`，考场上不写 `ElemType`。
- `LNode *` 用来操作单个结点（插入、删除、修改），`LinkList` 用来操作整条链表（建表、逆置、遍历），两者本质等价但函数签名要表达意图。
- 链表题默认带头结点：头结点不存数据，所以一切遍历一律从 `head->next` 起步，写 `p = head` 起步必错位。
- 头插法建出的表与输入顺序相反，尾插法建出的表与输入顺序一致；头插法天然就是逆置，2019 题的后半段直接拿它用。
- 判表尾不能一律写 `p->next == NULL`：循环单链表表尾判据是 `p->next == head`，循环双链表还要 `head->prior` 指向表尾，写错就是死循环；静态链表存的是下标、末尾存 -1。

▶ 结点定义

真题年份 | 选择常考，大题必写 | 2009 年官方评分说明：写出结构体类型定义与正确思想即给一半以上分数

```
/* 考试写到答题纸上就写这几行，题目数据是 int 就写 int */
typedef struct LNode {
    int data;                   /* 数据域：题目是 char 就改 char */
    struct LNode *next;         /* 指针域：C 里必须写 struct LNode * */
} LNode, *LinkList;             /* 别名① LNode = 一个结点；别名② LinkList = 一条链表 */
/* 静态链表（选择题用）：数组存数据和下一个元素的下标，末尾存 -1 */
typedef struct {
    int data;
    int next;                   /* 存的是下标不是地址，-1 表示结束 */
} SLinkList[100];
```

结尾分号漏掉，编译直接不过，这是最便宜的丢分点·`struct LNode *next` 去掉 struct 在标准 C 下编译不过·定义写在答题纸最前面，判卷老师看到它就先划一半分

▶ 头插法建表

真题年份 | 2019（第二段） | 建表是所有链表题的地基，头插法顺带就是逆置

```
/* 头插法：由数组 A[0..n-1] 建带头结点单链表，结果与输入顺序相反 */
LinkList HeadInsert(int A[], int n) {
    int i;
    LNode *head = (LNode *)malloc(sizeof(LNode));  /* 头结点，不存数据 */
    head->next = NULL;                             /* 空表：头结点后面暂时没东西 */
    for (i = 0; i < n; i++) {
        LNode *s = (LNode *)malloc(sizeof(LNode));
        s->data = A[i];
        s->next = head->next;   /* ① 新结点先接上原来的首结点 */
        head->next = s;         /* ② 再让头结点指向新结点：先接后断 */
    }
    return head;                /* 返回头结点，它就代表整条链表 */
}
```

两句赋值写反成「先 `head->next = s` 再 `s->next = head->next`」，s 会指向自己，链表成环死循环·循环内 s 是局部变量，每轮都重新 malloc，别在循环外只 malloc 一次·忘了 `head->next = NULL` 初始化，空表时头结点指向野指针

▶ 尾插法建表

真题年份 | 2019（第一段） | 要保序时必须用尾插，头插会把你建的表倒过来

```
/* 尾插法：由数组 A[0..n-1] 建带头结点单链表，结果与输入顺序一致 */
LinkList TailInsert(int A[], int n) {
    int i;
    LNode *head = (LNode *)malloc(sizeof(LNode));
    LNode *tail = head;          /* 尾指针先指头结点，之后一直指最后一个结点 */
    head->next = NULL;
    for (i = 0; i < n; i++) {
        LNode *s = (LNode *)malloc(sizeof(LNode));
        s->data = A[i];
        s->next = NULL;          /* 新结点将成为新表尾，指针域显式置空 */
        tail->next = s;          /* ① 接到当前表尾后面 */
        tail = s;                /* ② 尾指针后移一步，这两句都不能少 */
    }
    return head;                 /* 返回的是 head 不是 tail */
}
/* 求表长：所有链表题的第一步，除了头结点才计数 */
int Length(LinkList head) {
    int n = 0;
    LNode *p = head->next;       /* 从第一个实际结点开始 */
    while (p != NULL) { n++; p = p->next; }
    return n;
}
```

让 head 自己往后走充当尾指针，走完就找不回头结点了 · 漏掉 `tail = s` 这一句，下一轮会把新结点接在同一位置，链表断裂 · 返回 `tail` 而不是 `head`，调用者拿到的只是最后一个结点 · 求表长时从 `head` 数起，会把头结点算进去，全部结果差一

空表判据是 `head->next == NULL`，不是 `head == NULL`，带头结点的链表永远不空 · 建表循环里每轮都要新申请结点，在循环外申请一次等于所有结点共用同一块内存 · 求表长从 `head->next` 数起，从头结点数会让后面所有下标整体差一

通用兜底套路

真题年份 | 2009/2012/2015/2019/2025 | 任何一年实在想不出解法时，这条流水线都能保底 10 分

必背结论

- 兜底核心一句：链表题一律先转成数组，数组上用暴力解，能答满三问就有 **10** 分。代价是空间 O(n)，但比空着强太多。
- 转数组有两个硬前提：必须先求出表长 n（不知道长度就开不了数组），辅助数组要开 n+1 个且 0 号下标空着不用。
- 官方评分说明写明：采用两遍或多遍扫描、或用了与表长有关的辅助数组，最高给 10 分。这不是偏方，是被官方承认的保底线。
- 数组侧的兜底三件套：暴力枚举全部情况（三重循环也认）、排序后处理（选择排序 $O(n^2)$ 就够，不要求快排）、开辅助数组当哈希表。
- 实在没思路也要先写四样东西：结点/结构体定义、函数签名（把已知条件全部当参数）、一句设计思想、复杂度。这四样写完就是一半分。
- 兜底不适用的一种情况：题干明确要求「空间复杂度为 O(1)」，此时转数组会被判为不满足要求，只能老实用双指针，把三问写满仍有 11 分左右。

```
/* 兜底①：链表转数组。先求长，再存，0 号下标空着不用 */
int *ListToArray(LinkList head, int *pn) {
    int n = 0, i;
    int *A;
    LNode *p = head->next;
    while (p != NULL) { n++; p = p->next; }     /* 不知道长度就开不了数组 */
    A = (int *)malloc((n + 1) * sizeof(int));   /* 开 n+1 个，0 号浪费掉 */
    p = head->next;
    for (i = 1; i <= n; i++) {                  /* 从 1 号对齐链表第 1 个元素 */
```

```
        A[i] = p->data;
        p = p->next;
    }
    *pn = n;
    return A;                                   /* 调用者负责 free */
}
/* 兜底②：数组排序。要排序就写选择排序，别在考场上手写快排 */
void SelectSort(int A[], int n) {
    int i, j, k, t;
    for (i = 1; i <= n; i++) {
        k = i;
        for (j = i + 1; j <= n; j++)
            if (A[j] < A[k]) k = j;             /* 找最小值的下标 */
        if (k != i) { t = A[i]; A[i] = A[k]; A[k] = t; }
    }
}
/* 兜底③：全枚举。求最值把 ans 初始化成 INT_MAX，别初始化成 0 */
int MinDist(int A[], int B[], int C[], int na, int nb, int nc) {
    int i, j, k, d, ans = INT_MAX;              /* 初始值必须是最大值 */
    for (i = 1; i <= na; i++)
      for (j = 1; j <= nb; j++)
        for (k = 1; k <= nc; k++) {
            d = abs(A[i] - B[j]) + abs(B[j] - C[k]) + abs(A[i] - C[k]);
            if (d < ans) ans = d;               /* 三重循环 O(n³) 也给分 */
        }
    return ans;
}
```

五步兜底流水线

1. 先把结点/结构体定义写上（对链表题），这是白送的一半分。
2. 判断能不能转数组：不需要空间 O(1) 就转，需要就地就退回双指针。
3. 转成数组后用「排序 + 遍历 + 辅助数组」这套组合拳硬解，不要在考场上想最优解。
4. 代码写完补注释，注释只写在循环入口和关键判断处。
5. 复杂度照实写：多趟扫描就写 O(n²)、开了辅助数组就写 O(n)，别虚报成 **O(1)**，虚报反而是扣分项。

> ans 初始化成 0 求最小值，任何距离都比 0 大，永远不会更新 · 转数组前不先求表长，malloc 的参数是未知量，数组根本开不出来 · 数组从 0 号下标存起却按 1 号取，整体差一位 · 第 5 步虚报复杂度，比老老实实写 O(n²) 扣得更多

链表题五套路总览

真题年份 | 2009/2012/2015/2019 | 链表算法题固定 13–15 分，四个年份四道题，五套路全覆盖

必背结论

- 动笔前先确认两件事：带不带头结点（大题默认带）、题干的域名与数据域类型（link 还是 next，int 还是 char），照抄不改。
- 五套路按「改什么」分类：双指针改速度与间距、长度差对齐改起点、头插法改链接方向、前驱删除改指针停位、拆分合并改表的条数。
- 复杂度写两行就够：时间扫几遍就是几个 $O(n)$；没开辅助数组或辅助链表就是 $O(1)$，开了就是 $O(n)$，求表长的那一趟也要算进时间。
- 空间 $O(1)$ 是隐藏评分点：题目出现「空间复杂度为 O(1)」「尽可能高效」，就不许开等长数组。
- 兜底第 0 档：转数组（求长→存数组→按下标做→倒着重建成链表），思路分先拿到，绝不空着。
- 答题四步固定：结点定义→设计思想→代码（10 到 15 行量级）→时空复杂度，四段各占分。
- 得分口径：这四道题满分 13 分或 15 分（按年份），思路对且代码写出来约 10 分；只写思路不写代码分数很低；结点定义单独有分。

套路	识别特征	真题年份	代码骨架一句话
双指针	倒数第 k 个、中点、判环、两条链表	2009 / 2019	快指针先走 k 步，再与慢指针同步走
头插法逆置	逆序、反转、后半段翻转	2019	先断链，再逐个摘下往哨兵后面插
长度差对齐	两条链表求第一个公共结点	2012	长的先走长度差步，再同时走
前驱删除	去重、删满足条件的结点	2015	cur 停在前驱，删时不动、留时才走
拆分 + 重排 + 合并	要求 O(1) 空间重排、判回文	2019	找中点→逆后半段→交替合并

模板/流水线

本章零件代码共用这一份结点定义；四道真题的题干域名与类型各不相同（2009 用 `link`、2012 用 `next` 且 `data` 为 `char`、2015 用 `link`），那三段各自带一份题干版定义，照抄即可：

```
typedef struct LNode {
    int data;                   /* 题干是 char 就写 char */
    struct LNode *next;         /* 题干叫 link 就写 link */
} LNode, *LinkList;
```

返回值类型按题干定：2009 返回 int、2012 返回结点指针、2015 无返回值 · 题目说「不改变链表」时不许逆置、不许新建结点 ·「尽可能高效」既管时间也管空间，开数组前先看有没有 O(1) 字样

双指针 · 倒数第 k 个

真题年份 | 2009（15 分） | 双指针母题，2012、2019 都是它的变体，不会它后面全崩

必背结论

- 识别特征：题面出现「倒数第 k 个」「中间位置」「环」这类位置词，而不是「第几个值」。
- 推进规则：fast 先走 k 步，之后 fast、slow 一起走；fast 到 NULL 时 slow 正好落后 k 步，落在倒数第 k 个。
- 快指针先走的那一轮必须带 `fast != NULL`：k 大于表长时提前停，否则越界。
- 判「不存在」写 `if (slow == list) return 0`：慢指针一步都没动，就是 k 超过了表长，不是`!=`。
- 表长 n 的链表从头结点到 NULL 共 n+1 步，所以 k 只能取 1 到 n；k 取 n 时 slow 最后停在第一个数据结点。
- 写法二：把两轮合成一轮 `while (fast != NULL) { fast = fast->link; if (k-- <= 0) slow = slow->link; }`，时空复杂度与写法一完全相同，考场挑顺手的写。

模板/流水线

```
/* 2009：带头结点单链表，求倒数第 k 个结点；成功输出 data 返回 1，否则返回 0 */
typedef struct LNode {
    int data;
    struct LNode *link;                     /* 注：题干域名就是 link，不是 next */
} LNode, *LinkList;

int FindKth(LinkList list, int k) {
    LNode *fast = list, *slow = list;       /* 两个都从头结点出发 */
    while (k > 0 && fast != NULL) {         /* 注：fast != NULL 不能省 */
        fast = fast->link;
        k--;
    }
    while (fast != NULL) {                  /* 同步走：fast 到 NULL 时 slow 即答案 */
        fast = fast->link;
        slow = slow->link;
    }
    if (slow == list) return 0;             /* 注：慢指针没动 → 倒数第 k 个不存在 */
    printf("%d\n", slow->data);
    return 1;
}
```

复杂度：时间 $O(n)$，只扫一遍；空间 $O(1)$，只有两个指针。

漏写 `fast != NULL` 是这道题第一大坑，k 超长直接崩 · 判不存在写成 `if (slow != list)` 逻辑正好反，找到答案时反而返回 0 · k 等于表长时 slow 停在第一个数据结点，不是头结点

快慢指针 · 求中点

真题年份 | 2019（13 分） | 重排链表三步里的第一步，丢了中点整道题 13 分归零

必背结论

- 识别特征：要把链表「从中间劈开」「分前后两半」「判断回文」，第一步都是求中点。
- 循环条件必须写两个：`fast != NULL && fast->next != NULL`；少了后一个，`fast->next->next` 在最后一轮越界。
- 奇数个结点：slow 停在正中间；偶数个结点：slow 停在前半段的最后一个。奇偶都不含头结点。
- 起点取 `head` 本身而不是 `head->next`，否则上面的奇偶结论整体偏移一格。
- 求中点只走一遍链表，与后面的逆置、合并加起来仍是 $O(n)$，不要写成 $O(n\log n)$。
- 要「后半段」则起点是 `slow->next`；要「前半段含中点」则终点是 slow 本身。

模板/流水线

```
/* 求中点：奇数个停正中间，偶数个停前半段最后一个 */
LNode *FindMid(LinkList head) {            /* head 是头结点 */
    LNode *fast = head, *slow = head;
    while (fast != NULL && fast->next != NULL) {   /* 注：两个条件都要 */
        fast = fast->next->next;                   /* 快指针一次两步 */
        slow = slow->next;                         /* 慢指针一次一步 */
    }
    return slow;
}
```

复杂度：时间 $O(n)$，快指针走完半程即结束；空间 $O(1)$。

只写 `fast != NULL` 是最高频错误，偶数个时 `fast->next` 为空再取 next 就越界 · 求中点后若还要判「slow 是不是最后一个」，偶数个时 slow 后面还有结点 · 中点用返回值传出去，别用全局量

双指针 · 判环与入环点

真题年份 | 选择常考，大题未考 | 服务于 2019 那类快慢指针推理，是双指针的第三种推进规则

必背结论

- 有环必相遇：fast 每轮 2 步、slow 每轮 1 步，环内相对速度是 1，最迟一圈内追上。
- 无环的退出条件是两个：`fast == NULL || fast->next == NULL`，判断顺序不能反，先判 fast 再判 fast->next。
- 入环点公式：相遇后把一个指针放回起点，两者都改成每轮 1 步，再次相遇处就是入环点。
- 只需记住结论：起点到入环点的距离，等于相遇点到入环点的距离（按环长取模）。
- 复杂度：时间 $O(n)$，无环时快指针走约 n/2 轮就出界；空间 $O(1)$。
- 同型延伸「删除环」：先按上式找到入环点，再从入环点绕一圈回到它自己，把那个尾结点的 next 置空。

模板/流水线

```
/* 判环：有环返回 1 */
int HasCycle(LinkList head) {              /* head 是头结点 */
    LNode *fast = head, *slow = head;
    while (fast != NULL && fast->next != NULL) {
        fast = fast->next->next;
        slow = slow->next;
        if (fast == slow) return 1;
    }
    return 0;
}
```

```
/* 求入环点：无环返回 NULL */
LNode *CycleEntry(LinkList head) {
    LNode *fast = head, *slow = head;
    while (fast != NULL && fast->next != NULL) {
        fast = fast->next->next;
        slow = slow->next;
        if (fast == slow) break;           /* 第一次相遇 */
    }
    if (fast == NULL || fast->next == NULL) return NULL;   /* 无环 */
    slow = head;                           /* 一个指针回头结点 */
    while (slow != fast) {                 /* 同步走，再遇即入环点 */
        slow = slow->next;
        fast = fast->next;
    }
    return slow;
}
```

相遇后忘记把指针放回起点，直接返回相遇点（那是环内某点，不是入环点）· 无环时 break 后 fast 为 NULL，要先判空再进第二段 · 先判 `fast->next` 后判 `fast` 会先解引用空指针

头插法 · 整体逆置

真题年份 | 2019（13 分） | 重排链表的核心零件，也是「逆序输出、逆置」的通杀模板

必背结论

- 识别特征：要「逆序」「反转」「从尾往头」，或题目要求 $O(1)$ 空间重排。
- 五步固定顺序：断头 `head->next = NULL` →存后继 `temp = cur->next` →前插 `cur->next = head->next` →挂回 `head->next = cur` →推进 `cur = temp`。
- 第一步最容易漏：不断头，头插就变成在原串上打转，直接死循环。
- 逆置后原来的第一个结点变成最后一个，它的 next 由第一步置空来保证。
- 适用面：凡要「反向访问」又限空间 $O(1)$（逆序输出、判回文、两数相加、重排），第一刀都是它。
- 递归写法也能逆置，但递归栈会让空间变成 $O(n)$；题干要 $O(1)$ 就只认迭代版。

模板/流水线

```
/* 头插法整体逆置：空间 O(1)，不新建结点 */
void ReverseAll(LinkList head) {           /* head 是头结点，本身不参与逆置 */
    LNode *cur = head->next;               /* cur 指向第一个数据结点 */
    head->next = NULL;                     /* 注：第一步先断开，它成了已逆置串的哨兵 */
    while (cur != NULL) {
        LNode *temp = cur->next;           /* ① 先存后继，不存就断链 */
        cur->next = head->next;            /* ② cur 插到已逆置串最前面 */
        head->next = cur;                  /* ③ 头结点接上 cur */
        cur = temp;                        /* ④ 推进，不推进就死循环 */
    }
}
```

复杂度：时间 $O(n)$，每个结点摘下再插入一次；空间 $O(1)$，只借用 temp。

`head->next = NULL` 漏掉，逆置后表尾不空、遍历停不下来 · 不存 temp 直接改 `cur->next`，后半段整条丢失 · 把 head 也当成要逆置的结点，头结点被插进数据结点中间

头插法 · 局部逆置

真题年份 | 2019（13 分） | 同一段头插代码，把 head 换成子链前驱就是局部逆置

必背结论

- 三段式：切（找到子链前驱 pre、子链第一个 cur）→逆（对 cur 之后的结点做头插）→接（pre->next 已自动指向新子链头）。
- 局部逆置时 cur 固定不动，它就是逆置后的子链尾；真正在动的是 cur->next 之后的结点。

- 接回不需要额外指针：pre->next 始终跟着最新插入的结点走。
- m 等于 1 或 n 等于表长时同样成立，不用特判（再加哑结点就彻底统一）。
- 下笔前先画四个指针：pre（子链前驱）、cur（子链首，逆置后成尾）、temp（临时）、以及不动的表头 L。
- 与整体逆置的差别只有一个：插入位置从「表头之后」改成「pre 之后」，其余四步一模一样。

模板/流水线

```
/* 逆置第 m 到第 n 个数据结点（1 <= m <= n <= 表长），返回表头 */
LNode *ReverseRange(LinkList L, int m, int n) {
    LNode *pre = L;                         /* pre 最终停在 m-1 号位置 */
    int i;
    for (i = 1; i < m && pre != NULL; i++) pre = pre->next;
    if (pre == NULL || pre->next == NULL) return L;   /* m 越界，原样返回 */
    LNode *cur = pre->next;                 /* 子链第一个：逆置后它变成子链尾 */
    for (i = 0; i < n - m; i++) {
        LNode *temp = cur->next;
        if (temp == NULL) break;            /* n 超过表长就提前停 */
        cur->next = temp->next;             /* 把 temp 从 cur 后面摘下来 */
        temp->next = pre->next;             /* 头插到 pre 后面 */
        pre->next = temp;
    }
    return L;
}
```

复杂度：时间 $O(n)$（找 pre 用 $O(m)$，逆置用 $O(n-m)$）；空间 $O(1)$。

忘记先保存子链尾（cur）就逆置，接不回后半段 · m 越界时不返回原表而继续走，野指针崩溃 · 内层用 `n-m` 而不是 `n-m+1` 计数，会多逆一个结点

长度差对齐 · 公共结点

真题年份 | 2012（13 分） | 双指针的第二种推进规则，改的是起点而不是速度

必背结论

- 识别特征：两条链表、求「第一个公共结点」「交点」「共同后缀的起始位置」。
- 交点之后两表完全重合，所以长度差就等于交点之前那段的结点数差；长的先走差值步，两指针距表尾就等距了。
- 求表长要走两趟，必须算进复杂度：时间 $O(m+n)$，不是 $O(n)$。
- 循环写 `while (p != q)` 一起走，退出即交点；无交点时两者同时为 NULL，天然返回 NULL，不用特判。
- 长度差比较用严格大于：写成 `>=` 会多走一步（7 和 5 时走 3 步而不是 2 步）。
- 命名用 p、q，不用 fast、slow——谁长谁短事先不知道。
- 两表等长时长度差为 0，两个 while 都不执行，直接进入同步走，代码不用改。
- 备选：用 map 或哈希表存第一条链表的结点地址再扫第二条，时间 $O(m+n)$、空间 $O(n)$；硬性要求 $O(1)$ 空间时不能用。

模板/流水线

```
/* 2012：两条带头结点单链表（data 为 char），求共同后缀起始结点；无公共结点返回 NULL */
typedef struct LNode {
    char data;                              /* 注：题干是字符类型，不是 int */
    struct LNode *next;                     /* 注：题干域名是 next */
} LNode, *LinkList;

int Length(LinkList L) {
    int n = 0;
    LNode *cur = L->next;                   /* 头结点不计入长度 */
    while (cur != NULL) { n++; cur = cur->next; }
    return n;
}
LNode *FindCommon(LinkList str1, LinkList str2) {
```

```
    int len1 = Length(str1), len2 = Length(str2);
    LNode *p = str1, *q = str2;            /* 注：用 p、q，不用 fast、slow */
    while (len1 > len2) { p = p->next; len1--; }   /* 注：严格大于，不是大于等于 */
    while (len2 > len1) { q = q->next; len2--; }
    while (p != q) { p = p->next; q = q->next; }
    return p;                              /* 无交点时 p、q 同时为 NULL */
}
```

复杂度：时间 $O(m+n)$，两趟求长加一趟同步走；空间 $O(1)$，两个指针。

返回结点的值而不是结点指针，题目要的是「起始位置 p」· 长度差写成 >= 多走一步，交点永远差一格 · data 定义成 int 而题干是 char，结点定义分直接丢

前驱删除 · 绝对值去重

真题年份 | 2015（15 分） | 删结点类题的唯一起手式，「去重、删满足条件」全套它

必背结论

- 单链表删结点必须停在前驱：cur 从头结点出发，判断的永远是 cur->link 指向的那个结点，这样第一个数据结点也在删除范围内。
- 删分支里 cur 不动：cur->link 已被改写成下下个结点，此时再后移就漏判它。
- 留分支里才写 cur = cur->link。
- 辅助数组开 n+1 个（题干给 |data| <= n，下标最大取到 n），并且必须显式清零。
- 题干说「时间复杂度尽可能高效」等于允许用 $O(n)$ 空间换 $O(n)$ 时间。
- 判定用的是绝对值：-15 与 15 算同一个值，保留先出现的那个，顺序按原表，不许排序。

模板/流水线

```
/* 2015：删除绝对值相等的多余结点，仅保留第一次出现；题干给 |data| <= n */
typedef struct LNode {
    int data;
    struct LNode *link;                    /* 注：题干域名是 link */
} LNode, *LinkList;

void DeleteDup(LinkList head, int n) {
    int *a = (int *)malloc(sizeof(int) * (n + 1));   /* 注：下标取到 n，开 n+1 个 */
    int i;
    for (i = 0; i <= n; i++) a[i] = 0;               /* 注：必须清零，malloc 不清零 */
    LNode *cur = head;                               /* 注：从头结点出发：删除要停在前驱 */
    while (cur->link != NULL) {
        int v = cur->link->data;
        if (v < 0) v = -v;                           /* 取绝对值，判定用的是绝对值 */
        if (a[v]) {                                  /* 出现过 → 删掉它 */
            LNode *q = cur->link;
            cur->link = q->link;                     /* 注：cur 不动 */
            free(q);
        } else {                                     /* 第一次出现 → 留下 */
            a[v] = 1;
            cur = cur->link;                         /* 注：只有这里才后移 */
        }
    }
    free(a);                                         /* 辅助数组记得还 */
}
```

复杂度：时间 $O(n)$，每个结点只判一次；空间 $O(n)$，一个 n+1 的标记数组。

数组开 n 个，下标 n 越界（题目给的界是小于等于 n，这是送分点也是送命题）· 删完还让 cur 后移，连续两个重复值时漏删第二个 · 用原值当数组下标，负数直接越界（必须先取绝对值）

拆分 · 重排 · 合并

真题年份 | 2019（13 分） | 全书代码最长的一道，三步拼装，考的是零件熟不熟

必背结论

- 三步：找中点→后半段头插逆置→两半交替合并；空间 $O(1)$，时间 $O(n)$。
- 拆链先取后断：先 `q = slow->next`，再 `slow->next = NULL`；顺序反了就永远找不到后半段。
- 合并循环以后半段为准：`while (q != NULL)`；奇数个结点时前半段会多剩一个。
- 循环结束后必须补 `tail->next = p`，漏了奇数个就丢掉正中间那个结点。
- 逆置时 slow 当哑结点用，它自己不参与逆置，正好成为后半段逆置后的前驱。
- 同型变式「判断回文链表」：找中点 + 逆后半段 + 逐个比较，不合并、不补 p，遇到不等就立刻返回假。
- 复杂度写「链表近似走两遍，常数忽略，仍是 $O(n)$」，别写成 $O(n \log n)$。

模板/流水线

```
/* 2019: L = (a1..an) 重排为 (a1, an, a2, an-1, ...)，空间 O(1) */
typedef struct node {
    int data;
    struct node *next;
} node;
void Reorder(node *L) {                    /* L 是头结点 */
    /* ① 找中点：偶数个停前半段最后一个，奇数个停正中间 */
    node *fast = L, *slow = L;
    while (fast != NULL && fast->next != NULL) {
        fast = fast->next->next;
        slow = slow->next;
    }
    /* ② 后半段头插逆置：slow 当哨兵，自己不参与逆置 */
    node *cur = slow->next;
    slow->next = NULL;
    while (cur != NULL) {
        node *temp = cur->next;
        cur->next = slow->next;
        slow->next = cur;
        cur = temp;
    }
    /* ③ 拆成两条链表：先取后断，顺序不能反 */
    node *q = slow->next;                  /* 先取后半段 */
    node *p = L->next;                     /* 前半段第一个 */
    node *tail = L;                        /* 合并时的尾指针 */
    slow->next = NULL;                     /* 再断开 */
    /* ④ 交替合并：以后半段（短的）为循环条件 */
    while (q != NULL) {
        tail->next = p;  p = p->next;  tail = tail->next;
        tail->next = q;  q = q->next;  tail = tail->next;
    }
    tail->next = p;                        /* 注：奇数个时补最后一个，偶数个接 NULL 无害 */
}
```

复杂度：时间 $O(n)$，链表近似走两遍（常数忽略）；空间 $O(1)$，只借用 4 个指针。

先断后取，q 恒为 NULL，后半段整条丢失 · 循环条件写成 `p != NULL`，奇数个时 p 先空、访问 `p->next` 崩溃 · 循环外漏写 `tail->next = p`，正中间那个结点被丢掉 · 把 slow 也一起逆置进去，后半段多一个结点错位

哨兵结点与 free 纪律

真题年份 | 2009–2019（通用） | 不直接得分，但决定代码能否一次写对、内存题会不会被扣

必背结论

- 哑结点的作用只有一个：让「第一个结点」也有前驱，删除和插入都不用特判表头。
- 题目给的是不带头结点的链表时：函数第一句建哑结点，最后 `return dummy.next`，中间逻辑与带头结点完全一样。

- 哑结点放栈上（LNode dummy;）最省事，不用 malloc，也就不存在忘 free 的问题。
- 删掉的结点要 free，但顺序是先让 cur 走、再 free：free 之后再读 cur->next 就是野指针。
- 返回 void 的题就地改链；返回 LinkList 的题要返回新表头（哑结点的 next）。
- 头结点与哑结点不是一回事：头结点是题干给的、真实存在的结点；哑结点是你临时加的、用完可以丢的结点。

模板/流水线

```
/* 哨兵法：删除所有值为 x 的结点；链表不带头结点也能统一处理 */
LinkList DeleteX(LinkList L, int x) {
    LNode dummy;                          /* 注：栈上哑结点：不用 malloc，不用 free */
    dummy.next = L;                       /* 哑结点指向原来的第一个结点 */
    LNode *pre = &dummy, *cur = L;
    while (cur != NULL) {
        if (cur->data == x) {
            pre->next = cur->next;        /* 前驱越过 cur */
            LNode *del = cur;
            cur = cur->next;              /* 注：先让 cur 走 */
            free(del);                    /* 注：再 free，顺序反了就野指针 */
        } else {
            pre = cur;                    /* 只有留下时 pre 才跟着走 */
            cur = cur->next;
        }
    }
    return dummy.next;                    /* 新表头：原来第一个可能就是被删的那个 */
}
```

复杂度：时间 $O(n)$ 一遍；空间 $O(1)$，哑结点在栈上。

> free 之后又用 cur->next，野指针（最容易被扣分的一处）· 用 malloc 建哑结点却忘 free，内存泄漏 · 返回 L 而不是 dummy.next，删掉第一个结点后表头就丢了

树 · 递归遍历四件套

真题年份 | 2014/2017/2022/2026 | 树的大题 100% 落在递归或遍历上，四件套是唯一入口

必背结论

- 四件套：结点定义（照抄题干）→递归出口（if (T == NULL) return;永远第一句）→单层做什么（访问、累加、判断）→递归左右子树的顺序（决定先序、中序、后序）。
- 先序、中序、后序只差「访问根」那一行放在两次递归之前、之间、之后。
- 递归函数三种形态：void 加全局量（最省事，考场首选）、返回 int（求和、计数、高度）、返回结点指针（查找、插入）。
- 全局量就写在函数外面，不用 static，也不用引用参数。
- 先写出口再写函数体：出口写错整题 0 分，而树题出口只有 T == NULL 一种形态。
- 前序适合自顶向下传参（传深度、传路径），后序适合自底向上收集（返回高度、返回 WPL）。

模板/流水线

```
/* 二叉链表结点定义：2014 用 weight、2017 用 char data[10]、2022 用顺序存储，都按题干换 */
typedef struct BiTNode {
    int data;
    struct BiTNode *lchild, *rchild;
} BiTNode, *BiTree;

/* 形态一：void + 全局量 —— 遍历、计数、求和的万能壳 */
int cnt = 0;                              /* 全局量写在函数外 */
void PreOrder(BiTree T) {
    if (T == NULL) return;                /* ① 递归出口永远第一句 */
    cnt++;                                /* ② 单层：访问根（先序放这里） */
    PreOrder(T->lchild);                  /* ③ 左子树 */
    PreOrder(T->rchild);                  /* ④ 右子树 */
```

```
}
/* 中序：把 cnt++ 挪到两句递归之间；后序：挪到最后。三行位置决定一切 */

/* 形态二：返回 int —— 结点数、高度 */
int Count(BiTree T) {
    if (T == NULL) return 0;
    return Count(T->lchild) + Count(T->rchild) + 1;
}
int Height(BiTree T) {
    if (T == NULL) return 0;              /* 空树高 0，单结点高 1 */
    int l = Height(T->lchild), r = Height(T->rchild);
    return (l > r ? l : r) + 1;
}
```

复杂度：时间 $O(n)$，每个结点访问一次；空间 $O(h)$，递归栈深度等于树高，最坏 $O(n)$。

递归出口写成 `if (T->lchild == NULL)` 当出口，叶子结点会漏 · 全局量定义在函数内，每层递归都被重置成 0 · 高度空树返回 0 还是 -1 必须全局统一，混用会让所有高度整体差 1

树 · 带权路径长度 WPL

真题年份 | 2014（13 分） | 树题里代码最短、最好背的一道，递归不到十行就满分

必背结论

- 定义：WPL 是所有叶结点的「权值×深度」之和；根深度取 0 还是 1 都行，但全文必须一致（取 0 时单结点树 WPL 为 0）。
- 叶子判定：`T->left == NULL && T->right == NULL`，两个都要判，只判一个会把单分支结点当叶子。
- 递归参数必须多带一个 depth：往下传 `depth + 1`，回溯时不用减，这就是「递归自带路径」。
- 非叶结点不贡献权值，只贡献深度。
- 用层次遍历按层号算 WPL 能得到同一结果，两个答案互验，用来检查递归写没写错。
- 左右子树的递归顺序不影响结果，先左后右或先右后左都行，不必纠结。

模板/流水线

```
/* 2014：结点 weight 存权值，求 WPL */
typedef struct BiTNode {
    int weight;
    struct BiTNode *left, *right;
} BiTNode, *BiTree;

int wpl(BiTree T, int depth) {
    if (T == NULL) return 0;                          /* 空树不贡献 */
    if (T->left == NULL && T->right == NULL)          /* 注：叶子判定要左右都空 */
        return T->weight * depth;                     /* 权值 × 深度 */
    return wpl(T->left, depth + 1) + wpl(T->right, depth + 1);
}
int WPL(BiTree T) { return wpl(T, 0); }               /* 根深度取 0 */
```

复杂度：时间 $O(n)$，每个结点恰好进一次；空间 $O(h)$ 递归栈。

只判 `T->left == NULL` 就当叶子，单分支结点被误算 · 根深度取 1 却在公式里又减 1，结果差一个常数 · 把非叶结点的 weight 也累加进去（题干明说只有叶结点存非负权值）

树 · 中序遍历转中缀

真题年份 | 2017（15 分） | 遍历加括号的综合题，考的是中序框架不是新算法

必背结论

- 中序序列天然就是中缀表达式的顺序：左子树→根（运算符）→右子树。
- 括号规则：根那一层不加；只要不是根、且子树里还有运算符，整棵子表达式外面套一对括号。

- 叶结点是操作数，直接输出，绝不加括号。
- 递归参数带 depth（层号），靠 depth 判断「是不是根」，比另设标志位干净。
- 结点 data 是 char[10]，用%s 输出，不是%c。
- 单目运算符（负号）只有一个孩子，只输出一侧，括号仍要成套；左右孩子都空才是操作数。
- 反向题（中缀转表达式树）是同一套栈操作：操作数建叶结点入栈，运算符弹两棵子树做左右孩子。

模板/流水线

```
/* 2017：表达式树转等价中缀表达式，结点 data 是 char[10] */
typedef struct node {
    char data[10];
    struct node *left, *right;
} BTree;

void InOrder(BTree *T, int depth) {        /* 首次调用传 depth = 0 */
    if (T == NULL) return;
    if (T->left == NULL && T->right == NULL) {   /* 叶子：操作数，直接输出 */
        printf("%s", T->data);
        return;
    }
    if (depth > 0) printf("(");                  /* 注：非根的子表达式整体加括号 */
    InOrder(T->left, depth + 1);                 /* 左子树 */
    printf("%s", T->data);                       /* 根：运算符放中间 */
    InOrder(T->right, depth + 1);                /* 右子树 */
    if (depth > 0) printf(")");                  /* 与左括号配对 */
}
/* 调用：InOrder(T, 0) —— 根这一层不加括号 */
```

复杂度：时间 $O(n)$，每个结点输出一次；空间 $O(h)$ 递归栈。

给根也加括号，输出最外层多一对 · 给叶子也加括号，操作数变成 (a) · 用%c 输出 char[10]，只打出首字符 · 括号只加左边忘了右边，左右不配对

树 · 二叉搜索树判定

真题年份 | 2022（13 分） | 顺序存储加 BST 判定的组合，坑全在下标和 -1 上

必背结论

- 顺序存储下标：根为 0，下标 i 的左孩子 2i+1、右孩子 2i+2；i 大于等于 ElemNum 即越界（空）。
- -1 表示空结点：判值之前必须先判不等于 -1，否则 -1 会当成合法值参与比较。
- 判定等价于「中序遍历序列严格递增」：递归中用全局 pre 记住前驱值，当前值小于等于 pre 就返回 false。
- 权值为正整数，pre 初值取 -1 就够（若权值可能为 0，初值要取更小的负数）。
- 不要用「左孩子小于根、根小于右孩子」的局部判定，那只保证局部有序，不保证整棵是 BST，这是经典错法。
- 另一条等价路线：先中序遍历把非 -1 的值存进数组，再判数组严格递增；时间仍 $O(n)$、空间 $O(n)$，比全局 pre 好写但多花空间。

模板/流水线

```
/* 2022：顺序存储的二叉树，SqBiTNode[] 存值，-1 表示空，判是否 BST
   MAX_SIZE 题干已定义为常量，这里不再重复定义 */
typedef struct {
    int SqBiTNode[MAX_SIZE];
    int ElemNum;                           /* 实际占用的数组元素个数 */
} SqBiTree;

int pre = -1;                              /* 中序前驱的值；权值为正整数，初值 -1 可用 */
int JudgeBST(SqBiTree T, int i) {
    if (i >= T.ElemNum) return 1;                  /* 越界当作空树，是 BST */
    if (!JudgeBST(T, 2 * i + 1)) return 0;         /* 左子树不是就整体不是 */
    if (T.SqBiTNode[i] != -1) {                    /* 注：-1 是空结点，必须跳过 */
        if (T.SqBiTNode[i] <= pre) return 0;       /* 中序必须严格递增 */
        pre = T.SqBiTNode[i];
```

```
    }
    return JudgeBST(T, 2 * i + 2);                /* 右子树 */
}
/* 调用：pre = -1; JudgeBST(T, 0) */
```

复杂度：时间 $O(n)$，每个数组元素访问一次；空间 $O(h)$，递归深度不超过树高。

忘判不等于 -1，-1 被当值比较导致误判 · pre 用局部变量，每次递归重置，判定永远通过 · 判成大于等于而不是严格大于，重复值被当成合法 BST · 下标写成 2i、2i+1（那是 1 起编号的写法）

树 · 层次遍历

真题年份 | 选择常考，大题未考 | 2014 的 WPL 题可以用它替代递归，求宽度、判完全二叉树也靠这一层队列

必背结论

- 队列用数组实现：front、rear 两个下标，入队 rear 加一，出队 front 加一，容量取 MAX_SIZE 或 100。
- 循环条件是 `front < rear`（队非空）；队空就是 front 等于 rear，入队前先确认数组没满。
- 出队时才判叶子、才累加；入队时判空，空孩子不入队。
- 想按层算 WPL：每轮先记 `cnt = rear - front`（本层结点数），内层循环正好处理一层，depth 在外层加一。
- 数组队列不能循环复用，rear 一路只增不减，容量必须按最多结点数给（满二叉树约 n）。
- 判完全二叉树也用它：一旦出队遇到空孩子，此后出队的结点必须全是叶子，否则不是完全二叉树。

模板/流水线

```
/* 层次遍历 + 按层求 WPL（非递归版） */
typedef struct BiTNode {
    int weight;
    struct BiTNode *lchild, *rchild;
} BiTNode, *BiTree;

int WPL_Level(BiTree T) {
    BiTree q[100];
    int front = 0, rear = 0, depth = 0, sum = 0, i;
    if (T == NULL) return 0;
    q[rear++] = T;                              /* 根入队 */
    while (front < rear) {
        int cnt = rear - front;                 /* 注：本层结点数 */
        for (i = 0; i < cnt; i++) {
            BiTree p = q[front++];              /* 出队 */
            if (p->lchild == NULL && p->rchild == NULL)
                sum += p->weight * depth;       /* 叶子：权值 × 当前层号 */
            if (p->lchild) q[rear++] = p->lchild;
            if (p->rchild) q[rear++] = p->rchild;
        }
        depth++;                                /* 本层处理完，层号加一 */
    }
    return sum;
}
```

复杂度：时间 $O(n)$；空间 $O(n)$，队列最多存一层结点（满二叉树最宽层约 n/2）。

队列容量开小了，满二叉树最后一层就溢出 · depth 加一的位置写错（写进内层 for 或干脆忘了），同一层叶子被算成不同深度 · 出队前不判空、空孩子也入队，rear 一路冲出数组

树 · 线索二叉树

真题年份 | 2026 | 最新一年考「遍历与线索二叉树」，2027 极可能延续

必背结论

- 线索化的本质：把 n+1 个空指针域改成指向前驱、后继的线索，用 ltag、rtag 区分「孩子」和「线索」，1 表示线索。

- 中序线索化就是中序遍历，在「访问根」的位置补两件事：左空则建前驱线索，前驱右空则建后继线索，然后 pre 后移。
- 必须有一个全局 pre 指向「刚访问过的结点」，初值 NULL。
- 中序后继：rtag 等于 1 直接取 rchild；rtag 等于 0 则是右子树中最左下的结点。
- 线索化收尾要给中序最后一个结点补 `rtag = 1`，否则它的 rtag 仍是 0、rchild 又是空，求后继时解引用空指针。
- 线索化之后中序遍历不用栈、不用递归，顺着 rchild 一路走，空间 $O(1)$。
- 中序前驱对称：ltag 等于 1 直接取 lchild；ltag 等于 0 则是左子树中最右下的结点。

模板/流水线

```
/* 中序线索二叉树：ltag=1 表示 lchild 存前驱，rtag=1 表示 rchild 存后继 */
typedef struct ThreadNode {
    int data;
    struct ThreadNode *lchild, *rchild;
    int ltag, rtag;                         /* 建树时把两者都置 0 */
} ThreadNode, *ThreadTree;

ThreadNode *pre = NULL;                     /* 全局：刚访问过的结点 */

void InThread(ThreadTree p) {
    if (p == NULL) return;
    InThread(p->lchild);                    /* 左子树线索化 */
    if (p->lchild == NULL) {                /* 左空 → 建前驱线索 */
        p->lchild = pre;  p->ltag = 1;
    }
    if (pre != NULL && pre->rchild == NULL) {   /* 前驱右空 → 建后继线索 */
        pre->rchild = p;  pre->rtag = 1;
    }
    pre = p;                                /* 注：访问完就后移 */
    InThread(p->rchild);                    /* 右子树线索化 */
}

/* 线索化收尾：中序最后一个结点没有后继，必须显式把 rtag 置 1，否则遍历到它就崩 */
void CreateInThread(ThreadTree T) {
    pre = NULL;
    if (T != NULL) {
        InThread(T);
        pre->rchild = NULL;
        pre->rtag = 1;
    }
}

/* 中序第一个结点：从根一路向左 */
ThreadNode *FirstNode(ThreadNode *p) {
    while (p != NULL && p->ltag == 0 && p->lchild != NULL) p = p->lchild;
    return p;
}
/* 中序后继：rtag=1 直接取 rchild，否则是右子树的最左下结点 */
ThreadNode *NextNode(ThreadNode *p) {
    if (p->rtag == 1) return p->rchild;
    return FirstNode(p->rchild);
}
/* 线索树上不用栈、不用递归的中序遍历 */
void InOrderThread(ThreadTree T) {
    ThreadNode *p;
    if (T == NULL) return;
    for (p = FirstNode(T); p != NULL; p = NextNode(p))
        printf("%d ", p->data);
}
```

复杂度：线索化时间 $O(n)$、空间 $O(h)$ 递归栈；线索化后遍历时间 $O(n)$、空间 $O(1)$。

建树时忘把 ltag、rtag 置 0，后续判断全反 · pre 没在「访问根」之后后移，后继线索全部指错 · 求后继时只看 rtag 不看 rchild 是否为空，最后一个结点会跳到 NULL 之后 · 把线索当孩子去递归，遍历进入死循环

哈夫曼树构造流水线

真题年份 | 2012（哈夫曼树的应用） / 2020（哈夫曼编码） | 树章唯一连考两次的应用大题，几乎每年还配 1 道选择，大题 8-13 分

必背结论

- 每次从集合中取【两个最小】的权值合并，新结点【必须放回集合】再参与比较，重复 $n-1$ 次，最后只剩一个根。
- n 个叶子的哈夫曼树共 $2n-1$ 个结点、$n-1$ 个非叶结点；不存在度为 1 的结点，初始结点最终全部成为叶子。
- WPL $=\sum w_i d_i$，d_i 是第 i 个叶子到根的边数；另一条路是 WPL $=\sum$（所有非叶结点的权值）。
- 树形不唯一，但 WPL 唯一，并且每片叶子所在的层高唯一——这是「画不出标准树」时的救命结论。
- 小权放左、大权放右，画出来才和标准答案的编码逐位一致。
- 题目给「频次」「出现次数」就是权值；给「概率」先化成分数或整数再算，别带小数合并。

两条算 WPL 的路都必须会：按定义算一遍、按非叶结点之和再验一遍，两数相等才继续往下做。

模板/流水线

```
step1  把权值集合 W 升序排好，抄在草稿纸最上面一行
step2  取 W 中最小的两个 a、b（a<=b），合并成 a+b，a 画左、b 画右
step3  从 W 中删掉 a、b，把 a+b 按大小插回 W
step4  重复 step2~step3，共 n-1 次，直到 W 只剩一个数（根）
step5  自根向下数每片叶子的边数 d，列成表，WPL = sum(w*d)
step6  验算：WPL = 所有非叶结点的权值之和，两法不等就是合并次序错了
step7  左 0 右 1，从根向下读每个叶子，得到编码，码长 = d
```

五次合并的演算表（权值 3 4 5 6 8 10）：

轮	合并前集合	本次合并	新结点
1	3 4 5 6 8 10	3+4	7
2	5 6 7 8 10	5+6	11
3	7 8 10 11	7+8	15
4	10 11 15	10+11	21
5	15 21	15+21	36

叶子层次表：

叶子权	边数 d	贡献 w*d
3	3	9
4	3	12
5	3	15
6	3	18
8	2	16
10	2	20
合计 36	—	90

▶ 由编码反推哈夫曼树

- 从根出发逐位读：读到 0 往左、读到 1 往右，路上缺结点就新建一个。

- 一条编码读完，就在停下的位置标上该字符，它必须是一片【新叶子】。
- 全部字符都这样落完，把叶子上的权值加起来，树就还原了，再按定义算 WPL。
- 没有选项时先按码长筛：码长为 d 的字符一定在第 d 层，同层的字符个数不能超过 2^d。
- 出现码长为 1 的字符，说明根的两个孩子都是叶子，其余字符的码长必然都大于 1。
- 反推完再验一次：所有叶子权值之和必须等于根的值，不等就是中间少建或多建了结点。

> 合并出新结点忘了放回集合，后面两轮选的最小值就全错，一步错步步错

> WPL 只乘叶子，把非叶结点也乘进去是最常见的失分点

> 画出的树和选项不一致不要慌：先按码长（叶子所在层）筛选项，再用无前缀排除

哈夫曼编码与平均码长

真题年份 | 2012 / 2020 | 大题最常问「加权平均长度」，也常配压缩率、编码总长，8-13 分

必背结论

- 左 0 右 1，每条编码都从根重新读，读到叶子才停；编码长度就是叶子到根的边数。
- 加权平均码长 = WPL ÷ $\sum w_i$ = 总位数 ÷ 总字符数，它 ≠ WPL。
- 定长编码长度 = $\lceil \log_2 n \rceil$，n 是不同字符的个数；定长要求所有叶子在同一层。
- 压缩率 = (定长总长 − 哈夫曼总长) ÷ 定长总长，用百分数写，保留一位小数。
- 前缀编码 $\Longleftrightarrow$ 没有任何一个编码是另一个的前缀 $\Longleftrightarrow$ 所有字符都落在叶子结点上。
- 编码总长 = WPL，所以「总位数」和「WPL」是同一个数，题目换个说法而已。
- 给的是概率或小数时，先同乘一个倍数化成整数再合并，平均码长不会变。

「加权平均长度」问的是一个字符平均占几位，答案一定小于定长码长；WPL 是总位数，两个都要写清楚。

模板/流水线

```
step1  由树读出每个叶子的码长 d（= 所在层，根为第 0 层）
step2  列表：字符 / 频次 w / 码长 d / 贡献 w*d
step3  WPL = sum(w*d)    ← 这就是全部编码的总位数
step4  S = sum(w)        ← 全部字符的总个数
step5  加权平均码长 = WPL / S
step6  n 个字符的定长码长 L = ceil(log2(n))，定长总长 = S*L
step7  压缩率 = (S*L - WPL) / (S*L)
```

演算表（字符 A-F 依次对应权 3 4 5 6 8 10）：

字符	频次	码长	贡献位数
A	3	3	9
B	4	3	12
C	5	3	15
D	6	3	18
E	8	2	16
F	10	2	20
合计	36	平均 2.5	90

压缩率：$n = 6$，$L = \lceil \log_2 6 \rceil = 3$，定长总长 $36 \times 3 = 108$，压缩率 $= (108 - 90) \div 108 = 16.7\%$。

▶ 前缀特性的三种判定情形

- 情形一：某条编码走到中途的非叶结点上就停了，没到叶子 → 当前这条是别人的前缀。
- 情形二：某条编码半路撞上了已有的叶子结点 → 已有那条是当前这条的前缀。
- 情形三：每个字符都新建一片叶子，既不中途停下也不撞旧叶子 → 满足前缀特性。
- 三种情形必须全部写出来，「漏一种就扣分」，这是判定类设问的标准给分点。
- 「哪种结构适合保存前缀编码」标准答案是二叉树，不是哈夫曼树——任何二叉树都能表示前缀编码。

▶ 定长编码集与哈夫曼编码集的区别

- 相同：字符都落在叶子上，都能反映前缀编码。
- 不同：定长要求所有叶子同层，哈夫曼允许叶子在不同层。
- 因此两棵树的结点总数可以不同，高度也可以不同。
- 频次不同的字符可以落在同一层，频次相同也可以落在不同层，别用频次判层数。

▶ 2012 那道「合并 k 个有序表」就是哈夫曼思想

- 长度 m、n 的两个有序表归并，最坏比较 $m+n-1$ 次，最好 $\min\{m,n\}$ 次，时间复杂度 $O(m+n)$。
- 让 k 个表的比较总次数最少 $\Longleftrightarrow$ 每次合并两个最短的表 $\Longleftrightarrow$ 按哈夫曼树合并。
- 总比较次数 = 所有新表长之和 − 合并次数 = WPL− 非叶结点个数。

长度 10 35 40 50 60 200 的演算表：

本次合并	新表长	比较次数
10+35	45	44
40+45	85	84
50+60	110	109
85+110	195	194
195+200	395	394
合计	—	825

验算：$\text{WPL} = (10+35)\times 4 + 40\times 3 + (50+60)\times 3 + 200\times 1 = 830$，非叶结点 5 个，$830-5=825$，两法一致。

> 加权平均长度不是 WPL，忘了除以权值之和整题就错

> 最坏比较次数是 m+n-1，写成 m+n 全错；只有问时间复杂度才写 O(m+n)

> 定长码长取 ⌈log2 n⌉，n 是字符种数，不是字符总个数

Dijkstra 手算填表

真题年份 | 2009（Dijkstra 与贪心策略） / 2014（邻接表 + 最短路径） | 单源最短路径的唯一考法，大题 2 次、选择 3 次

必背结论

- 表的第一行只填「源点直接到达」的边，到不了写 ∞，源点自己写 0。
- 每轮：在【未确定】的距离里挑最小 → 该顶点确定；再拿它当中转，能到且更小才更新。
- 最后一行即各顶点的最短距离；每轮选中顶点的先后即「选顶点顺序」。
- 选点顺序可事后反推：把最终距离从小到大排序，对应出来的顶点顺序就是选点顺序。
- 实现上是一维数组逐行覆盖，不是二维数组；$O(n^2)$，只与顶点数有关，与边数无关。
- 权值必须非负；有负权边时 Dijkstra 会给出错误结果，这是它「贪心不回退」的必然代价。

- 只求一个源点用 Dijkstra，求任意两点之间才用 Floyd（$O(n^3)$，408 大题没正面考过）。
- 邻接矩阵实现时每轮找最小 $O(n)$、共 n 轮，所以是 $O(n^2)$；只有用堆才降到 $O((n+e)\log n)$。

表是「一行覆盖一行」的一维数组；每轮只做两件事——在没定的里面选最小、用它松弛一圈。

模板/流水线

```
step1  画表：列 = 除源点外的各顶点，行 = 初始化 + 每一轮
step2  初始化：dist[v] = 源点到 v 的直接边权，没有边写 inf，dist[源] = 0
step3  在未确定的 dist 里挑最小值，该顶点 v 本轮确定，记下选点顺序
step4  以 v 为中转：若 dist[v]+w(v,u) < dist[u] 就改 dist[u]，否则保持原值
step5  重复 step3~step4，共 n-1 轮，直到全部顶点确定
step6  最后一行即最短距离；路径按「谁的更新改出了它」一步步回溯中转点
step7  验算：最终距离从小到大排出的顺序 = 选点顺序
```

演算表（1-2=10、1-5=5；5-2=3、5-3=9、5-4=2；4-3=6；2-3=1）：

轮	2 号	3 号	4 号	5 号	选中	关键更新
初始化	10	inf	inf	5	5	只写直达边
经 5 中转	8	14	7	5	4	5+3 改 10；5+2 改 inf；5+9 改 inf
经 4 中转	8	13	7	5	2	7+6 改 14；4 到不了 2
经 2 中转	8	9	7	5	3	8+1 改 13
最终	8	9	7	5	—	顺序 5 4 2 3

路径：到 2 是 1-5-2（8）；到 3 是 1-5-2-3（9）；到 4 是 1-5-4（7）。

▶ 四种问法的对应答法

- 求最短距离：直接抄最后一行，$n-1$ 个数一个不少。
- 求选点顺序：抄每轮「选中」那一列的先后，或用最终距离升序反推。
- 求具体路径：按「谁改出了这个 dist」回溯，写成 1-5-2 这样的顶点序列，不要只写长度。
- 问中间某一步的数值：只能逐行往下推，肉眼法在这一问完全失效。

> 初始化只能写直接边，写进「预判的最短值」会让整张表失去意义

> 题目问「最短路径及费用」时路径和长度都要写，只写距离要扣分

> 只有更小才更新，相等保持原值；更新条件写成「小于等于」会写出多余的中转

> 有负权边不能用 Dijkstra，这是算法适用条件，也是简答题的常见设问

Prim 与 Kruskal 手算

真题年份 | 2017（最小生成树 Prim） / 2018（最小生成树，包装成「通信网最经济方案」并跨考 TTL） | 连续两年大题

必背结论

- Prim：任选起点，每轮在「已选顶点 → 未选顶点」的边里取最小，把该顶点并入集合；$O(n^2)$，只与顶点有关 → 稠密图。
- Kruskal：全部边按权升序，依次取，成环就跳过，取满 $n-1$ 条停；$O(e\log e)$，只与边有关 → 稀疏图。
- n 个顶点的生成树恰有 $n-1$ 条边；MST 形态不唯一，但权值之和唯一。
- 唯一性的充分条件是「所有边权互不相同」；某一步有多种备选，并不等于整棵树不唯一。
- 题面出现「所有顶点连通」+「代价最小 / 最经济 / 最省钱」→ 就是最小生成树。
- 没指定算法就用 Kruskal；但两个算法的名字、流程、适用图都要能默写，考题问过「算法叫什么」。

- Prim 的 $O(n^2)$ 来自「$n-1$ 轮、每轮扫 n 个候选」；Kruskal 的 $O(e\log e)$ 瓶颈在边排序，所以只与边数有关。

Prim 是「从一个点往外长」，Kruskal 是「从小到大捡边、用并查集防环」，两句话定选型。

模板/流水线

```
/* Prim：给点选边 */
step1  任取起点 v0 入已选集 S，S = {v0}
step2  在「一端在 S、一端不在 S」的所有边里取权最小的 e
step3  把 e 的另一端点并入 S，e 入选边集，记下选边顺序
step4  只更新与新并入顶点相连的候选边，重复 step2~step3 共 n-1 次
step5  选够 n-1 条边即得 MST，权值相加

/* Kruskal：给边选点 */
step1  列出全部边，按权升序排成边集表
step2  依次取最小边，用并查集判断两端是否已连通
step3  不连通就选（合并两个集合），已连通就跳过（成环）
step4  取满 n-1 条立刻停，剩下的边不用再看
step5  权值相加得总代价；题意含「所有可能」时换同样权值的边再画一次
```

Prim 演算表（A 起点；边 AB=1、AC=3、BC=2、BD=4、CD=5）：

轮	已选顶点	候选边（权）	选中	新顶点
1	A	AB 1、AC 3	AB	B
2	A B	AC 3、BC 2、BD 4	BC	C
3	A B C	AC 3（两端都已选，不计）、BD 4、CD 5	BD	D

Kruskal 边集表：

序	边	权	取舍	理由
1	AB	1	选	两端不连通
2	BC	2	选	两端不连通
3	AC	3	弃	A、C 已连通，成环
4	BD	4	选	不连通，已满 3 条即停
5	CD	5	不看	已取满 n-1 条

两法都得总权 $1+2+4=7$，互为验算。

▶ 包装题与唯一性判定

- 「连通所有城市」+「线路总费用最小」→ 最小生成树；先用一句话把题面翻译成图论问题再动手。
- Prim 题问「选边顺序」时，边和并入的顶点都要写，只写边会漏分。
- 判唯一：把整棵树画出来；或看同权值的边能否互换后仍连通且不成环。
- 问「什么条件下 MST 唯一」答的是【充分条件】，写「所有边权互不相同」即可得分。

> 取满 n-1 条必须停，多选一条就是回路，边数写成 n 也是常见错

> Kruskal 不是「权值小就一定选」，第三个必弃，防环只能靠并查集

> 「有多种备选」不代表不唯一，要把整棵树画出来才能下结论

> 题意含「所有可能」时只画一种方案，会直接扣掉一半分

拓扑排序与 AOE 网

真题年份 | 2025（AOE 网） / 2024（拓扑排序，问唯一性） | 关键路径大题只 1 次但选择多次；拓扑排序约 4 年一遇

必背结论

- 拓扑排序：反复取入度为 0 的顶点输出，删除它和以它为尾的边，其直接后继入度减 1。
- 逆拓扑：反复取出度为 0 的顶点；也可先求拓扑序列再整个倒过来。
- 有向图存在拓扑序列 $\Longleftrightarrow$ 图中无环（DAG）；有环时答案是「不存在」，不是随便排一个。
- AOV 网顶点表示活动；AOE 网边表示活动、顶点表示事件，源点入度 0、汇点出度 0，且各只有一个。
- 关键路径是从源点到汇点【路径长度最大】的那条，不是最短，这与 Dijkstra 正好相反。
- $ve(j)=\max\{ve(i)+w_{ij}\}$（拓扑正推取 max）；$vl(i)=\min\{vl(j)-w_{ij}\}$（逆拓扑倒推取 min）。
- 活动最早 $e=ve$(前驱)；活动最晚 $l=vl$(后继)$-w$；余量 $d=l-e$，$d=0$ 为关键活动。
- 校验：vl(源点)$=0$，ve(汇点)$=vl$(汇点)，汇点的两个时间一定相等。
- 关键路径的长度就是 ve(汇点)，题目让你「计算工期」要的就是这个数。
- 逆拓扑序列可能不唯一，但每个顶点的 vl 值唯一；把拓扑序列整个倒过来是最省事的求法。

全部时间只有「发生」和「开始」两种叫法，没有「结束时间」，答题用词照这个写才不会扣表述分。

模板/流水线

```
step1  按拓扑序正推 ve: ve(源)=0, ve(j) = max{ ve(i) + w(i,j) }
step2  按逆拓扑序倒推 vl: vl(汇)=ve(汇), vl(i) = min{ vl(j) - w(i,j) }
step3  对每条边求活动最早 e = 该边前驱顶点的 ve
step4  求活动最晚 l = 该边后继顶点的 vl - 该边权值
step5  求余量 d = l - e, d = 0 的边即关键活动, 串起来就是关键路径
step6  校验 vl(源) = 0; 关键路径长度 = ve(汇)
```

顶点表（v1-2=3、v1-3=2、v2-4=2、v2-5=3、v3-4=4、v3-6=3、v4-6=2、v5-6=1）：

顶点	v1	v2	v3	v4	v5	v6
ve	0	3	2	6	6	8
vl	0	4	2	6	7	8

$ve(v4)=\max\{3+2,2+4\}=6$，$ve(v6)=\max\{6+2,6+1,2+3\}=8$；$vl(v3)=\min\{6-4,8-3\}=2$，$vl(v2)=\min\{7-3,6-2\}=4$。

活动表：

活动	边	时长	e	l	余量
A1	v1-v2	3	0	1	1
A2	v1-v3	2	0	0	0
A3	v2-v4	2	3	4	1
A4	v2-v5	3	3	4	1
A5	v3-v4	4	2	2	0
A6	v3-v6	3	2	5	3
A7	v4-v6	2	6	6	0
A8	v5-v6	1	6	7	1

余量为 0 的三条边 A2、A5、A7 连成关键路径 v1-v3-v4-v6，长度 $2+4+2=8$。

▶ 拓扑排序唯一性判定

- 第一步先把每个顶点的入度求出来，写成一行数组。
- 入度为 0 的顶点入队；初始就同时有多个入度为 0 的，直接判不唯一。
- 出队即删除该顶点，把它的直接后继入度减 1，减完立刻判断是否变成 0，是就入队。
- 过程中只要有【一次】同时入队两个及以上，就不唯一；全程队列里只有一个才唯一。

▶ 多条关键路径

- 若算出一堆边「最早 = 最晚」，不是算错，而是存在多条关键路径。
- 想缩短总工期，必须把每条关键路径都缩短；只缩一条，最长的那条没变，工期一天都不减。
- 判断该缩短哪条活动：这条活动必须同时落在所有关键路径上。

> ve 取 max、vl 取 min，写反整题作废

> 活动最晚开始是「后继 vl − 持续时间」，直接写后继的 vl 就错了

> 源点 vl 必须等于 0，这是全题唯一能自查的校验位，算完一定回头看一眼

> 多条关键路径时只缩短一条，工期一秒都不会变

邻接矩阵幂与度计算

真题年份 | 2015（邻接矩阵幂运算的含义） / 2014（邻接表） / 2021、2023（度统计） | 幂运算常只考「含义」一问，4-5 分

必背结论

- $A^n[i][j]$ = 从顶点 i 到 j 长度为 n 的【路径条数】；$n=1$ 时就是边数（0 或 1）。
- 无向图 A 对称，A^n 也对称；有向图一般不对称，这是快速自查的抓手。
- $\sum_i A^n[i][i]$ = 长度为 n 的回路条数，对角元不为 0 就说明有环。
- 无向图：度 = 第 i 行非零元个数 = 邻接表第 i 个链表的长度；$\sum \deg = 2e$。
- 有向图：出度 = 行和，入度 = 列和，出度之和与入度之和都等于 e。
- 邻接矩阵空间 $O(n^2)$ 适合稠密图；邻接表空间 $O(n+e)$ 适合稀疏图，无向图存 $2e$ 个边结点、有向图存 e 个。
- 用邻接矩阵判断两点是否相邻是 $O(1)$，用邻接表要扫链表、平均 $O(\text{度})$。
- 有向图第 i 行 1 的个数是出度、第 i 列 1 的个数是入度，两者绝不能互相顶替。

A 的一次幂回答「有没有边」，n 次幂回答「有几种走法」——问条数就答数字，不要答「有 / 无」。

模板/流水线

```
step1  写出 A: n 行 n 列, A[i][j]=1 表示 i 到 j 有一条边
step2  手算 A^n 用「行乘列」: A^n[i][j] = sum_k A^(n-1)[i][k] * A[k][j]
step3  只算题目问的那一行或那一列, 不要整张矩阵都乘
step4  度: 无向图数行内 1 的个数; 有向图行和 = 出度、列和 = 入度
step5  邻接表: 把每行的 1 写成链表, 链表长度就是度
step6  验算: 无向图 sum(度) = 2e; 有向图 sum(出度) = sum(入度) = e
```

矩阵幂演算（1-2、1-3、2-3、3-1 四条有向边）：

矩阵	1 列	2 列	3 列
A 第 1 行	0	1	1
A 第 2 行	0	0	1
A 第 3 行	1	0	0

A2 的行	结果	对应路径
第 1 行	1 0 1	1-3-1（回到 1）、1-2-3
第 2 行	1 0 0	2-3-1
第 3 行	0 1 1	3-1-2、3-1-3

全部元素之和为 5，即长度 2 的路径共 5 条；对角元之和为 $1+0+1=2$，即长度 2 的回路 2 条。

无向图（边 1-2、1-3、2-3、2-4）的邻接表与度：

顶点	邻接链表	度
1	2 3	2
2	1 3 4	3
3	1 2	2
4	2	1

$\sum \deg = 2+3+2+1 = 8 = 2e$，得 $e=4$，与边数一致。

▶ 四种存储结构各一句话

- 邻接矩阵：$O(n^2)$ 空间，判相邻 $O(1)$，稠密图首选，还能按一维数组压缩存储。
- 邻接表：$O(n+e)$ 空间，求度快、遍历省时，判相邻要扫链表，稀疏图首选。
- 十字链表：有向图专用，出边和入边都能顺着链快速找。
- 邻接多重表：无向图专用，每条边只存一个结点，省一半空间。

> A^n 是「路径条数」不是「是否可达」，把 2 写成 1 就白算

> 只问某一行时别整矩阵都乘，手算规模越大越容易错

> 无向图邻接表存 2e 个边结点、有向图存 e 个，题里问存储量最容易翻车

> 邻接矩阵题给一维数组时要想到「压缩存储」，别以为条件不够

图的应用速查表

真题年份 | 2009 / 2014 / 2015 / 2017 / 2018 / 2025（图的应用几乎年年是第二道大题） | 考前最后五分钟只看这一张

必背结论

- 「连通所有点 + 代价最小」→ 最小生成树：稠密用 Prim，稀疏用 Kruskal。
- 「一个源点到其余各点 + 距离最小」→ Dijkstra，$O(n^2)$，要求非负权。
- 「任意两点之间 + 最短」→ Floyd，$O(n^3)$，408 大题从未正面考过。
- 「任务先后约束 / 能否排完」→ 拓扑排序，有环即无解。
- 「工期、最早 / 最迟、关键活动」→ AOE 网关键路径，$O(n+e)$。
- 「分成几块 / 连通分量、度统计」→ DFS 或并查集，邻接矩阵上 $O(n^2)$、邻接表上 $O(n+e)$。
- 这一章的题几乎都能靠「画图 + 填表」拿分，最忌讳空着不写过程直接给答案。

判断考点只花五秒：看题目问的是「连起来省钱」「走最短」「排先后」还是「工期最长」。

模板/流水线

```
step1  读题抓关键词：连通/最经济 -> MST；最短/费用 -> 最短路
step2  数清规模：稠密图优先 Prim，稀疏图优先 Kruskal，点少边少的直接手算
step3  定起点：MST 任取，Dijkstra 用题给源点，AOE 用入度为 0 的源点
```

```
step4  按对应模板逐轮填表，每轮只写变化的数
step5  收尾必做校验：MST 看边数 = n-1，Dijkstra 看选点个数 = n-1，AOE 看 vl(源)=0
```

速查表：

问题	算法	求什么	复杂度	真题
最小生成树	Prim	边权和最小的生成树	O(n²)	2017
最小生成树	Kruskal	同上	O(e log e)	2018
单源最短路	Dijkstra	源点到各点的最短距离	O(n²)	2009、2014
多源最短路	Floyd	任意两点的最短距离	O(n³)	大题未考
拓扑排序	入度法	满足先后约束的线性序列	O(n+e)	2024
关键路径	AOE 五步	最大路径长度与关键活动	O(n+e)	2025
连通分量	DFS 并查集	分成几块	O(n+e)	2021
度统计	矩阵或表	出度、入度、度	O(n²) 或 O(n+e)	2023

▶ 交卷前三句校验

- 最小生成树：边数必须等于 $n-1$，权值和唯一，画两张图时两图边数相同。
- 最短路径：选点顺序的个数等于 $n-1$（不含源点），数不够就是漏了一轮。
- 关键路径：$vl(\text{源})=0$、$ve(\text{汇})=vl(\text{汇})$，两个等式不全就是 max/min 写反了。

> 关键路径求最大、最短路求最小，两个「路径」千万别混

> Floyd 的 n 是顶点数，三重循环，别把边数代进去

> 拓扑排序遇到环，正确答法是「无拓扑序列」，不是随便排一个

排序总表（必背）

真题年份｜选择常考，大题未考｜这张表是排序全部选择题的出处；服务 2022 堆排序、2023 外部排序两道应用题

必背结论

- 稳定的 5 个：直接插入、折半插入、冒泡、二路归并、基数排序；不稳定的 4 个：希尔、快速、简单选择、堆（口诀「快些选堆」）。
- 与初始序列无关的 4 个：简单选择、堆、归并、基数——比较次数或总时间与数据初态无关，正序逆序一样慢。
- 平均 $O(n\log n)$ 而最坏掉到 $O(n^2)$ 的只有快速排序；最好、平均、最坏三态同阶的只有堆、归并、基数、简单选择。
- 空间不是 $O(1)$ 的只有 3 个：快排 $O(\log n)$ 到 $O(n)$（递归栈）、归并 $O(n)$、基数 $O(r)$。
- 比较次数恒为 $n(n-1)/2$ 的是简单选择；移动次数至多 $n-1$ 次的也是它。
- 必须用数组实现：希尔、快速、堆（不相邻交换、要由孩子找父结点）；数组链表都行：直接插入、冒泡。
- 同为最好 $O(n)$ 但走法不同：冒泡是「1 趟乘 $n-1$ 次比较」，直接插入是「$n-1$ 趟各 1 次比较」。
- 基数排序的 d 是关键字位数（三位数 $d=3$），r 是每位的基数（十进制 $r=10$），两者别混。

模板/流水线

排序	最好	平均	最坏	空间	稳定	与初始序列有关
直接插入	O(n)	O(n^2)	O(n^2)	O(1)	稳定	有关

排序	最好	平均	最坏	空间	稳定	与初始序列有关
折半插入	O(n)	O(n^2)	O(n^2)	O(1)	稳定	仅比较有关
希尔	O(n^1.3)	O(n^1.3)	O(n^2)	O(1)	不稳定	有关
冒泡	O(n)	O(n^2)	O(n^2)	O(1)	稳定	有关
快速	O(nlogn)	O(nlogn)	O(n^2)	O(logn)	不稳定	有关
简单选择	O(n^2)	O(n^2)	O(n^2)	O(1)	不稳定	无关
堆排序	O(nlogn)	O(nlogn)	O(nlogn)	O(1)	不稳定	无关
二路归并	O(nlogn)	O(nlogn)	O(nlogn)	O(n)	稳定	无关
基数	O(d(n+r))	O(d(n+r))	O(d(n+r))	O(r)	稳定	无关

四问四答就能选：稳定吗、空间是 1 吗、三态同不同阶、受不受初态影响。表里九行背到能默写，排序选择题一题不失。

> 快排空间不是 O(1)，写 O(logn) 才是标准答案，写 O(nlogn) 会被判错 · 堆排序三态都是 O(nlogn)，别顺手写成「最坏 O(n^2)」· 折半插入只减比较次数、不减移动次数，总时间仍 O(n^2)

一趟结果的判定

真题年份 | 选择常考，大题未考 | 排序选择题出现最多的一类反推题；服务 2016 划分结果的判定

必背结论

- 直接插入：第 k 趟后前 $k+1$ 个有序，且第 $k+2$ 位起与原序列逐位相同——后缀原封不动是它唯一的指纹。
- 折半插入：一趟结果与直接插入完全一样，序列分不出来，只能靠「比较次数变少」认。
- 冒泡：第 k 趟后末尾（大数下沉）或开头（小数上浮）连续 k 个就位，其余部分被打乱。
- 简单选择：第 k 趟后前 k 个是全局最小的 k 个且升序，看不出「逐步插入」的痕迹。
- 堆排序：第 k 趟后末尾 k 个升序就位，前面 $n-k$ 个仍是堆序而不是有序。
- 快排：一趟只定一个元素（枢轴），且它左边全不大于它、右边全不小于它；若一趟后已就位两个以上且分处两端，直接排除快排。
- 归并：第 k 趟后每 2^k 个一段内部有序（段长 1、2、4、8 倍增），段间无序。
- 希尔：每隔 d 个有序；反推 d 就看有序元素之间的最小下标差。
- 基数：按某一位整体有序、低位优先，看个位还是十位已经全序，就知道做了几趟。

模板/流水线

1. 先数「已站到最终位置」的元素个数：0 个→归并、希尔、基数、直接插入；恰好 1 个在中间→快排。
2. 就位元素在两端连成连续一块：块在后→冒泡或堆；块在前→简单选择。
3. 直接插入与冒泡、选择的硬分界：直接插入的后缀与原序列逐位相同，另外两个的后缀被打乱。
4. 有序段长度按 1、2、4、8 倍增→归并；每隔 d 个有序→希尔。
5. 冒泡与堆都让末尾就位，区别在未就位段：冒泡段内无序但有交换痕迹，堆的未就位段是标准大根堆（首位是剩余最大值）。

```
#include <stdio.h>
#define N 7
/* p 是" 某一趟后" 的序列: 存在 k 使 p[0..k-1] 升序且 p[k..n-1] 与原序列逐位相同 -> 插入类 */
int PrefixSorted(int p[], int k) {
    int i;
    for (i = 1; i < k; i++) if (p[i - 1] > p[i]) return 0;
```

```
    return 1;
}
int SuffixSame(int a[], int p[], int n, int from) {
    int i;
    for (i = from; i < n; i++) if (a[i] != p[i]) return 0;
    return 1;
}
int IsInsertPass(int a[], int p[], int n) {
    int k;
    for (k = 2; k <= n; k++)                /* 第 1 趟后前 2 个有序，第 k 趟后前 k+1 个 */
        if (PrefixSorted(p, k) && SuffixSame(a, p, n, k)) return k;
    return -1;                              /* 不是插入类 */
}
```

「第二趟后前 2 个有序」是错的：第一趟才前 2 个，第二趟是前 3 个，选项里出现这种说法就是设给直接插入的陷阱 · 快排第二趟定义是左右两边各划分一次，不是左边一趟右边一趟 · 判定题先找有没有单个元素「左全小右全大」，这是快排的独家签名

快排划分模板

真题年份 | 2016 | 数据结构算法题里唯一以排序为外壳的题，划分模板值 8 分

必背结论

- 一趟划分 $O(n)$、空间 $O(1)$、不稳定；整体快排平均 $O(n\log n)$、最坏 $O(n^2)$、空间 $O(\log n)$ 到 $O(n)$。
- 枢轴取首元素就先 `j--`，取末元素就先 `i++`：方向写反会覆盖还没用到的数据。
- 一趟划分只保证「左边全部不大于枢轴、右边全部不小于枢轴」，两侧内部仍无序。
- 2016 的真身是快速选择：找第 n_1 小（$n_1=\lceil n/2\rceil$），枢轴落到下标 n_1-1 时划分即达标，平均 $O(n)$、空间 $O(1)$。
- 只递归「含 n_1 的那一侧」，另一侧整块丢弃——这就是从 $O(n\log n)$ 降到 $O(n)$ 的唯一原因。
- 划分的副产品：枢轴下标 p 就是「左边有几个比它小」，是第 k 小问题和大题里定位切点的通用工具。
- 每趟比较次数只与枢轴比，总比较约 $n\log n$；移动只发生在交换处，划分越不均匀趟数越多。
- 最坏退化：原序列已升序且取首元素做枢轴，每趟只能固定一个元素，比较次数为 $n+(n-1)+\cdots+1$。

模板/流水线

```
#include <stdio.h>
#define N 9
/* 一趟划分：枢轴取 a[l]，返回枢轴最终下标；左 <= 枢轴 <= 右 */
int Partition(int a[], int l, int r) {
    int pivot = a[l], i = l, j = r;             /* 枢轴先取出，l 处成为空位 */
    while (i < j) {
        while (i < j && a[j] >= pivot) j--;     /* 先右后左，方向不能反 */
        a[i] = a[j];
        while (i < j && a[i] <= pivot) i++;
        a[j] = a[i];
    }
    a[i] = pivot;                               /* 相遇处填回枢轴 */
    return i;
}
/* 2016：划分成 A1、A2，A1 有 n1 个且全不大于 A2（题干要求 n1<=n2 时把 k 改成 n/2） */
int Split(int a[], int n) {
    int k = (n + 1) / 2, l = 0, r = n - 1, p;
    while (l < r) {
        p = Partition(a, l, r);
        if (p == k - 1) return p;               /* 枢轴正好落在分界线上，收工 */
        if (p < k - 1) l = p + 1;               /* 只在含分界点的一侧继续 */
        else r = p - 1;
    }
    return l;
}
```

设计思想第一句照抄：「采用快速排序的划分思想，以首元素为枢轴做一趟划分，枢轴落位即把数组分成不大于与不小于两半；只在含目标位置的一侧继续划分，平均时间 $O(n)$，空间 $O(1)$。」

枢轴必须用临时变量存住，直接拿 a[l] 当空位最后填不回原值 · Split 两侧改成 while 迭代，写成递归虽不扣分但空间就不是 O(1) 了 · n_1 取上取整还是下取整看题干给 $n_1 \geq n_2$ 还是 $n_1 \leq n_2$，方向抄反整题白做

堆三动作与堆排序

真题年份 | 2022（应用题） | 堆的建立、插入、删除三个动作的比较次数几乎隔年一道选择，2022 直接考大题

必背结论

- 建堆：从 $\lfloor n/2 \rfloor$ 号结点倒着调到 1 号，每个结点向下调整；比较次数不超过 $4n$，时间 $O(n)$，不是 $O(n \log n)$。
- 插入：新结点放末尾，只与父结点比、只往上走，不左右比、不向下调整；比较次数不超过 $\lceil \log_2(n+1) \rceil$。
- 删除堆顶：末元素顶上来，从根持续向下；每层先左右比一次选出较大孩子，再与待调元素比一次，共约 $2\log_2 n$ 次。
- 堆排序 = 建堆 + $n-1$ 次删除堆顶；时间恒 $O(n \log n)$、空间 $O(1)$、不稳定、只能用数组。
- 下标关系（从 1 开始存）：父 $\lfloor x/2 \rfloor$、左孩子 $2x$、右孩子 $2x+1$；大根堆满足 $a[x] \geq a[2x]$ 且 $a[x] \geq a[2x+1]$。
- 找最小的 k 个数用大根堆（堆顶是这 k 个里最大的），找最大的 k 个反过来用小根堆；时间 $O(n)$、空间 $O(1)$。
- $O(n)$ 的来源：建 k 个元素的堆只要 $O(k)$，之后每个元素最多下沉 $\log k$ 层，共 $O(n \log k)$，k 为常数时写作 $O(n)$。

模板/流水线

```
#include <stdio.h>
#define N 7
/* 下沉：a[x] 与较大的孩子比，小就换，换完继续往下——不能调一次就停 */
void SiftDown(int a[], int x, int n) {
    int ch = 2 * x, t;
    while (ch <= n) {
        if (ch + 1 <= n && a[ch + 1] > a[ch]) ch++;  /* 先左右比，选出较大的孩子 */
        if (a[ch] <= a[x]) break;                    /* 再与待调元素比，不大就停 */
        t = a[x]; a[x] = a[ch]; a[ch] = t;           /* 交换后继续向下 */
        x = ch; ch = 2 * x;
    }
}
void BuildHeap(int a[], int n) {
    int i;
    for (i = n / 2; i >= 1; i--) SiftDown(a, i, n);  /* 从最后一个非叶子结点倒着调 */
}
void HeapSort(int a[], int n) {
    int i, t;
    BuildHeap(a, n);
    for (i = n; i > 1; i--) {                        /* n-1 趟，每趟取走堆顶放末尾 */
        t = a[1]; a[1] = a[i]; a[i] = t;
        SiftDown(a, 1, i - 1);
    }
}
```

拿 [6,1,5,9,8,4,7] 默推一遍当自检：建堆得 [9,8,7,1,6,4,5]（比较 8 次）→删堆顶得 8、再删得 7 →最终 [1,4,5,6,7,8,9]。2022 那道应用题就是把这个过程逐趟写出来。

建堆和删除都要「持续向下」，换下去的结点还得再比一层，漏了这一步全错（阅卷最容易扣分处）· 插入只往上比，别顺手写左右比，那会把比较次数算多 · 堆排序的数组从 1 开始用，写成 0 起始时父子公式变成 2x+1、2x+2，考场上别两种混用

归并与逆序对

真题年份 | 选择常考，大题未考 | 服务 2011： 两有序序列中位数就是归并双指针走到第 n 步

必背结论

- 趟数 $S = \lceil \log_2 n \rceil$，每趟 $O(n)$（全部元素扫一遍），最好平均最坏都是 $O(n \log n)$、都是 S 趟。
- 空间 $O(n)$（辅助数组），必须开；想原地合并会覆盖还没搬走的数据。

- 稳定：合并时写 `a[i] <= a[j]` 取左边，相等元素不换位；写成 `<` 就变不稳定。
- 逆序对：右段元素先出时，左段剩下的 $m-i+1$ 个全比它大，逆序对数直接加 $m-i+1$，总时间仍是 $O(n\log n)$。
- 合并两个有序表：双指针谁小取谁，一段走完把另一段整段搬过去，$O(m+n)$，不需要任何排序。
- 归并是外部排序的内核；多路归并（k 路）就是把两两比换成在 k 个指针里挑最小。
- 2011 求两个等长有序序列的中位数：中位数就是归并后第 n 个元素（两表各长 n），双指针走 n 步即得，不必真排完，$O(n)$、$O(1)$。
- 求逆序对另有树状数组 $O(n\log n)$ 与暴力 $O(n^2)$ 两种口径，考试只写归并这一种。

模板/流水线

```
#include <stdio.h>
#define N 7
int tmp[N], cnt = 0;                    /* cnt 统计逆序对 */
void MergeSort(int a[], int l, int r) {
    int m, i, j, k, t;
    if (l >= r) return;                 /* 只有一个元素天然有序，递归出口 */
    m = (l + r) / 2;
    MergeSort(a, l, m);
    MergeSort(a, m + 1, r);
    i = l; j = m + 1; k = l;
    while (i <= m && j <= r) {
        if (a[i] <= a[j]) tmp[k++] = a[i++];            /* 取等号才稳定 */
        else { cnt += m - i + 1; tmp[k++] = a[j++]; }   /* 右段先出，左段剩余全成逆序 */
    }
    while (i <= m) tmp[k++] = a[i++];                   /* 剩的整段接上 */
    while (j <= r) tmp[k++] = a[j++];
    for (t = l; t <= r; t++) a[t] = tmp[t];             /* 拷回原数组 */
}
```

归并空间记成 O(1) 是最高频错误，八种排序里只有它是 O(n) 辅助空间 · 趟数写成下取整会少算一趟：7 个元素 3 趟、8 个元素还是 3 趟，只认上取整 · 逆序对只在右段先出时累加，左段先出时为 0，不要两边都加

比较计数排序

真题年份 | 2021（应用题） | 排序章节的第二道应用题，只考这一类「统计位置」的思想

必背结论

- 比较计数排序：$c[i]$ 记「比 $a[i]$ 大」的元素个数（相等时按下标小者为大），$c[i]$ 就是排好后 $a[i]$ 的最终下标，直接 `b[c[i]] = a[i]`。
- 比较次数恒为 $n(n-1)/2$（每对元素恰好比一次），与初始序列完全无关；时间 $O(n^2)$、空间 $O(n)$。
- 相等元素按下标定序就能稳定，这一步是它唯一能考「稳定」的地方，不写下标规则就退化成不稳定。
- 计数排序（值域 0 到 $r-1$）：统计个数→前缀和→倒序回填，时间 $O(n+r)$、空间 $O(r)$、稳定；倒序回填是稳定性的来源。
- 两者都不交换元素，只做「算位置 + 直接落位」：前者用比较算位置，后者用值当下标。
- 值域 r 远大于 n 时计数排序空间爆掉，此时还不如快排；基数排序就是把它按位拆成 d 趟。

模板/流水线

```
#include <stdio.h>
#define N 7
/* 比较计数排序：c[i] = 比 a[i] 大的元素个数（相等时下标小者算大），得降序 */
void CmpCountSort(int a[], int b[], int n) {
    int c[N] = {0}, i, j;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            if (a[j] > a[i] || (a[j] == a[i] && j < i)) c[i]++;  /* 每对恰好比一次 */
    for (i = 0; i < n; i++) b[c[i]] = a[i];                     /* c[i] 就是最终下标 */
}
```

```
#include <stdio.h>
#define N 8
/* 计数排序：关键字取值 0..r-1，r <= 10 */
void CountSort(int a[], int b[], int n, int r) {
    int c[10] = {0}, i;
    for (i = 0; i < n; i++) c[a[i]]++;                  /* 1 统计各值出现次数 */
    for (i = 1; i < r; i++) c[i] += c[i - 1];           /* 2 前缀和：c[v]-1 是末位下标 */
    for (i = n - 1; i >= 0; i--) b[--c[a[i]]] = a[i];   /* 3 倒序回填，保证稳定 */
}
```

$c[i]$ 的方向要跟题干一致：统计「比它大」得降序，统计「比它小」得升序，方向反了答案整体倒过来 · 相等元素不给索引规则，稳定性必错 · 计数排序回填必须从后往前，从前往后会把相同元素的顺序颠倒

外部排序三件套

真题年份 | 2023（应用题） | 外排序冷门但近年上升，2023 一题；败者树概念另有选择题

必背结论

- 外部排序 = 归并思想，两个阶段：置换选择排序造初始归并段，再多路归并；内存装不下全部数据是它存在的唯一理由。
- 归并趟数 $S = \lceil \log_k m \rceil$（$m$ 个初始归并段、k 路归并）；每多一路，趟数少一层，I/O 少一轮。
- 每趟归并要把全部数据读一遍写一遍，N 个磁盘块共 $2N$ 次 I/O；S 趟共 $2NS$ 次，连造初始段算上是 $2N(S+1)$。
- 置换选择排序：工作区保留 M 条记录，每次输出「不小于当前段最大值的元素中最小的那个」，输出一个就从输入补一个进工作区；工作区全部小于当前段最大值时本段结束。
- 置换选择的段长：输入随机时段长平均 $2M$；输入完全有序时只生成 1 个段，段长就是 n。
- 败者树：内部结点存败者所在段号、叶子存关键字、`ls[0]` 是冠军；一次调整只沿一条链走，比较次数由 $k-1$ 降到 $\lceil \log_2 k \rceil$，$k = 8$ 时是 3 次。
- 最佳归并树本质是哈夫曼树：每次合并 k 个最短的段；磁盘块读写总数 $= 2 \times WPL$（权取归并段长度）。
- k 路归并段数不够时补长度 0 的虚段，条件是 $(m-1) \bmod (k-1) = 0$，否则哈夫曼树建不成严格 k 叉。

模板/流水线

```
#include <stdio.h>
#define K 4                       /* 4 路归并 */
#define INF 0x7fffffff
int b[K + 1];                     /* 各段当前关键字；b[K] 为极小哨兵 */
int ls[K];                        /* 内部结点存败者段号；ls[0] 是冠军 */
int seg[K][8] = {{1,6,9,20},{3,5,22},{2,7,8,30},{4,10,11,12,25}};
int len[K] = {4, 3, 4, 5}, pos[K], total = 16, done = 0;
/* 沿 s 的父链往上：大的留下当败者，小的继续上升，一次只走树高 log k 步 */
void Adjust(int s) {
    int t = (s + K) / 2, tmp;
    while (t > 0) {
        if (b[s] > b[ls[t]]) { tmp = s; s = ls[t]; ls[t] = tmp; }
        t /= 2;
    }
    ls[0] = s;                    /* 冠军结点：全树最小者的段号 */
}
```

最佳归并树算账模板：段长 2、3、2。顺序合并 (2+3) 再 +2 ⇒ 读 5 写 5 加读 7 写 7 共 24 次；哈夫曼先合 (2+2) 再 +3 ⇒ 8 加 14 共 22 次。$WPL = 2\times2 + 2\times2 + 3\times1 = 11$，$2 \times 11 = 22$，与逐次相加一致。

败者树内部结点存的是段号不是关键字，叶子才存关键字 · 置换选择排序漏了「输出一个补一个」，工作区就不再是 M 条 · 归并趟数写成 m/k 或 $\lceil m/k \rceil$ 都错，正确的是 $\lceil \log_k m \rceil$

排序思想解算法题

真题年份 | 2018、2020 | 「用排序换时间」是的通用骨架，两道满分答法都是 $O(n)$ 加 $O(1)$

必背结论

- 2018 最小未出现正整数：值域当桶的原地版——把值 x 换到下标 $x-1$（只处理 $1 \le x \le n$），第一个 $a[i] \neq i+1$ 的位置就是答案，全对返回 $n+1$。
- 原地哈希每个元素最多被换到正确位置一次，交换总次数不超过 n，时间 $O(n)$、空间 $O(1)$，比先排序 $O(n \log n)$ 更优。
- 2020 三元组距离最小：三个升序数组各设一个指针，每轮算出当前距离并更新最优，然后只把「当前最小元素」所在数组的指针前挪一位，时间 $O(n_1+n_2+n_3)$。
- 只挪最小元素的原因：距离由最大值减最小值主导，挪大的只会让距离更大，挪小的才可能变小。
- 排序换时间两种范式：先排序 $O(n \log n)$ 再双指针或二分（把 $O(n^2)$ 降成 $O(n \log n)$）；或把值域当下标直接 $O(n)$。
- 看到「值域是 1 到 n」「要求 $O(n)$ 且 $O(1)$」同时出现，基本就是原地哈希。

模板/流水线

```
#include <stdio.h>
#define N 4
#define M 4
/* 2018：最小未出现正整数，时间 O(n)、空间 O(1) */
int FirstMissing(int a[], int n) {
    int i, t;
    for (i = 0; i < n; i++)
        while (a[i] >= 1 && a[i] <= n && a[a[i] - 1] != a[i]) {
            t = a[i]; a[i] = a[t - 1]; a[t - 1] = t;   /* 把 x 换到下标 x-1 */
        }
    for (i = 0; i < n; i++)
        if (a[i] != i + 1) return i + 1;               /* 第一个不合位的就是它 */
    return n + 1;
}
/* 2020：三元组距离最小，三个升序数组，每轮只挪最小元素所在数组的指针 */
int MinDist(int A[], int n1, int B[], int n2, int C[], int n3) {
    int i = 0, j = 0, k = 0, best = 0x7fffffff, d, mn;
    while (i < n1 && j < n2 && k < n3) {
        d = (A[i]>B[j]?A[i]-B[j]:B[j]-A[i]) + (B[j]>C[k]?B[j]-C[k]:C[k]-B[j])
          + (C[k]>A[i]?C[k]-A[i]:A[i]-C[k]);
        if (d < best) best = d;
        mn = A[i]; if (B[j] < mn) mn = B[j]; if (C[k] < mn) mn = C[k];
        if (A[i] == mn) i++; else if (B[j] == mn) j++; else k++;
    }
    return best;
}
```

原地哈希的 `while` 不能写成 `if`，换过去的新值可能还没到位，要一直换到落位或越界 · `while` 里不要写 `i++`，同一位要换到落位或越界才放行，写了 `i++` 就会跳过换进来的元素 · 2020 每轮只挪一个指针，同时挪两个或挪最大的写法必错；距离公式三段绝对值都要算，别只算两段

第二编

计算机组成原理

综合应用题第 43、44 题，共 23 分｜每年两题，一题算数一题算时间

补码十条性质速查

真题年份 | 2011/2017/2019/2020/2021/2022/2024/2025/2026（选择题年年有，大题里作第 1 小问） | 9 年 9 次，是所有运算类题目的地基，2-4 分

必背结论

1. 表示范围：n 位补码 $-2^{n-1} \sim 2^{n-1}-1$；4 位 $-8 \sim +7$，8 位 $-128 \sim +127$，16 位 $-32768 \sim +32767$。
2. 补码的 0 唯一（全 0），比原码多表示一个负数 -2^{n-1}；-2^{n-1} 只有补码，没有原码和反码。
3. 正数：$[x]_{补}=[x]_{原}$，符号位 0，其余位就是二进制本身。
4. 负数求补：符号位不动，从右往左找第一个 1，这个 1 和它右边的 0 都不动，左边的数据位全部取反；等价说法是「按位取反、末位加 1」。
5. 补码求真值：符号位 0 直接读；符号位 1 则数值位取反加 1 得绝对值再加负号。
6. 特殊值：全 1 是 -1；1 后面跟 k 个 0 是 -2^k（1000 是 -8，1000 0000 是 -128）。
7. 求相反数：$[-A]_{补}$ 是 $[A]_{补}$ 全部位（含符号位）取反加 1；机器内部就是「全取反 + 低位进 1」两步。
8. 比大小：同为负数时把补码当无符号数比，大者真值也大；两补码当无符号数差 d，真值也差 d。
9. 模运算：$[x]_{补}=2^n+x \pmod{2^n}$，所以加减溢出后截断取低 n 位仍然自洽。
10. 符号扩展：有符号数高位补符号位（负数补 1），无符号数高位补 0，$0xFFF7 \to 0xFFFFFFF7$。

- 同一个机器数按有符号和无符号读，差值是 2^n：$0x86$ 读成 -122 或 134，差 256，机器数本身没变，变的只是解释方式。
- 边界值必须背成条件反射：$0x7F=+127$、$0x80=-128$、$0xFF=-1$、$0x7FFF=+32767$、$0x8000=-32768$、$0xFFFF=-1$。

编码	正数	负数	关键差别
原码	符号位 0 加数值	符号位 1 加数值	0 有两种写法，表不出 -128
反码	同原码	数据位按位取反	0 有两种写法
补码	同原码	反码末位加 1	0 唯一，能表 -128
移码	补码符号位取反	同左	全 0 对应最小值，常用于阶码

一串 F 开头的负数：加几能进位成全 0，几就是它的绝对值。$0xFFD3+0x2D=0x10000$，真值 -45；$0xFFFF+1=0x10000$，真值 -1。反过来给 -45 求 16 位补码，直接写 $0xFFD3$。

模板/流水线

```
/* ① 真值 x → n 位补码 */
if (x >= 0) M = x;                      /* 正数：原码即补码，高位补 0 */
else        M = (1 << n) - (-x);        /* 负数：模 2^n 减去绝对值 */
/* ② n 位补码 M → 真值 */
if ((M >> (n-1)) & 1) v = M - (1 << n); /* 负数 */
else                  v = M;            /* 正数 */
/* ③ 十六进制速算：-45 的 16 位补码 */
// 45 = 0x2D → 想" 加什么能进位到 0x10000" → 0xD3 → 前面补满 F → 0xFFD3
/* ④ 符号扩展：16 位 → 32 位 */
// 负数高位全填 1：0xFFD3 → 0xFFFFFFD3；正数高位全填 0：0x007F → 0x0000007F
```

1000 0000 在 8 位补码里是 -128，不是 $+128$ · 「全 1 是 -1」「1 后 k 个 0 是 -2^k」必须秒答 · 进位反推法只对「前面连续一串 F」的数成立，$0xCAF8$ 这种老实转原码

补码加减与三种判溢出

真题年份 | 2011（补码加减与溢出判断）/2022（ALU 符号与溢出标志逻辑表达式） | 判溢出几乎年年出现，选择或大题第 1 小问，3-5 分

必背结论

- 加法：$[x+y]_{补}=[x]_{补}+[y]_{补}\ (\mathrm{mod}\ 2^n)$，符号位当数值位一起加，最高位进位丢弃。
- 减法：$[x-y]_{补}=[x]_{补}+[-y]_{补}$，先求 $[-y]_{补}$ 再当加法做；也可以直接竖式相减，不够减从最高位借，结果与转加法完全一致。
- 电路实现只有一套：$Sub=1$ 时把 Y 全部取反、C_{in} 给 1；$Sub=0$ 时 Y 不动、C_{in} 给 0。所以减法不是另做一个减法器，而是「加相反数」。
- 三种判溢出互相等价，但机器只能用符号法和进位法；范围法只是人算时的验算手段，答题时写「计算机内部只能按符号位或进位判」才拿满分。
- 正 + 负、负 + 正一定不溢出；只有同号相加、异号相减才可能溢出。
- 上溢指超过 $2^{n-1}-1$，下溢指小于 -2^{n-1}；补码下溢不是「归零」，而是符号位翻转、结果被读成一个大正数。
- 进位和溢出是两件事：$0x7F+0x01$ 有溢出但没有进位出，$0xFF+0x01$ 有进位出但没有溢出，所以 OF 和 CF 必须分开算。
- 题问「寄存器里最后是什么值」时不用管溢出：机器只保留低 n 位，模运算保证截断后的机器数自洽；只有题干明确问「是否溢出」才动笔判。

方法	加法判据	减法判据	用在哪
范围法	结果超出 -2^(n-1) 至 2^(n-1)-1	同左	人算、验算
符号法	正 + 正得负、负 + 负得正	正-负得负、负-正得正	机器 OF
进位法	C_n 异或 C_(n-1) 等于 1	同左	机器 OF
双符号位	00 或 11 正常，01 上溢，10 下溢	同左	人算、判上下溢

模板/流水线

```
/* 8 位补码 x - y：四步走，一步都不能省 */
step1：写 [x] 补、[y] 补          // 负数：符号位不动，最右 1 及其右边 0 不动，其余取反
step2：[-y] 补 = [y] 补 全部 8 位取反，末位加 1
step3：[x] 补 + [-y] 补 列竖式   // 符号位一起加，第 9 位进位直接丢
step4：判溢出                    // 正-负得负 或 负-正得正 → 溢出；同号相加变号 → 溢出
/* 例：4 位 +5 + 3 = 0101 + 0011 = 1000，读作 -8，正 + 正得负 → 上溢 */
/* 例：4 位 -5 + -4 = 1011 + 1100 = 1 0111 → 0111 = +7，负 + 负得正 → 下溢 */
/* 例：16 位 x = 0xFFDF(-33)，y = 0x0041(+65) */
// x - y = 0xFFDF + 0xFFBF = 0x1FF9E → 取低 16 位 0xFF9E = -98
// 判溢出：负 - 正 = 负，符号没变 → 不溢出，结果可用
```

溢出后机器照样输出一个数，不判溢出直接写答案是大题第一扣分点 · 减法必须先转 $[-y]_{补}$ 再当加法判，别拿「正-负」去套「正 + 负」· 双符号位只是人算工具，别写成机器判据

标志位 CF/ZF/OF/SF

真题年份 | 2022（ALU 溢出与符号标志逻辑表达式）/2013（条件转移与标志位）/2021/2024（RISC-V 指令 ALU 溢出） | 2-3 年一次，选择必考，大题作小问，2-4 分

必背结论

- $OF=C_n\oplus C_{n-1}$，判有符号数溢出，只对有符号数有意义。
- $CF=Sub\oplus C_{out}$，加法时 CF 是进位、减法时 CF 是借位，只对无符号数有意义。
- $ZF=1$ 当且仅当结果各位全 0，有符号数与无符号数都有意义。

- SF 取结果的最高位，判有符号数的正负。
- 要求写成逻辑表达式时（2022 考法）：$OF = \bar{A}_n\bar{B}_nS_n + A_nB_n\bar{S}_n$，即「两个加数符号相同、结果符号与之不同」。
- $Sub = 1$ 做减法：Y 每一位取反且 $C_{in} = 1$，即 $A - B = A + \bar{B} + 1$；$Sub = 0$ 做加法，Y 不动且 $C_{in} = 0$。
- 同一个加法器既算无符号也算有符号，机器数相同结果就相同，差别只在标志位怎么解释。

运算	结果	CF	OF	ZF	SF
0x05 + 0x02	0x07	0	0	0	0
0x05 - 0x02	0x03	0	0	0	0
0x7F + 0x01	0x80	0	1	0	1
0xFF + 0x01	0x00	1	0	1	0
0x80 + 0x80	0x00	1	1	1	0

条件转移就是查这张表

- `CMP A,B` 实质是做 $A - B$ 但不写回，只置标志，所以后面的条件转移全靠 CF、ZF、SF、OF 四个位。
- 无符号比较看 CF 和 ZF：$A > B$ 是 $CF = 0$ 且 $ZF = 0$；$A \geq B$ 是 $CF = 0$；$A < B$ 是 $CF = 1$。
- 有符号比较看 SF、OF、ZF：$A > B$ 是 $SF = OF$ 且 $ZF = 0$；$A < B$ 是 $SF \neq OF$；$A = B$ 是 $ZF = 1$。
- 2013 年那道「条件转移指令的转移条件」就考这个对应关系，别去背汇编助记符，背「无符号看 CF、有符号看 SF 与 OF」。
- 移位指令也会改 CF：最后移出去的那一位进 CF，所以「CF 里是刚才移出的位」这句话在移位指令后成立，在加减指令后是进位或借位。
- 答题时先问一句「这条指令的数是有符号还是无符号」，再决定看 OF 还是 CF；两个都算出来但只解释一个，另一个按标准写「无意义」。

模板/流水线

```
/* n 位加减法器：sub=0 加，sub=1 减，带回四个标志 */
unsigned AddSub(unsigned x, unsigned y, int sub, int n,
                int *CF, int *OF, int *ZF, int *SF) {
    unsigned mask = (n == 32) ? 0xFFFFFFFFu : ((1u << n) - 1);
    unsigned lo   = (1u << (n-1)) - 1;              /* 低 n-1 位 */
    unsigned t    = sub ? (~y & mask) : y;          /* 减法先把 Y 全取反 */
    unsigned s    = x + t + (sub ? 1u : 0u);        /* 第 n 位就是 C_out */
    unsigned r    = s & mask;                       /* 结果低 n 位 */
    unsigned cout = (s >> n) & 1u;
    unsigned cn1  = (((x & lo) + (t & lo) + (sub ? 1u : 0u)) >> (n-1)) & 1u;
    *CF = (sub ^ cout) & 1u;                        /* 加法看进位，减法看借位 */
    *OF = cout ^ cn1;                               /* C_n 异或 C_(n-1) */
    *ZF = (r == 0);
    *SF = (r >> (n-1)) & 1u;
    return r;
}
```

CF 对无符号、OF 对有符号，别把两者当同一个东西 · 减法时 $CF = 1$ 表示不够减（有借位），不是「有进位」· ZF 与溢出无关：0x80+0x80 结果 0、ZF=1 但已经溢出

移位：算术逻辑循环

真题年份 | 2020（移位乘法）/2017（无符号数循环移位） | 很少单独成题，但乘法流水线与循环移位判断年年要用，2-3 分

必背结论

- 算术左移与逻辑左移相同：低位补 0，左移 1 位相当于乘 2，符号位变了就溢出。
- 算术右移：高位补符号位，低位丢弃；补码算术右移 1 位等于除以 2 后向下取整，-5 右移得 -3 不是 -2。

- 逻辑右移：高位补 0，只用于无符号数；有符号数如果误用逻辑右移，负数会变成大正数。
- 循环移位：移出的位补到另一端，n 位循环移动 n 位数值复原；小循环只在本寄存器内转，大循环把 CF 串进来一起转。
- 原码移位规则不同：符号位不动，数据位左移补 0、右移补 0；只有补码右移才补符号位。
- 左移判溢出：移位前后符号位不同即溢出，不是看有没有位被移出去；右移不会溢出，只会丢低位。
- 移位量是常数时算「×/÷ 2 的幂」，题目问「用移位和加法实现乘 6」就拆成 $x << 2$ 加 $x << 1$。
- 乘法流水线里的「右移」是把部分积和乘数寄存器连成一个整体右移一位，不是只移部分积，写流程时别漏掉乘数寄存器。

类型	规则	1011 操作后
算术左移	低位补 0	0110，符号位由 1 变 0，溢出
算术右移	高位补符号位 1	1101，即 -3
逻辑右移	高位补 0	0101，即 +5
循环右移	移出位补到最高位	1101
循环左移	移出位补到最低位	0111

模板/流水线

```
/* n 位循环移位：k 必须先对 n 取模，题目常给 k > n */
unsigned rotl(unsigned x, int k, int n) {
    unsigned mask = (n == 32) ? 0xFFFFFFFFu : ((1u << n) - 1);
    k %= n;  x &= mask;
    return k ? (((x << k) | (x >> (n - k))) & mask) : x;
}
/* 例：8 位 0xA5 = 1010 0101 循环左移 3 位 */
// (0xA5 << 3) & 0xFF = 0x28，0xA5 >> 5 = 0x05 → 0x2D = 0010 1101
/* 算术右移手算：负数最高位补 1，其余右移 */
// -5 的 4 位补码 1011 算术右移 1 位 → 1101 = -3
/* 算术左移判溢出：移位前后符号位不同即溢出 */
// 0101(+5) 左移 1 位 = 1010(-6)，符号位变了 → 溢出
```

算术右移补的是符号位不是 0，负数右移后仍是负数 · 左移判溢出看符号位是否变化，不看有没有进位出 · 循环移位前先 $k \bmod n$，忘了取模整题错

乘法与除法流水线

真题年份 | 2020（移位乘法与 2n 位乘积溢出）/2019（乘法溢出）/2017/2025（补码除法器与除法异常） | 乘法约 3 年一次，除法 2025 新考，5-8 分

必背结论

- 没有乘法指令时：乘法 = 加法 + 移位。$13 \times 6 = 13 \times (4 + 2)$，即 13 左移 2 位加 13 左移 1 位，得 110100 加 011010 等于 1001110。
- 控制逻辑的作用是控制累加次数（循环次数），次数等于乘数的位数 n；这就是「ALU 只有加减和移位也能做乘法」的标准答法。
- 时间从小到大：阵列乘法器（1 个时钟周期）< 乘法指令（一条指令多周期）< 加法加移位多条指令（每条都要取指）。
- n 位乘 n 位用 $2n$ 位存乘积一定不溢出；只取低 n 位才判溢出：无符号看高 n 位是否全 0，有符号看高 n 位是否全为符号位的重复（正数全 0、负数全 1）。
- 补码一位乘（Booth）：乘数末尾加附加位 $y_{n+1} = 0$，看 $y_i y_{i+1}$ 两位，00 或 11 不加，01 加 $[x]_{补}$，10 加 $[-x]_{补}$，然后部分积与乘数一起算术右移一位，共 n 次；符号位自然形成，不用单独异或。
- 原码一位乘：符号位单独用异或算，数值部分当无符号数做移位乘法，最后拼回去。

- 补码除法（加减交替，又叫不恢复余数法）：余数与除数同号则商 1 并减除数，异号则商 0 并加除数，每步先左移一位；恢复余数法每步要试减、不够减还要加回来，步数不固定，考试写加减交替。
- 除法异常只有两个来源：除数为 0，以及 $(-2^{n-1}) \div (-1)$ 的商溢出（结果 $+2^{n-1}$ 表示不了），两条都要写。

方法	符号处理	步数
原码一位乘	符号单独异或	n 次加与右移
补码一位乘 Booth	符号自然形成	n 次加与算术右移
原码除法恢复余数	符号单独异或	不固定，最多 2n 次
补码除法加减交替	符号自然形成	固定 n 次

模板/流水线

```
/* 补码一位乘 Booth: n 位 x 乘 n 位 y, D 是双符号位部分积, 2n 位 */
D = 0; A = x; Q = y; q = 0;              /* q 是附加位 y(n+1) */
for (i = 0; i < n; i++) {                /* 循环 n 次, n 由乘数位数定 */
    switch (((Q & 1) << 1) | q) {        /* 看 y(i) y(i+1) 两位 */
        case 0: case 3: break;           /* 00 / 11: 不加 */
        case 1: D = D + A; break;        /* 01: 加 [x] 补 */
        case 2: D = D - A; break;        /* 10: 加 [-x] 补 */
    }
    q = Q & 1;                           /* 乘数最低位进附加位 */
    Q = (Q >> 1) | ((D & 1) << (n-1));   /* 部分积最低位补进乘数最高位 */
    D = (D >> 1) | (D & (1 << (2*n)));   /* 算术右移, 保持双符号位 */
}
/* 乘积 = D 的低 n 位拼接 Q, 共 2n 位 */
/* 只取低 n 位判溢出: 高 n 位全 0（无符号）或全为符号位重复（有符号）才不溢出 */
/* 例: 乘积 2^32-2, 无符号 32 位上界 2^32-1 → 不溢出; 有符号上界 2^31-1 → 溢出 */
```

判溢出必须分有符号、无符号两套判据，混用必错 · 循环次数由乘数位数决定，不是被乘数 · 除法异常要写全两条，只写「除数为 0」丢分

Cache 五角度总览

真题年份 | 2010/2012（交叉存储器带宽）/2013/2014/2016/2018/2019/2020/2021/2023/2025（第 43 或 44 题） | 11 年 11 次，计组第一大题来源，10-15 分

必背结论

- 一句话：映射方式决定地址结构，其余四问都是从地址结构上长出来的。
- 五个角度固定：映射方式、总容量、命中率、访问过程、比较器个数与位数。
- 考法极固定，题干每句话都有用，没有一句废话；读完先把「块大小、行数、路数、地址位数」四个数圈出来。
- 命中判定两把锁：地址切出来的标记等于该行存的标记，且该行有效位为 1，缺一个都算缺失。
- 命中之后还有一步：按块内地址从该块中取出那一个数据，答「访问过程」漏这一步就扣分。
- 缺失处理固定四步：整块调入对应行、写入标记、有效位置 1、再按块内地址取数。
- 比较器个数：直接映射 1 个，全相联等于行数，组相联等于路数；位数恒等于标记位数，因为它比的就是标记。
- 与 TLB 的关系：Cache 查物理地址、存数据；快表查逻辑地址、存页框号；两者都是缓存，都不属于主存。

角度	典型问法	落点
映射方式	直接还是组相联，标记几位	地址结构，标记等于块号除以行数（或组数）的商
总容量	Cache 总容量多少位	行数乘以每行（数据加标记加有效位加脏位加替换位）
命中率	给一段循环，求缺失率或缺失次数	每块首次访问必缺失，读写各算一次访问
访问过程	给一个地址，是否命中、怎么取数	定位行或组，比标记与有效位，按块内地址取数
比较器	几个比较器、位数几位	直接 1 个，全相联等于行数，组相联等于路数，位数等于标记位数

模板/流水线

```
/* Cache 大题固定五步，按顺序写；某步写不下去说明上一步漏了条件 */
step1: 定地址位数          // 按字节编址：地址空间字节数取 log2；按字编址：先除字长
step2: 拆低位              // 块内地址 = log2(块大小)；行数 = 数据区容量 / 块大小
step3: 拆中位              // 组号 = log2(行数 / 路数)；直接映射用行号 = log2(行数)
step4: 剩余高位当标记      // 标记 = 地址位数 - 块内 - 行号（或组号）
step5: 容量与命中率        // 每行容量乘行数；先验证是否占整数个块再算缺失率
```

标记位数只能由地址结构算，不能拿「Cache 里存了几个标记」去凑 · 命中要「标记相等且有效位为 1」两个条件同时成立 · 说明访问过程最后一步必须写「按块内地址取出那一个数据」，漏了扣分

Cache 地址拆分模板

真题年份 | 2010（映射与命中率）/2016（映射与虚实地址分段）/2019（地址分段）/2021（TLB 组相联） | 地址拆分是每年第 1 小问，4-6 分

必背结论

- 块内地址位数 $=\log_2$(块大小)；按字编址时 $=\log_2$(块大小/字长)。64B 块配 4B 字只占 16 个地址，块内地址 4 位而不是 6 位。
- 直接映射：地址 = 标记 | 行号 | 块内，行号 = 块号 mod 行数，标记 = ⌊块号/行数⌋。
- 全相联：地址 = 标记 | 块内，标记就是整个块号，没有行号字段，所以全相联必须配替换算法。
- 组相联：地址 = 标记 | 组号 | 块内，组号 = 块号 mod 组数，标记 = ⌊块号/组数⌋。
- 组相联是组间直接映射、组内全相联；k 路组相联就是每组 k 块，行数 = 组数 × 路数。
- 快表完全同构：块号换成虚页号，块内地址换成页内地址，行数或组数换成快表的行数或组数；4KB 页对应页内地址 12 位。
- 组相联的路数必是 2 的幂，所以组数也是 2 的幂，组号位数才对得上；题目给「8 行、二路组相联」就自动是 4 组、组号 2 位。
- 二进制切法：从低位往高位切，先块内，再行号或组号，剩下全是标记；十进制除法则反过来先求块号再拆。
- 两种方法互为验算：3200 拆成 12 位二进制是 1100 1000 0000，低 6 位 000000 是块内地址 0，接着 3 位 010 是行号 2，剩下 110 是标记 6，与除法结果完全一致。
- 虚实地址转换顺序（2016/2019 考法）：虚地址拆虚页号加页内地址，查 TLB 或页表得页框号，页框号与页内地址拼接成物理地址，再用物理地址拆 Cache 的标记、组号、块内。

模板/流水线

```
/* 十进制两步除法，任何映射都能用，还能反查位数 */
q1 = addr / block_size;  r1 = addr % block_size;  /* q1 = 块号，r1 = 块内地址 */
if (直接映射) { tag = q1 / lines; idx = q1 % lines; }  /* idx = 行号 */
if (组相联)   { tag = q1 / sets;  idx = q1 % sets;  }  /* idx = 组号 */
if (全相联)   { tag = q1; }                            /* 整个块号就是标记 */
/* 校验：tag < 2^(地址位数 - log2(块大小) - log2(行数或组数)) */
/* 例：直接映射，块 64B，8 行，地址 3200 */
// 3200 / 64 = 50 余 0 → 块号 50，块内地址 0
```

```
// 50 % 8 = 2 → 行号 2；50 / 8 = 6 → 标记 6
/* 例：32 位地址，块 64B，数据区 32KB，8 路组相联 */
// 块内 6 位；32KB / 64B = 512 行，512 / 8 = 64 组 → 组号 6 位
// 标记 = 32 - 6 - 6 = 20 位
/* 例：4KB 页，32 位虚地址，快表 8 组 4 路 */
// 页内地址 12 位；虚页号 20 位；组号 3 位；标记 = 32 - 12 - 3 = 17 位
```

块号取余得行号、取商得标记，商和余数写反最致命 · 按字编址时块内地址由地址个数决定，直接写 $\log_2$ 块字节数必错 ·「地址里的标记」与「Cache 行里存的标记」位数相同含义不同，命中就是比较这两个

Cache 总容量与性能计算

真题年份 | 2010/2013（缺失下总执行时间）/2018（Cache 参数）/2020（缺失次数）/2023（数组遍历命中率）/2025（缺失率与缺页统计） | 每年至少 1 小问，4-8 分

必背结论

- 总容量 = 行数 × (数据块 + 标记 + 有效位 + 脏位 + 替换位)，全部换成比特再相加。
- 数据块位数 = 块大小 ×8；标记位数由地址结构定；有效位恒 1 位，题目不会白给，必须自己加。
- 脏位：写回 1 位、写直达 0 位；替换算法位：直接映射 0 位、全相联 $\log_2$ 行数、组相联 $\log_2$ 路数。
- 行数 = 数据区容量 ÷ 块大小；组数 = 行数 ÷ 路数。题目给的「数据区 32KB」只是数据部分，不是总容量。
- 缺失率：数据占满整数个块时 = 每块缺失次数 ÷ 每块访问次数；不占整数块时只能 = 总缺失次数 ÷ 总访问次数。
- $h = 1 - m$，缺失次数 = 总访问次数 × 缺失率；块的首次访问必缺失，同一块后面的访问才命中。
- 平均访问时间 $T = t_c + (1 - h) \cdot t_m$；缺失代价 = 主存访问延迟 + 整块传输时间，通常几十个时钟周期。
- CPU 总时间 $= IC \times (CPI + (1 - h) \times$ 缺失代价$) \times T_{clk}$，2013 年那道「Cache 缺失下的总执行时间」就是直接代这个式子。
- 加速比 = 不用 Cache 的时间 ÷ 用 Cache 的时间，别把分子分母写反。
- 算完记得回单位：274432 位是 34304 字节，数据区 32KB 是 32768 字节，多出来的 1536 字节就是标记、有效位、替换位这些开销，答题时点一句「额外开销 1.5KB」是加分细节。

场景	每行位数	总容量
直接映射 8 行 64B 块 256MB 主存	512+19+1 = 532	4256 位
8 路组相联 512 行 64B 块直写 32 位地址	512+20+1+0+3 = 536	274432 位

模板/流水线

```
/* 容量与性能：一行一行代，单位统一成比特 */
lines = data_size / block_size;        /* 数据区容量 ÷ 块大小 */
sets  = lines / ways;                  /* 组相联才需要除 */
bits_data = block_size * 8;            /* 字节转比特，最常漏的一步 */
bits_tag  = addr_bits - log2(block_size)
          - (直接 ? log2(lines) : 组相联 ? log2(sets) : 0);
bits_line = bits_data + bits_tag + 1
          + (回写 ? 1 : 0)
          + (直接 ? 0 : 组相联 ? log2(ways) : log2(lines));
total = lines * bits_line;             /* 单位：比特 */
miss = miss_times / total_access;      /* 读写各一次时访问次数要乘 2 */
T_avg = t_cache + miss * t_miss;       /* 平均访问时间 */
T_cpu = IC * (CPI + miss * miss_penalty) * T_clk;  /* CPU 总时间 */
/* 例：块 16B 每数据 4B，A[i]=A[i]+32，1000 个元素 */
// 每块 4 个数据、每元素读 1 次写 1 次 → 每块 8 次访问缺 1 次 → 缺失率 1/8
// 1000 / 4 = 250 块（占整数个块，前提成立）；缺失 250 次，命中率 7/8
/* 例：t_cache = 1 周期，t_miss = 50 周期，h = 0.95 */
// T_avg = 1 + 0.05 * 50 = 3.5 周期
/* 例：IC = 10^6，CPI = 2，缺失率 0.02，缺失代价 20 周期 */
// CPI' = 2 + 0.02 * 20 = 2.4，总时间 = 10^6 * 2.4 * T_clk
```

标记按比特、块大小按字节，不乘 8 直接相加是最常见算错 · 「数据区容量」只是数据部分，总容量必须再加标记、有效位、脏位、替换位 · $A[i] = A[i] + 32$ 是读一次写一次共 2 次访问，缺失率 1/8；只有读或只有写才是 1/4

Cache 写策略与脏位

真题年份 | 2016（Cache 参数与虚实地址分段）/2025（缺失率与缺页统计） | 写策略常嵌在容量与缺失统计里作小问，2-4 分

必背结论

- 写直达又称全写法、直写：每次写都同时改 Cache 和主存，主存始终最新，没有脏位。
- 写回又称回写法：只改 Cache 并置脏位 1，该行被替换时才写回主存；脏位为 0 则直接丢弃。
- 写分配：写缺失时先把整个块调入 Cache 再写，通常配写回，因为「既然要写，先把整块拿来」。
- 非写分配：写缺失直接写主存、不调入 Cache，通常配写直达，避免为一次写搬一整块。
- 写回时写主存的次数等于被替换且脏位为 1 的行数，不等于写操作次数；写直达时写主存次数等于写操作次数。
- 写直达虽然简单，但每次写都占总线，所以要配写缓冲；多级 Cache 的 L1 多用写回以减少总线流量。
- 脏位又叫修改位、一致性维护位；写直达不需要脏位，因为它不存在「主存落后」的状态。
- 对总容量的影响只有 1 位：512 行的 Cache 从写回改直写，总容量减少 512 位，即 64B。
- 指令 Cache 只读不写，所以脏位只在数据 Cache 上有意义；题干说「指令 Cache 与数据 Cache 分离」时，算指令 Cache 可以不配脏位。

策略组合	写命中	写缺失	脏位
写直达加非写分配	同时写 Cache 与主存	只写主存，不调入	无
写回加写分配	只写 Cache，置脏位 1	先调入块再写	1 位

模板/流水线

```
/* 写策略只影响两处：脏位位数、写主存次数 */
bits_line += (回写 ? 1 : 0);            /* 直写加 0 位，别多给 */
total = lines * bits_line;              /* 总容量随之变化 */
write_back_times = 脏行被替换的次数;      /* 直写策略下额外写主存为 0 次 */
/* 四个组合照抄 */
// 写直达 + 非写分配：写命中 → 两级同写；写缺失 → 只写主存
// 写回   + 写分配   ：写命中 → 只写 Cache 并置脏；写缺失 → 调入块后写
/* 判据一句话：问" 总容量" 看脏位是否 1 位，问" 写主存几次" 看脏行数 */
// 例：替换 100 次、其中脏行 60 个 → 写回时额外写主存 60 次；直写时替换不额外写回 → 0 次
```

写直达没有脏位，给直写加 1 位是白送分 · 「是否调入」由写分配决定，与写直达写回是两个独立维度 · 写回策略下写主存次数不等于写操作次数，只有被替换的脏行才写回

定长扩展操作码切法

真题年份 | 2026/2021/2015（第 43/44 题） | 三年三考，指令格式题的第一小问，4–6 分起步

必背结论

- 指令 = 操作码 + 地址码。指令字长固定时，操作码每扩展一级就吃掉一个地址码字段，地址码个数减 1。
- 操作码位数只决定指令条数：n 位操作码最多 2^n 条指令，与地址码几位无关。问「最多多少条」只数操作码。
- 扩展的合法做法：短操作码只留出恰好 1 个编码当扩展窗口，长操作码只能用这个窗口内的编码，短操作码已用掉的编码不许再出现。
- 通用式：t 级扩展、每级 k 位→前 $t-1$ 级各 2^k-1 条，最后一级 2^k 条，总数 $(t-1)(2^k-1)+2^k$。
- 最常考的四个数字（4 位一段，三/二/一/零地址四级）：15 + 15 + 15 + 16 = 61 条。

- 各地址格式要能一句答出：三地址 = 两源一目的；二地址 = 目的兼作源；一地址 = 另一操作数隐含在 ACC；零地址 = 无操作数或操作数在栈顶。
- 2015 年「访存时钟」那一问：地址码里每有一个是内存地址，执行阶段就多一次访存；三个地址全在内存→执行阶段读两次写一次，共 3 次访存。
- 2021 年是 R/I/J 三类格式：R 型操作数全在寄存器、I 型带立即数或地址、J 型带跳转地址；问「某类最多几条」仍只看该类操作码位数。
- 先扣地址码再算操作码：指令字长 16 位、地址码各 4 位、双操作数→操作码位数为 $16-4\times2=8$ 位，最多 $2^8=256$ 条。
- 2026 年多格式定长编码：同一 16 位字长下几种格式共用前几位操作码，求值时先按操作码判格式，再按该格式的字段边界取数，不许跨格式借位。

切位顺序永远是「先短码、后长码」，短码数 = 2^k-1，因为必须留 1 个窗口给下一级。

模板/流水线

```
/* 定长指令字 + 扩展操作码：4 位一段，三/二/一/零地址四级 */
/* 指令字长 16 位 = OP(4) + A1(4) + A2(4) + A3(4) */
/* 第 1 级 三地址：0000~1110 共 15 条，1111 留作扩展窗口 */
int c3 = (1 << 4) - 1;                  /* 15 */
/* 第 2 级 二地址：1111 + 原 A1 的 4 位，仍留 1 个窗口 */
int c2 = (1 << 4) - 1;                  /* 15 */
/* 第 3 级 一地址：再吃一个 4 位，仍留 1 个窗口 */
int c1 = (1 << 4) - 1;                  /* 15 */
/* 第 4 级 零地址：不用再留窗口，全部可用 */
int c0 = (1 << 4);                      /* 16 */
int total = c3 + c2 + c1 + c0;          /* 61 条 */
/* 通用式：t 级扩展、每级 k 位 => (t-1)*(2^k-1) + 2^k */
/* 不做扩展时：条数 = 2^ 操作码位数，别再自作主张减 1 */
/* 反向题：给了各级条数，按 2^k 减去本级条数反推窗口数 */
```

长操作码只能取扩展窗口内的编码，短码用过的编码再出现就是错 · 每一级必须恰好留 1 个窗口，留 0 会越界、留 2 条数就对不上 · 扩展操作码的前提是指令字长固定，遇到不定长格式这套算法不能套

寻址方式全表与 EA

真题年份 | 2010/2023/2024（第 43/44 题） | 大题里只作一问，选择题几乎年年考

必背结论

- 形式地址 A 是地址码里那个数；有效地址 EA 是操作数真正所在的地址；操作数 X 要在 EA 上再取一次内容。
- 括号就是「取内容」：$EA=(A)$ 表示 A 号单元的内容才是地址，操作数 $X=((A))$，多一层括号就多访存一次。
- 访存次数（不含取指）：立即 0、寄存器 0、直接 1、寄存器间接 1、间接 2。答题写「多一次访存」比写「慢」得分。
- 基址与变址只差一句：基址是 BR 不变、地址码变（便于程序浮动与重定位）；变址是 IX 变、地址码不变（专治数组）。
- 真题口径比书上多两步：变址取数组元素地址 $=A+i\times$ 元素字节数（A 是数组首地址、i 是下标）；相对寻址的 PC 用自增后的值。
- 立即寻址的数就摆在指令里，取值范围受指令字长限制；间接寻址的 A 是形式地址，寻址范围由存储字长决定。
- 指令里为什么要多一个寻址特征字段：同一个寄存器编号，寄存器寻址与寄存器间接寻址的执行流程不一样，只能靠它区分。
- 堆栈寻址的操作数在栈顶，EA 由 SP 隐含给出：入栈先 $SP-1\to SP$ 再存数，出栈先取数再 $SP+1\to SP$（栈向低地址生长）。

- 用途对照比公式好记：相对寻址配转移指令，基址配多道程序重定位，变址配数组循环，堆栈配子程序调用与参数传递。

寻址方式	EA	取操作数访存	典型用途
隐含	无（操作数在 ACC）	0	一地址指令的第二个操作数
立即	无（数在指令中）	0	常数、计数初值
直接	`EA=A`	1	一般变量
间接	`EA=(A)`	2	指针、扩大寻址范围
寄存器	`EA=R`（数在 R 中）	0	中间结果、循环变量
寄存器间接	`EA=(R)`	1	指针、参数传递
相对	`EA=(PC)+A`	1	转移指令、程序浮动
基址	`EA=(BR)+A`	1	多道程序重定位
变址	`EA=(IX)+A`	1	数组、字符串
堆栈	栈顶由 SP 隐含	1	现场保护、子程序调用

模板/流水线

```
/* 判寻址方式只看括号：括号套几层，就多访存几次 */
/* R4      -> 寄存器寻址：操作数 = (R4)，不访存 */
/* (R4)    -> 寄存器间接：EA = (R4)，再访存一次才拿到数 */
/* ((R4))  -> 再间接一层：EA = ((R4))，访存两次 */
/* A       -> 直接寻址：EA = A，访存一次 */
/* (A)     -> 间接寻址：EA = (A)，访存两次 */
/* #10     -> 立即寻址：操作数就是 10，一次也不访存 */
/* 相对寻址：EA = 当前指令地址 + 指令长度 + 系数 * 偏移量 */
/* 变址取数组：EA = 首地址 + 下标 * 元素字节数（不是下标本身） */
unsigned int ea;
if (reg_indirect)   ea = R[4];            /* 一次访存 */
else if (indirect)  ea = mem[A];          /* 两次访存 */
else if (direct)    ea = A;               /* 一次访存 */
else if (indexed)   ea = IX + A * 4;      /* A 是首地址，4 是元素字节数 */
```

间接寻址写 $EA=(A)$、操作数写 $((A))$，差一层括号就丢分 · 寄存器寻址 $EA=R$ 而操作数是 (R)，一次也不访存 · 基址与变址「谁不变」最容易背反，记「基址不动地址码动，变址动地址码不动」

相对寻址求转移目标

真题年份 | 2013/2014/2019（第 43/44 题） | 汇编大题的核心一问，三年三考

必背结论

- 目标地址 $=(PC)+$ 指令长度 $+$ 系数 $\times A$，式子里的 PC 是「取指后已自增」的值，不是当前指令地址。
- 一句话记：先把 PC 推到下一条指令，再加偏移量；顺序执行时 PC 直接加指令长度。
- 偏移量是有符号补码，必须先求真值再乘系数：8 位补码 $E3H$ 的真值是 -29，不是 227。
- 8 位补码范围 $-128\sim+127$：$A=-1$ 原地踏步（跳回自己），$A\geq 0$ 向高地址，$A\leq -2$ 向低地址。
- 向后可用条数：$(-2)-(-128)+1=127$ 条，闭区间计数一律「末 − 首 + 1」。
- 系数从题干读：偏移量以「字/半字」为单位、或「每条指令 n 字节」时要乘；2013、2014 两年系数都是 2。
- 2013 年条件转移还考标志位：无符号比较「小于等于」判 CF 与 ZF（小于→ $CF=1$；等于→ $CF=0$ 且 $ZF=1$），检测位写 110；问的是「检测哪几位」不是「CF 等于几」。

- 反过来给目标地址求偏移量：$A = (\text{目标} - (PC + \text{指令长度})) \div \text{系数}$，算出的负数就是补码真值，再写成补码字节；2019 年递归题就是这么倒推出调用入口的。
- 跳转范围由偏移量字段的宽度决定（8 位→ 127 条），不是由整条指令字长决定；指令字长只管系数，两件事别混。
- 指令长度从题干的行地址差读出：下一行地址减本行地址就是本条指令的字节数（差 5 就是 5 字节），别一律默认 2 字节。

模板/流水线

```
/* 转移目标 = 取指后 PC + 系数 * 偏移量（偏移量是有符号补码） */
/* 例：当前指令 200CH，指令长 2 字节，偏移量字节 E3H，系数 2 */
int pc_next = 0x200C + 2;                   /* 先推到下一条 = 200EH */
signed char off = (signed char)0xE3;        /* 求真值 -29，不能直接加 E3 */
int target = pc_next + 2 * off;             /* 0x200E - 58 = 1FD4H */
/* 等价算法：0x200C - 0x38 = 0x1FD4，两条路必须对得上 */
/* 向后条数：8 位补码 -128 ~ -2 可用 => (-2)-(-128)+1 = 127 条 */
/* 已知目标反求偏移：(target - pc_next) / 系数，负数写成补码 */
/* 若问「顺序执行时的地址」：不转移就是 pc_next，别加偏移量 */
```

PC 用「取指后」的值，拿当前指令地址加偏移量必错 · 无符号地址与有符号偏移量不能直接按补码相加，先求真值 · 区间里有多少个数别忘 +1，向后是 127 条不是 128 条

机器码与汇编互译

真题年份 | 2023/2024（第 43/44 题） | 新大纲机器级表示，2023 起连续两年出现，5–8 分

必背结论

- 一道汇编语句 = 操作码 + 若干「寻址特征位 + 寄存器号/地址码」字段，字段宽度与顺序全部按题目给的格式表，不按书上。
- 拼机器码只做两件事：按格式表逐字段填二进制，再每 4 位一组转十六进制；先补足位数再分组。
- 反方向（机器码→汇编）先按同一张格式表切位，再查操作码表与寻址方式表还原助记符，最后按括号规则写成汇编。
- 操作数读法只看括号：写 R4 是寄存器（数在寄存器里），写 (R4) 是寄存器间接（寄存器里放的是地址），写 ((R4)) 再取一层。
- 大小端判据只看「字节落在哪个地址」：低字节放低地址 = 小端，高字节放低地址 = 大端。
- 判断步骤：把数写成 4 个字节（高位在前），再看内存从低地址到高地址给的字节序列与哪个顺序一致。
- 2024 年「字节重组」口径：先定大小端，再按操作数宽度（1/2/4 字节）从内存取够字节，按地址由低到高拼回一个整数，别把字节序当数值序抄。
- 2010 年「各类寻址机器码」口径：操作码 4 位 + 两个操作数各 3 位寻址方式 + 3 位寄存器号，凑满 16 位再转十六进制。
- 2023 年把跳转与大小端放在一起考：偏移量字段落在指令的高字节还是低字节，取决于指令编码的字节序，取字段前先定序。
- 定长与不定长是判 RISC 与 CISC 的唯一判据：指令长度固定→精简指令集；长度不定→复杂指令集。

模板/流水线

```
/* 机器码 <-> 汇编：按格式表切位，别自己发明字段宽度 */
/* 例：16 位定长，OP(4) MOD(3) R(3) MOD(3) R(3) */
unsigned short code = 0;
code |= (op & 0xF) << 12;                   /* 操作码 4 位 */
code |= (md1 & 0x7) << 9;                   /* 目的操作数寻址方式 3 位 */
code |= (r1 & 0x7) << 6;                    /* 目的寄存器号 3 位 */
code |= (md2 & 0x7) << 3;                   /* 源操作数寻址方式 3 位 */
code |= (r2 & 0x7);                         /* 源寄存器号 3 位 */
printf("%04XH\n", code);                    /* 每 4 位一组转十六进制 */
```

```
/* 反方向：先切位，再由表还原助记符 */
op  = (code >> 12) & 0xF;  md1 = (code >> 9) & 0x7;  r1 = (code >> 6) & 0x7;
md2 = (code >> 3)  & 0x7;  r2  =  code       & 0x7;
/* 大小端：把 x 拆成 4 个字节，看最低地址放的是谁 */
unsigned char b[4];
b[0] = x & 0xFF; b[1] = (x >> 8) & 0xFF;
b[2] = (x >> 16) & 0xFF; b[3] = (x >> 24) & 0xFF;
/* b[0] 落在最低地址 => 小端；b[3] 落在最低地址 => 大端 */
```

寻址方式位 3 位最多 8 种方式，别和 4 位操作码混着数 · 大小端看的是字节在内存里的先后，不是数值书写顺序 · 结果写回的是「目的操作数」，寄存器自增、指针回写这类隐含动作别漏

汇编循环三件套

真题年份 | 2014/2019（第 43/44 题） | 循环与递归汇编三年两考，和相对寻址一起给分

必背结论

- 循环三件套：入口（标号地址）+ 计数器（或指针）递减 + 条件转移回入口。缺一件就是死循环或漏一次。
- 计数器型循环：先置初值 n，循环体末尾 DEC（或 SUB 置零标志），再用 JNE 回跳；判据是上一条指令设的标志位。
- 指针型循环（数组）：基址或变址寄存器当指针，按字节编址时每次 ADD R,4（元素 4 字节），按字编址时加 1。
- 循环体执行 n 次、出口转移执行 n 次（最后一次不跳）；顶部先判的 while 型，判断要执行 $n+1$ 次。
- 递归：CALL 压栈、RET 弹栈；调用次数 = 递归深度（$f(10)$ 依次算 10,9, ⋯,1，共 10 次调用）。
- 数组求和套路：R0 清零当累加器、R1 存计数、R2 存指针，循环体 ADD R0,(R2) → ADD R2,4 → DEC R1 → JNE 回跳。
- 一定会让程序跳转执行的三条：CALL、RET、无条件 JMP；Jcc 要看标志位，条件不成立就顺序执行。
- 循环结束条件写反最致命：计数器递减用 JNE（不为零继续），计数器递增用 JNE 也行、但配 CMP 判上界时要看清是 JLE 还是 JGE。
- 2014 年循环汇编还要接着算流水线：循环体里相邻两条若构成写后读且间隔不足 3 条，每轮都要阻塞，轮数 ×阻塞拍数才是总代价。
- 数组循环里首地址与下标分工：形式地址 A 放首地址、变址寄存器放下标，取第 i 个元素先算 $A+i\times$ 元素大小；改成指针写法时指针初值才是首地址，每轮加元素大小。

模板/流水线

```
/* 汇编循环模板：先写 C，再逐句译，别跳过计数器 */
int s = 0;                      /* MOV R0, 0       ; 累加器清零    */
int i = n;                      /* MOV R1, n       ; 计数器 = 次数 */
loop:                           /* L1:             ; 循环入口地址 */
    s = s + a[i - 1];           /* ADD R0, (R2)    ; 取元素累加    */
    i = i - 1;                  /* DEC R1          ; 计数器减一    */
    p = p - 4;                  /* SUB R2, 4       ; 指针前移一个元素 */
    if (i != 0) goto loop;      /* JNE L1          ; 非零回跳      */
/* 三件套自查：① 有入口标号 ② 计数器递减 ③ 条件转移回入口 */
/* 按字节编址加元素字节数，按字编址加 1，别加元素边长 */
/* 递归版：CALL 前参数入栈，RET 后取返回值；调用次数 = 递归深度 */
```

条件转移的判据是「上一条指令设置的标志位」，CMP 之后若插了不设标志的指令判断就废了 · 按字节编址时指针加的是元素字节数不是元素个数 · 循环体次数与转移执行次数不一定相等，先判后做的会多判一次

单总线取指周期微操作

真题年份 | 2009/2015（第 43/44 题） | 数据通路大题的开局送分题，默写即得分

必背结论

- 取指周期四步，顺序背死：(PC) → MAR、M(MAR) → MDR、(MDR) → IR、(PC)+1 → PC。

- 顺序不能颠倒的理由：地址不进 MAR 就发不出访存，MDR 没拿到数就送不进 IR；PC 自增必须晚于 PC → MAR。
- 单总线铁律：一条 CPU 内部总线，一个时钟只能传一次数据；走内部总线的两步必须分两个时钟。
- 能合并的唯一判据：两步走的是两条不同总线（一步 CPU 内部总线、一步系统总线）才允许同拍。
- 访存必经 MAR 与 MDR，任何「直接 M → IR」的写法都是错的。
- 控制信号的 out/in 语义：数据流出谁就谁 out，流入谁就谁 in；一拍 = 一个 out 加一个 in。
- 执行阶段范例（2009 年 ADD (R1),R0，功能 $(R0)+((R1)) \rightarrow (R1)$）：6 拍；把「R0 → A」与「读内存→ MDR」合并后 5 拍。
- 系统总线三根缩写要认：CB 控制总线、DB 数据总线、AB 地址总线；取指时地址从 AB 出去、数据从 DB 回到 MDR。
- 单总线与多总线的区别只影响「一拍能传几个数据」：题目给一条总线就一拍一步写，给多总线才敢并行。
- 同一段微操作在两种总线上写法不同：单总线要求每拍只出现一个 out 与一个 in，答题前先判题目给的是哪一种，合并才有依据。

取指四步 + 「访存必经 MAR/MDR」+ 「ALU 不存储」，这三句覆盖数据通路题一半的判分点。

模板/流水线

```
/* 取指周期：单总线，一个时钟只走一次内部总线 */
/* C1 */ (PC) -> MAR       /* PCout=1, MARin=1              */
/* C2 */ M(MAR) -> MDR     /* 读命令 =1, MDRinE=1            */
/* C2 */ (PC) + 1 -> PC    /* 并行：走的是另一条总线，可同拍 */
/* C3 */ (MDR) -> IR       /* MDRout=1, IRin=1              */
/* 若题目只给内部总线一组信号：C2 只写读命令，不写 MDRin */
/* 执行阶段 ADD (R1), R0：功能 (R0)+((R1)) -> (R1) */
/* 1 */ (R0) -> A          /* R0out, Ain                     */
/* 2 */ (R1) -> MAR        /* R1out, MARin                   */
/* 3 */ M(MAR) -> MDR      /* 读命令，MDRinE                 */
/* 4 */ (A) + (MDR) -> AC  /* ACin（ALU 不写 out）           */
/* 5 */ (AC) -> MDR        /* ACout, MDRin                   */
/* 6 */ (MDR) -> M          /* MDRoutE, 写命令               */
/* 优化：第 1 步与第 3 步合并（CPU 总线 vs 系统总线）=> 5 拍 */
```

微操作顺序错一步全盘错：PC → MAR 必在访存之前，MDR → IR 必在 MDR 拿到数之后 · 同一拍里排两次内部总线传输直接判错 · ALU 不存储，运算前先占住一个操作数、结果先落暂存器或 ACC

控制信号表怎么写

真题年份 | 2022/2026（第 43/44 题） | 数据通路题的第二问，写法固定，按行给分

必背结论

- 一行一条微操作，左边写功能、右边写控制信号；题目要控制信号就必须写控制信号，只写微操作等于没答。
- 生成规则三条：数据流出谁写谁 out、流入谁写谁 in、其余部件不写。
- 与内存打交道的两个专用信号：MDRinE（内存数据线→ MDR）、MDRoutE（MDR →内存数据线），带 E 的是走系统总线那一侧。
- 同名信号看方向：MDRin 是内部总线→ MDR，MDRinE 是内存→ MDR，混用是这一问最大的坑。
- 读/写命令必须单列：只写 MDRinE 不写读命令，存储器不知道要读。
- ALU 参与运算的那条微操作只写目的寄存器的 in（A、MDR 是 ALU 的固定输入端，没有传输动作）。
- 为什么设暂存器（2022 年问）：ALU 无存储功能，两个操作数要先进暂存器，结果也要先落暂存器或 ACC 再上总线，否则输出又接回输入会互相干扰。
- 2026 年「扩展器控制信号」：译码器把 n 位编号映射成 2^n 路选通（多对多）；多路选择器从多路里选一路输出；三态门置 1 才让数据通过，置 0 就阻断。
- 控制信号由 CU 产生，依据是 IR 里的操作码；地址码则送去寄存器堆或 MAR 负责取数，两半分工要写清。

- 信号名以题目所给表格为准：不同教材对同一动作命名不同，语义对、方向对就给分，但 out/in 反了必错。

微操作	需要有效的控制信号
(PC) → MAR	PCout、MARin
M(MAR) → MDR	读命令、MDRinE
(MDR) → IR	MDRout、IRin
(R0) → A	R0out、Ain
(A)+(MDR) → AC	ACin（ALU 输入端固定）
(AC) → MDR	ACout、MDRin
(MDR) → M	MDRoutE、写命令
(PC)+1 → PC	PCin（或自增信号）

模板/流水线

```
/* 控制信号表模板：一行一条微操作，先功能后信号 */
/* 判据：流出谁 -> 谁 out；流入谁 -> 谁 in；ALU 不进 out/in 位置 */
/* 1 */ (PC) -> MAR        /* PCout, MARin          */
/* 2 */ M(MAR) -> MDR      /* 读命令, MDRinE        */
/* 3 */ (MDR) -> IR        /* MDRout, IRin          */
/* 4 */ (PC) + 1 -> PC     /* PCin（或 +1 信号）    */
/* 访存类别忘了读/写命令；带 E 的信号 = 与内存打交道那一侧 */
/* 问「为什么设暂存器」：ALU 没有存储功能 + 防止输出回灌输入 */
/* 问「控制信号谁产生」：CU，依据是指令的操作码（地址码去取数） */
```

题目只给一组内部总线信号时写 MDRin 就是错，那是 MDRinE 的活 · 只写微操作不写控制信号，这一问直接白给 · 读/写命令漏写，存储器不知道要读还是要写

流水线五段与三类相关

真题年份 | 2012/2014（第 43/44 题） | 五段流水线隔年考，阻塞周期数必给分

必背结论

- 五段：IF 取指 | ID 译码且读寄存器 | EX 执行 | MEM 访存 | WB 写回；每个流水段后面都有流水段寄存器，锁存本段结果供下一段用。
- 三类指令在 EX/MEM 的差别必须背（见下表）：这是判阻塞的地基，尤其「转移类在 MEM 段才把目的地址送进 PC」。
- 结构相关 = 资源冲突：同一时刻两条指令抢同一部件，典型是 IF 访存撞上访存类的 MEM；排图时推迟 IF，根本解法是指令 Cache 与数据 Cache 分离。
- 数据相关 = 写后读（RAW）：后面要用前面还没产生的结果；解法是阻塞、插入空操作、数据旁路（转发）。
- load-use 特例：前一条是 load，数据在 MEM 段末才拿到，转发也救不了，必须「延迟 + 转发」，即后一条 EX 排在前一条 MEM 之后。
- 控制相关：转移指令让后面已取的指令可能白做；解法是尽早算出转移目标地址，且下一条的 IF 排在转移指令的 MEM 之后。
- 判数据相关不能只看写后读，要看间隔：WB 在第 5 拍、ID 在第 2 拍，差 3 拍，中间隔 ≥ 3 条指令就没有冒险。
- 阻塞周期数：$T = ((K + N - 1) + \text{阻塞拍数}) \times \Delta t$；问「多少个时钟周期」只答拍数，不乘 Δt。
- 2012 年实算：四条无相关指令进五段流水线 $= 5 + 4 - 1 = 8$ 个时钟周期，问的是拍数不是时间。
- 2023 年那道「已转发 + 已硬件阻塞」还剩谁被阻塞：逐条卡下面四条约束，被卡住的是 load 后面那条、以及前面是转移指令那条，前两条正常。
- 三种冒险的修复手段别写串：分离 Cache 治结构、转发治数据、早算目标地址治控制。

指令类别	EX 阶段	MEM 阶段	WB 阶段
计算类（ALU）	运算	空，什么都不做	写回寄存器
访存类（load/store）	算有效地址	访存（读或写）	读出的数写回寄存器
转移类（分支）	算转移目的地址	目的地址送进 PC	无

模板/流水线

```
/* 五段：IF ID EX MEM WB；第 i 条指令各段比上一条晚 1 拍 */
/* 后一条 IF 在第 p 拍，则 ID 在 p+1、EX 在 p+2、MEM 在 p+3、WB 在 p+4 */
int K = 5, N = 4;
int cycles = K + N - 1;          /* 无相关：5+4-1 = 8 拍 */
/* 阻塞判据（逐条卡）：
   未用转发 + 写后读 -> 后一条 ID 必须 >= 前一条 WB
   已用转发          -> 后一条 EX 必须 >= 前一条 EX
   前一条是 load     -> 后一条 EX 必须 >= 前一条 MEM
   前一条是转移指令  -> 后一条 IF 必须 >= 前一条 MEM
   任何情况          -> 后一条 IF 不早于前一条 ID          */
cycles += stall;                 /* stall = 为满足以上约束多等的拍数 */
/* 若问时间：T = cycles * dt，dt = max(各段时间) + 流水段寄存器延迟 */
```

只看「写后读」就断言有数据冒险，中间隔够 3 条其实没事 · 前一条是 load 时仍只保证「EX 在 EX 之后」必错 · 上一条是转移指令还把 IF 提到 MEM 之前，整张时空图就废了

流水线性能三公式

真题年份 | 2012/2014（第 43/44 题） | 加速比与吞吐率大题未单独设问，是流水线题的公式底座

必背结论

- K 段流水线执行 N 条指令：$T=(K+N-1)\Delta t$，这是一切小问的起点。
- 吞吐率 $TP=\dfrac{N}{(K+N-1)\Delta t}$，最大吞吐率 $TP_{max}=\dfrac{1}{\Delta t}$（$N$ 很大时趋近）。
- 加速比 $S=\dfrac{NK\Delta t}{(K+N-1)\Delta t}=\dfrac{NK}{K+N-1}$，最大加速比 $S_{max}=K$，上限就是段数。
- 效率 $E=\dfrac{S}{K}=\dfrac{N}{K+N-1}$，$N$ 越大越接近 1。
- 时钟周期不是各段平均值：$\Delta t=\max(t_i)+t_{\text{流水段寄存器}}$，取最长段再加锁存延迟。
- 有时空图先数拍：横轴时钟、纵轴流水段，一条指令占一条阶梯斜线；空格就是阻塞拍，总拍数 = 最后一行的结束列。
- 有阻塞时把 $K+N-1$ 换成实际拍数，四个公式同步改，别只改一个。
- 时空图画法：横向一格一个周期、纵向一行一个流水段，第 i 条指令的 IF 画第 i 列，其余段依次右移一格，相邻指令同一段错开一格成阶梯。
- 加速比公式只有在各段等长时才直接成立；各段不等长要先把时钟周期取成最长段再加锁存延迟，再套同一式子。
- 流水线装满之前有空闲段：$N<K$ 时效率低于 50%、加速比也远不到 K；题目把 N 给得很大时，可直接按 $N\to\infty$ 取上限答。

T、TP、S、E 四个量共用 $(K+N-1)$，先把拍数算准，其余套公式，不会连环错。

模板/流水线

```
/* k 段、n 条、一个时钟 dt */
double T   = (k + n - 1) * dt;          /* 总时间              */
double TP  = n / T;                     /* 实际吞吐率          */
double TPm = 1.0 / dt;                  /* 最大吞吐率          */
double S   = (double)n * k * dt / T;    /* 加速比 = nk/(k+n-1) */
double Sm  = k;                         /* 最大加速比 = 段数   */
```

```
double E   = S / k;                          /* 效率 = n/(k+n-1)   */
/* 时钟周期: dt = max(各段时间) + 流水段寄存器延迟 */
/* 有阻塞: 把 (k+n-1) 换成实际拍数 k+n-1+stall, 四式同步改 */
/* 问「多少个时钟周期」只答拍数, 问「时间」才乘 dt */
```

加速比上限是段数 K，不是无限大 · 时钟周期取最长段再加锁存延迟，取平均必错 · N 很小时 S 远小于 K，别把 S_{max} 当实际加速比填

三种 I/O 方式总表

真题年份 | 2018/2009 | 三种方式横向对比是本章最标准的三问式大题，8-9 分，一次问三小问

必背结论

- 程序查询：CPU 反复读状态端口，每传一个字都由 CPU 搬，CPU 参与 100%，三种里效果最差。
- 程序中断：外设放好数据后主动请求，每传一个字中断一次；数据仍经 CPU（服务程序用 load/store 搬）。
- DMA：外设在内存与设备间直传，数据不经过 CPU；只有预处理与后处理用到 CPU。
- 传送单位：查询与中断都是「字」(32 位 = 4B)；DMA 停止 CPU 访存是「块」(512B)，周期挪用是「字」。
- 能不能用中断只有一条判据：外设准备数据的时间必须大于中断处理（读走数据）的时间。
- 查询又分两种：持续查询把 CPU 全部时间花在 I/O 上；周期性（定时）查询按外设速率定间隔，占用率降下来但仍是浪费。
- 选型次序是 DMA →中断→查询，按一次要搬的数据量和设备速度从大到小排。

方式	CPU 参与程度	数据传输单位	是否经过 CPU	适用场合
程序查询	全程忙等，100% 被占	字（每次 1 个字）	经过	极低速、开关量、简单设备
程序中断	只占中断响应与服务程序	字（每次 1 个字）	经过	低速、随机出现的设备（键盘、打印机）
DMA	只占预处理与后处理	块（停止 CPU 访存）或字（周期挪用）	不经过	高速、成块传送（磁盘、显卡、网卡、声卡）

三种方式的差别只落在两处：数据经不经过 CPU；CPU 是全程陪着还是只在头尾露个面。

模板/流水线

```
/* 选型三判据, 按顺序问, 问到一个就停 */
if (数据率 >= 1e6 && 每次按块传送)        方式 = DMA;   // 磁盘/网卡/显卡/声卡
else if (准备数据时间 > 中断处理时间)     方式 = 中断;  // 键盘/打印机/鼠标
else                                      方式 = 查询;  // 开关量、极低速设备
/* 中断可用性的等价写法: n * (中断响应 + 服务程序) < 1 秒,
   n 为每秒中断次数; 不满足就是" 读不过来", 必须换 DMA */
```

说「中断方式不经过 CPU」是硬错，只有 DMA 不经过 · 别把键盘当高速设备，判高速看数据量不看手速 · DMA 也不是完全不占 CPU，预处理与后处理照样要 CPU

程序查询方式时间计算

真题年份 | 2018 | 查询只作为三问里的第一问，4 分，很少单独成题

必背结论

- 每秒查询次数 $n = R/B$，R 是数据传输率（B/s），B 是每次传送字节数；32 位缓冲即 $B = 4$。
- 最长查询间隔 $T = 1/n$；实际查询间隔必须 $\leq T$，否则数据被下一次覆盖。
- 每次查询开销 $C =$ 指令条数 $\times$ CPI。

- CPU 用于 I/O 的比例 $P = \dfrac{n \times C}{f} \times 100\%$，$f$ 是主频（每秒时钟数）。
- 查询间隔必须小于等于外设两次传送的间隔，否则数据缓冲寄存器被下一次数据覆盖，丢数据。
- 查询搬运一个数据也要 load 加 store，所以「至少执行 10 条指令」这种题面数字原样代入，不要自己改。
- 2018 第 43 题 (1)：$R = 2$ MB/s，$B = 4$，10 条指令，CPI 4，$f = 500$ MHz，得 $n = 5 \times 10^5$，$T = 2\,\mu$s，$C = 40$，$P = 4\%$。

模板/流水线

```
/* 查询方式四步，一行一个量，别跳 */
double Query(double R, double B, long ins, long cpi, double f) {
    long   n = (long)(R / B);        // 每秒查询次数
    double T = 1.0 / n;              // 最长查询间隔 (s)，写上"<= T" 才满分
    long   C = ins * cpi;            // 每次查询的时钟数
    return 100.0 * n * C / f;        // 占用率 (%)
}
/* 2018-43(1): Query(2e6, 4, 10, 4, 5e8)
   n=5e5 次/s，T=2 μ s，C=40，P=4% */
```

查询题的三件套：次数、间隔、开销。间隔是次数的倒数，别写成次数的倍数。

MB/s 在数据传输率里按 10 的幂走，2MB/s 就是 2×10^6 B/s · 32 位先化成 4B 再去除字节率，别拿 32 去除 · 问「最多间隔多久查一次」答的是上界，符号写 $\leq$

中断响应全过程与传输时间

真题年份 | 2016/2018/2009 | 十年三考，本章最高频，必练

必背结论

- 中断响应又叫中断隐指令，由硬件完成，不是真正的指令。
- 必背 5 步：①关中断②保存断点（PC，有的说法含 PSW）③识别中断源、取中断类型号④查中断向量表得中断向量（= 服务程序入口地址）⑤转去执行中断服务程序。
- 断点取谁：中断取「下一条指令地址」，异常取「当前指令地址」。
- 三个概念连成一条链：中断类型号（数据线送来）→查中断向量表→得中断向量，即服务程序入口地址。
- 中断请求与响应走控制线，中断类型号走数据线；这是唯一一处「响应在控制线、编号在数据线」的分工。
- 中断处理共 9 步：1-3 由硬件（即上面三步）；4-9 由服务程序做：保护现场、开中断、设备服务、关中断、恢复现场、中断返回。
- 完成这一任务的时间 = 外设准备时间 + 中断响应时间 + 服务程序前 k 条指令的时间（跑到启动设备那条为止）。
- CPU 用于该任务的时间 = 中断响应时间 + 整条服务程序的时间（n 条指令，不算外设准备）。
- 中断方式的开销就三块：中断响应（硬件，十来个时钟）、服务程序（指令数乘 CPI）、以及被中断程序的现场保存与恢复，后者已含在响应里。
- 服务程序第 k 条指令启动设备，下一轮准备与本次服务程序的后半段重叠，所以问「完成这一任务」时只算到第 k 条。
- 2009 第 43 题 (1)：0.5 MB/s、32 位为单位、服务程序 18 条加其他开销 2 条、CPI 5、500 MHz，得 $n = 1.25 \times 10^5$，$C = (18 + 2) \times 5 = 100$，$P = 2.5\%$。
- 2016 第 44 题：0.5 ms 折合 $0.5\text{ms}/20\text{ns} = 25000$ 个时钟，响应 10 个时钟，15 条 $\times$ 4 = 60，每字符 25070；1000 字符共 2.507×10^7 个时钟；CPU 只花 $(10 + 20 \times 4) \times 1000 = 9 \times 10^4$ 个时钟。

模板/流水线

```
/* 两个问法，只差一个"CPU" */
long all_task(long t_pre, long t_resp, long ins, long cpi, long k) {
    long run = ins * cpi;                  // 服务程序跑到" 启动设备" 那条为止: 15 条
    return k * (t_pre + t_resp + run);     // 问①完成这一任务
```

```
}
long cpu_only(long t_resp, long ins_all, long cpi, long k) {
    return k * (t_resp + ins_all * cpi);   // 问② CPU 用于这一任务：整 20 条
}
/* 2016-44(2)：t_pre = 0.5ms / 20ns = 25000，t_resp = 10，k = 1000
   问① 1000*(25000+10+15*4) = 2.507e7 个时钟
   问② 1000*(10+20*4)       = 9e4   个时钟  */
```

判「能不能用中断」：准备数据的时间小于中断处理时间就是读不过来。2018 第 43 题 (2) 设备 B 每 0.1 μs 来一个数据、中断一次要 0.8 μs，故设备 B 不能采用中断。

异步串行算位数要加 1 位起始位，7+1+1+1=10 位，只写 9 位丢一半 · 问①用 15 条、问②用 20 条，两问一混全错 ·「其他开销」就是中断响应的开销，不是新东西

中断嵌套、优先级与屏蔽字

真题年份 | 选择题常考，大题未考（真题年份总表未登记本题型） | 服务于 2009/2016/2018 那类中断概念问答题

必背结论

- 高优先级中断可以打断低优先级中断的执行，叫多重中断（中断嵌套）。
- 屏蔽字一位对应一个中断源：写 1 表示屏蔽（不响应），写 0 表示允许。
- 处理某级中断时，屏蔽字里「优先级低于或等于自己」的位全置 1，「高于自己」的位全置 0；自己的位恒为 1。
- 全 1 的屏蔽字对应最高优先级，它能打断别人，谁也打断不了它。
- 嵌套深度 = 能打断自己的中断源个数 + 1；4 级中断 A>B>C>D 的深度分别是 1、2、3、4。
- 响应要三条同时满足：当前指令执行完、中断允许触发器为 1（开中断）、没有更高级的请求。
- 所以关中断的区间越长，响应越迟；保存断点必须放在关中断之后、开中断之前。

正在处理	屏蔽字 A B C D	嵌套深度
A（最高）	1 1 1 1	1
B	0 1 1 1	2
C	0 0 1 1	3
D（最低）	0 0 0 1	4

模板/流水线

```
/* 屏蔽字模板：n 级中断，编号 0 为最高优先级 */
int mask[N][N], i, j;
for (i = 0; i < N; i++)              // i = 正在处理的那一级
    for (j = 0; j < N; j++)          // j = 其它中断源
        mask[i][j] = (j >= i);       // 自己不高于它 → 置 1 屏蔽
/* A:1111  B:0111  C:0011  D:0001
   画表时先把自己的那一位置 1，右边（更低级）全填 1，左边（更高级）全填 0 */
```

屏蔽字 1 是「关」不是「开」，这是全国统一的反直觉写法：置 1 = 屏蔽该源。

屏蔽字和自己的中断允许触发器是两层，别混为一谈 · 自己的位必须是 1，漏掉就变成自己打断自己 · 嵌套深度算的是能打断自己的源数加一，不是优先级序号

DMA 传送流程与争总线

真题年份 | 2022/2018/2012（DMA 优先级小问） | 优先级问法十年两现，必背两条理由

必背结论

- DMA 三阶段：预处理（CPU 设方向、首址、长度并启动设备）→数据传送（硬件，整块不用 CPU）→后处理（DMA 发中断，CPU 校验数据）。

- 停止 CPU 访存：单位是块（512B），整块传完才释放总线，CPU 在此期间完全停摆，适合块大且一次连续的场合。
- 周期挪用：单位是字（32 位 = 4B），每传完一个字立刻释放总线；CPU 不访存就直接拿，CPU 正在用就等它用完，同时请求则优先 DMA。
- 交替访存：CPU 与 DMA 分时轮流占用总线（一个周期给 CPU、一个周期给 DMA），不需要总线请求，效率最高但硬件最复杂（教材口径）。
- 预处理要写四件事：传送方向、主存首址、传送长度、启动设备；四件都由 CPU 执行，这是 DMA 唯一像中断的地方。
- 发生时机：中断只能在一条指令执行结束后响应，DMA 靠硬件，任意一个总线周期用完就能接管总线。
- DMA 优先级高于 CPU，也高于中断；答「谁获得总线」必须写两条理由：DMA 优先级高、数据必须尽快入内存否则被下一次覆盖丢失。

方式	传送单位	释放总线时机	对 CPU 的影响
停止 CPU 访存	块（512B）	整块传完才释放	CPU 完全停顿，适合块大且连续的场合
周期挪用	字（32 位）	每传完一个字立刻释放	只偷一个存储周期，CPU 基本不停
交替访存	按周期分时	固定分给 DMA 一半周期	CPU 与 DMA 各占一半，无请求开销

模板/流水线

```
/* DMA 三阶段与请求次数 */
// 1 预处理：CPU 写方向/首址/长度，启动设备          —— 占 CPU
// 2 传送  ：设备就绪 -> DMAC 发总线请求 -> 传一个单位 -> 释放   —— 硬件，不占 CPU
// 3 后处理：DMAC 发中断，CPU 校验                          —— 占 CPU
int unit = (mode == 0) ? 512 : 4;       // 停止 CPU 访存取块 512B；周期挪用取字 4B
long req = total_bytes / unit;          // 总线请求次数，每传一个单位发一次
/* 2022-44(3)：一个扇区 512B，缓冲区 64 位 = 8B -> 512/8 = 64 次 */
```

问「CPU 与 DMA 同时访存谁获得总线」，只答「DMA 优先级高」等于没答，必须补上「数据要尽快放入主存，否则缓冲区被下次数据覆盖」。

周期挪用的单位是字不是块，拿 512B 去算请求次数必错 · 把「DMA 高于 CPU」和「DMA 高于中断」搞混，两个都是 DMA 高 · 别写 DMA 完全不占 CPU，预处理和后处理要占

DMA 传送时间与 CPU 占用率

真题年份 | 2018/2009/2022 | 与中断问题对比，两种方式必考其一

必背结论

- 每秒 DMA 次数 $n = R/B_b$，R 是数据率（B/s），B_b 是每块字节数。
- CPU 占用率 $P = \dfrac{n \times C_c}{f} \times 100\%$，$C_c$ 是预处理加后处理的总时钟数。
- DMA 本身传送时间 $t = D/R$（D 为总字节数），这段时间 CPU 不被占用，可继续执行程序。
- 总线请求次数 = 一次传送总量 ÷ 每次传送单位，每传一个单位发一次；2022 第 44 题 (3)：512B ÷ 8B = 64 次。
- 2018 第 43 题 (3)：$40 \times 10^6/1000 = 4 \times 10^4$ 次/s，$\times 500 = 2 \times 10^7$，$\div 5 \times 10^8 = 4\%$。
- 2009 第 43 题 (2)：$5 \times 10^6/5000 = 1000$ 次/s，$\times 500 = 5 \times 10^5$，$\div 5 \times 10^8 = 0.1\%$。

模板/流水线

```
/* DMA 占用率：分子只有预处理 + 后处理 */
double DmaRate(double R, double blk, long c, double f) {
    long n = (long)(R / blk);        // 每秒 DMA 次数
    return 100.0 * n * c / f;        // 百分比
}
```

```
/* 2018-43(3): DmaRate(40e6, 1000, 500, 5e8) = 4%
   2009-43(2): DmaRate( 5e6, 5000, 500, 5e8) = 0.1%
    | 块大小题给 1000B 就用 1000，不要擅自换成 1KB = 1024B */

long DmaReq(long total, long unit) { // 总线请求次数
    return total / unit;             // 2022: 512 / 8 = 64
}
```

DMA 的占用率永远是三种方式里最低的，因为它把「搬运」这件最费时的活交给了硬件。

1KB 与 1000B 差 24 字节，题面给 1000B 时按 1000 算，否则数值对不上 · 占用率的分母是主频（每秒时钟数），不是 CPI · 周期挪用的请求次数按数据缓冲区宽度算，不是按主存字长算

磁盘参数计算

真题年份 | 2022 | 组原近年少见的磁盘计算大题，一整题多问都从磁盘参数出发

必背结论

- 容量 = 盘面数 × 每面磁道数 × 每道扇区数 × 每扇区字节数；盘面数 = 盘片数 ×2。
- 地址三字段位数各取 $\lceil \log_2 N \rceil$（向上取整）：柱面号（= 磁道号）、盘面号（= 磁头号）、扇区号；定位顺序也是柱面→盘面→扇区。
- 按柱面优先编址的好处：换柱面要动机械臂最慢，同一柱面换磁头只需电子切换，所以地址让柱面号在高位。
- 一圈时间 $T = 60/r$，r 为转速（r/min）；7200 r/min 得 8.33 ms。
- 平均存取时间 = 平均寻道时间 + $T/2$ + T/N，N 为每道扇区数；旋转延迟按半圈算，传输时间按占一圈的比例算。
- 数据传输率 = 每道容量 × 每秒转数。
- 一个扇区的传输时间也可以写成每扇区字节数除以数据传输率，与按角度比例算的结果一致，用哪个顺手用哪个。
- 2022 第 44 题：4 个双面盘片得 8 面（3 位）、20000 磁道（15 位）、500 扇区（9 位）；平均存取 = 5+4.17+0.017 ≈ 9.18 ms；容量 = $8 \times 20000 \times 500 \times 512$ B ≈ 40.96 GB；传输率 = $500 \times 512 \times 120 = 3.072 \times 10^7$ B/s。

模板/流水线

```
/* 磁盘五算，全部由" 一圈时间" 派生 */
long   faces = platter * 2;                    // 4 双面盘片 -> 8 面
long   cap   = faces * track * sector * 512L;  // 容量 (B)
int    cyb   = ceil_log2(track);               // 柱面/磁道 15 位
int    hdb   = ceil_log2(faces);               // 盘面/磁头  3 位
int    scb   = ceil_log2(sector);              // 扇区       9 位
double T     = 60.0 / rpm;                     // 一圈 (ms): 60/7200 = 8.33
double avg   = seek + T / 2 + T / sector;      // 平均存取 ≈ 5+4.17+0.017 = 9.18ms
double rate  = sector * 512.0 * (rpm / 60.0);  // 传输率 3.072e7 B/s
```

旋转延迟一律按半圈，理由是最靠前的位置一找就到、最靠后的要转一整圈，平均下来正好半圈。

盘面数是盘片数的两倍，4 个双面盘片是 8 面不是 4 面 · 柱面号等于每面磁道数，别把总磁道数再乘盘面 · 寻道时间题面直接给，不用自己算

三种方式 CPU 开销对比模板

真题年份 | 2018/2009 | 「占多少百分比」是本章最常见问法，一问 2-4 分

必背结论

- 一个公式打天下：$P = \dfrac{n \times C}{f} \times 100\%$，$n$ 是每秒 I/O 次数，C 是每次的 CPU 开销时钟数，f 是主频。
- 三种方式只有 C 不同：查询 C = 指令数 × CPI；中断 C = 中断响应时钟 + 服务程序指令数 × CPI；DMA C = 预处理 + 后处理时钟。

- n 都要自己求：n = 数据率 ÷ 每次传送字节数，注意 1 B = 8 位。
- 同题对比结论：DMA 占用率远低于中断；2009 第 43 题同一外设，中断 2.5%、DMA 0.1%，差 25 倍。
- 数据率越高，中断方式越撑不住（每秒次数线性涨），这时只能换 DMA，所以同一题常在 (2) 问逼你换 DMA。
- 答题就写三行：n 等于多少、C 等于多少、nC 除以主频乘 100%，不写文字过程。
- 数值相同不代表含义相同：2018 第 43 题设备 A 查询得 4%、设备 B DMA 也得 4%，两问彼此独立，别互相代入。

方式	每次开销 C 的组成	每秒次数 n 的算法
程序查询	每次查询的指令数乘 CPI	数据率除每次传送字节数
程序中断	中断响应时钟加服务程序全部时钟	数据率除每次传送字节数
DMA	预处理时钟加后处理时钟	数据率除每块字节数

模板/流水线

```
/* 一张表通吃三种方式：只有 c 不同 */
double Cost(double R, double B, long c, double f) {
    long n = (long)(R / B);              // 每秒 I/O 次数
    return 100.0 * n * c / f;            // 占用率（%）
}
// 查询：c = 指令数 * CPI           2018 设备 A：Cost(2e6, 4, 10*4, 5e8) = 4%
// 中断：c = 响应 + 服务程序时钟    2009:    Cost(0.5e6, 4, 100, 5e8) = 2.5%
// DMA ：c = 预处理 + 后处理时钟    2009:    Cost(5e6, 5000, 500, 5e8) = 0.1%
```

考场只写三行：n 等于多少、C 等于多少、nC 除主频乘百分之百。过程分极少，结果必须写单位。

主频 MHz 是 10^6，500 MHz 要写成 5×10^8 才不出错 · 百分比最后一步别忘乘 100 · 中断的「其他开销」就是中断响应，漏掉它 2009 那题会算成 2.25%

第三编

操作系统

综合应用题第 45、46 题，共 15 分 | PV 与内存/文件，每年一难一易

PV 大题五步流程

真题年份 | 2009/2011/2013/2014/2015/2017/2019/2020/2021/2022/2023/2024/2025（第 45 或 46 题） | 16 年里考了 13 次，只有 2010/2012/2016/2018 四年的 OS 大题位置被别的考点占掉；7–8 分

必背结论

- 判分只看三个点：定义了几个信号量、每个的初值、P/V 摆的位置。分号、花括号、`while` 全是形式分。
- 五步顺序不能乱：数资源→定信号量→写进程骨架→摆 P/V →检查。
- 总口诀：互斥初值 **1**，同步初值 **0**，资源量初值 **n**；先发生的后面 **V**，后发生的前面 **P**。
- 卷面统一写 `P()/V()`。题面若用 `wait()/signal()`，全文替换为 `wait=P`、`signal=V`，一份答案里绝不许混写两种。
- 题面里的「等」「必须等到」「才能」「若有…则」四个字眼，一律落成 P 操作。
- 动笔前先画图：把约束画成前驱图或资源流向图，图画完信号量个数就自己浮出来了。

答题纸先写信号量声明区（含每个信号量的含义注释），再写进程区——顺序反了容易写着写着漏一个。

模板/流水线

步	你要做的动作	落笔成什么
①数资源	圈出题里所有「一台/一个/N 个」和「最多/一次只能/必须」	一张资源清单，标出哪些被两个以上进程共用
②定信号量	共用且要独占→互斥；有先后→同步；有数量→计数	`semaphore` 声明，每个后面跟一句含义
③写进程骨架	按题给的进程个数和循环结构逐个写空壳	几个 `while(1){ }`，进程数与题面一致
④摆 P/V	资源/前驱先 P，自己产出或完成后 V，互斥夹在临界区两侧	每个 P 前问「我在等什么」，每个 V 前问「我给出了什么」
⑤检查	照下面 8 条清单逐条打勾	只改错，不重写

为什么第一步必须是数资源：408 的 PV 题信号量个数通常落在 3–5 个，个数错一个，后面 P/V 的位置一定跟着错。改卷口径里「定义了哪几个信号量、初值多少」是第一采分点，「P/V 摆在哪」是第二采分点，剩下的都是形式分。所以草稿纸上先写四行清单——资源名、类型（互斥/同步/计数）、初值、谁 P 谁 V，写完再落笔。

▶ 检查清单（照抄到草稿纸）

- 每个 P 都能一句话回答「我在等什么」；答不出的 P 就是多余的。
- 每个 V 都能一句话回答「我释放了什么」；答不出的 V 就是错的。
- 资源信号量（empty/full）的 P 是否在 mutex 的 P 之前。
- 初值：互斥是不是 1、同步是不是 0、资源是不是题给的那个数。
- 有没有一个信号量在表达两种含义（缓冲区奇偶混装、一个锁管两个信箱）——该拆。
- 有没有给同一进程内的天然先后写了信号量——该删。
- P 与 V 是否成对、个数是否相等。
- 信号量数组是否用 `for` 循环赋初值（`chopstick[n]=1` 这种写法直接扣分）。

```
/* ===== 答题纸第 1 段：信号量声明区（先写完，别边写进程边加） ===== */
semaphore mutex  = 1;    /* 互斥量：初值恒为 1，管一个临界资源 */
semaphore empty  = n;    /* 资源量：初值 = 题给的剩余数量，不是容量时要注意减 */
semaphore full   = 0;    /* 同步量：初值 0，或题给的「已有数量」 */

/* ===== 答题纸第 2 段：按题给进程数照抄结构 ===== */
void P1() { while (1) {
    /* ① 先 P 掉自己需要的资源 + 所有前驱的同步量 */
    /* ② 再 P 互斥量进临界区，临界区里只放「取/放」那一下，立刻 V 出来 */
    /* ③ 最后 V 自己产出的资源 + 后继的同步量 */
} }
```

只写 mutex 不写同步量（或反之）是最常见的丢分方式，一道题往往两者都要 · 信号量个数少一个扣一半，宁可多定义也别漏 ·「说明信号量含义及初值」是白送分小问，别省

信号量初值对照表

真题年份 | 2009/2011/2013/2014/2015/2019/2020（每年改卷最盯这一栏） | 初值写错，整题的逻辑分一起没了

必背结论

- 初值只有三个来源：**1**（互斥）、**0**（同步/前驱边）、题给的那个数（资源剩余量或已有数量）。
- 互斥量初值不是 1 就不再是互斥，而变成了同步量——改卷老师最认这条判定依据。
- 资源量要分清「剩多少」和「有多少」：空位初值 = 容量 − 已有数，满位初值 = 已有数。
- 同一个信号量不能背两种含义。缓冲区里既有奇数又有偶数，`full` 必须拆成 `odd`/`even`。
- 一个互斥量只管一个临界资源：两个信箱就要两个 `mutex`。
- 信号量数组必须 `for` 循环赋初值，这是形式分里唯一会真扣分的一条。
- 初值决定进程第一次能不能通过：同步量初值 0，没人 V 就永远过不去，顺序的强制力就来自这一点；资源量初值 n，前 n 个进程不用等。

模板/流水线

类型	初值	一句话含义	P/V 摆法
互斥量	1	临界资源只有 1 份	P 在临界区前、V 在临界区后，把临界区整个包住
同步量	0	事件尚未发生	先发生的进程后面加 V，后发生的进程前面加 P
空位资源量	容量 n 或 M−x	现在还能放几个	放入前 P，被取走一个后 V
已有数量	初始数量 x	现在已有几个可取	取出前 P，放入一个后 V
限流计数	min(m, n−1)	至多几人同时抢资源	抢资源前 P，全部还清后 V
前驱边	0	该前驱尚未完成	每条跨进程边一个，绝不共用
信号量数组	逐个 1	每份同类资源一个	`for` 循环逐个赋 1

初值算法只有两种：题面给「容量 M，已有 x 个」→空位 M−x、满位 x；题面给「一次只能一个」→ 1。凡是题里出现减法，一定对应「已经有东西占着」。凭感觉写 0 或 1 的，八成是把同步量当成了互斥量。

```
semaphore mutex = 1;          /* 互斥：出入口/缓冲区/文件，只有 1 份 */
semaphore empty = 500;        /* 资源剩余：馆内还能进 500 人 */
semaphore full  = 0;          /* 同步：初始没有产品可取 */
semaphore odd   = 0;          /* 拆出来的同步量：缓冲区里的奇数个数 */
semaphore even  = 0;          /* 拆出来的同步量：缓冲区里的偶数个数 */
semaphore chop[5];            /* 数组：5 根筷子，每根初值 1 */
int i;
for (i = 0; i < 5; i++) chop[i] = 1;   /* 必须循环赋值，一条语句赋全数组不合法 */
```

互斥写成 0、同步写成 1 是零分答案，顺序会整个反过来 · `empty` 别一律当容量，题里说「已有 x 个」就要用 M−x · 定义了六个信号量错一个初值，老师会判「未掌握同步互斥」

生产者—消费者模板

真题年份 | 2009/2014/2022 | 母题，三件套背下来可套六成年份

必背结论

- 三件套：`mutex=1`、`empty=` 缓冲区容量、`full=0`（初始为空）。

- 单缓冲就是把容量换成 1；多缓冲为 N 就用题给的 N，其余一字不改。
- 铁律：`empty/full` 的 P 必须在 `mutex` 的 P 之前，写反 = 死锁。
- 死锁原因：先 P(mutex) 拿到锁，再 P(empty) 因满而阻塞，锁没释放；消费者想取却拿不到锁，双方僵持。
- `mutex` 一层管三方互斥：生产者之间、消费者之间、生产者与消费者之间。
- V 的先后无所谓，习惯先 `V(mutex)` 再 `V(full)`；但 P 的顺序一个都不能动。
- 2009 变体：缓冲区混装奇偶，一个 `full` 分不出奇偶，必须拆 odd/even；统计 `countodd()` 在临界区外做，不互斥。
- 为什么 V 可换而 P 不可换：P 是「申请」，稀缺的那个（空位/产品）必须先申请，不够就尽早阻塞；占着锁去等，等于把锁带进了等待队列。

模板/流水线

```
semaphore mutex = 1;      /* 缓冲区互斥：同一时刻只许一个进程碰缓冲区 */
semaphore empty = N;      /* 空位个数，初值 = 缓冲区容量 N */
semaphore full  = 0;      /* 满位个数，初始空，故 0 */

Producer() { while (1) {
    produce();                      /* 生产在临界区外，不互斥 */
    P(empty);                       /* 要用一个空位：满了就在这里等 */
    P(mutex);                       /* 空位已占住，再抢缓冲区锁 */
    put();                          /* 临界区只包「放」这一下 */
    V(mutex);                       /* 放完立刻放锁 */
    V(full);                        /* 多了一个产品，唤醒等产品的消费者 */
} }

Consumer() { while (1) {
    P(full);                        /* 要有产品才能取：空的就在这里等 */
    P(mutex);                       /* 有货了，再抢锁 */
    get();                          /* 临界区只包「取」这一下 */
    V(mutex);
    V(empty);                       /* 腾出一个空位，唤醒等空位的生产者 */
    consume();                      /* 消费在临界区外 */
} }
```

```
/* 2009 变体：三进程，缓冲区混装奇偶 → 把 full 拆成 odd/even */
semaphore mutex = 1, empty = N, odd = 0, even = 0;

P1() { while (1) {                  /* 生产者 */
    x = produce();
    P(empty); P(mutex); put(x); V(mutex);      /* 放这一步互斥 */
    if (x % 2 == 1) V(odd); else V(even);      /* 按奇偶分派给两个消费者 */
} }

P2() { while (1) {                  /* 取奇数 */
    P(odd);                         /* 缓冲区里有奇数才取 */
    P(mutex); getodd(); V(mutex);   /* 取这一步互斥 */
    V(empty);                       /* 腾出空位 */
    countodd();                     /* 统计不需要互斥，放临界区外 */
} }

P3() { while (1) {                  /* 取偶数，与 P2 完全对称 */
    P(even);
    P(mutex); geteven(); V(mutex);
    V(empty);
    counteven();
} }
```

2009 若只用一个 `full`，P2/P3 会在奇数堆里取到偶数 · 统计类操作放临界区外，加锁反而扣「并发度」· 生产者不能用一个信号量管奇偶两种产出

读者—写者与计数互斥

真题年份 | 2019（读者—写者变体） | 「计数 + 双锁」是最容易少写一行的那类题

必背结论

- 双锁结构：rw=1 管文件本身，mutex=1 管计数器 count；count 是普通整型变量，不是信号量。
- 首个读者替全体读者 P(rw) 挡住写者；末个读者 V(rw) 放行写者；中间读者直接进——这就是「计数型互斥」。
- 读者侧的 count++ 与 count-- 前后各要一对 P(mutex)/V(mutex)。
- 写者只有一对 P(rw)/V(rw)，因为写者天生独占。
- 2019 的变体只需换皮：把「读文件」换成「占共享资源」，「写文件」换成「独占使用」，骨架一个字不改。
- 写者优先的改法：加门卫 w=1，写者先 P(w) 再 P(rw)，写完 V(w)；读者也先 P(w)，count 更新完立刻 V(w)，只要有写者在等，新读者就进不来。
- 独木桥、哲学家、图书馆座位都是这一族的换皮题。

判断口诀：能同时来多个的当读者，只能一个来的当写者。

模板/流水线

```
int count = 0;                  /* 当前读者数，普通变量，不是信号量 */
semaphore mutex = 1;            /* 只保护 count 的互斥量 */
semaphore rw    = 1;            /* 文件与写者互斥，初值 1 = 当前无人用 */

Reader() { while (1) {
    P(mutex);                   /* 先锁住计数器 */
    if (count == 0) P(rw);      /* 我是第一个读者：挡住所有写者 */
    count++;                    /* 读者数 +1，后来的读者靠 count>0 直接进 */
    V(mutex);                   /* 计数器用完立刻放，别的读者才能进 */
    读文件;                     /* 真正的临界操作在 mutex 之外，读者之间并行 */
    P(mutex);                   /* 离开前再锁计数器 */
    count--;                    /* 读者数 -1 */
    if (count == 0) V(rw);      /* 我是最后一个：放开写者 */
    V(mutex);
} }

Writer() { while (1) {
    P(rw);                      /* 有人读或有人在写就在这里等 */
    写文件;                     /* 独占临界区 */
    V(rw);                      /* 写完放开，读者和写者都可能被唤醒 */
} }
```

第二个 P(mutex)（count-- 那一侧）最容易漏，漏了 count 会算错 · if (count == 0) P(rw) 必须在 P(mutex) 保护之内 · 读者优先会让写者饿死，要求「写者优先」时再加一个门卫信号量 w

前驱图与先后同步

真题年份 | 2021（流水线装配）/2023/2024/2025（同步 PV） | 纯前驱边是同步的最简形式，几乎每年换皮出现

必背结论

- 每条跨进程的前驱边一个信号量，初值全 **0**；边的起点操作之后 V，边的终点操作之前 P。
- 同一进程内的天然先后（A → E → F）绝对不写信号量——线程自己就是串行的。
- 一个操作有多条入边→前面连续写多个 P；有多条出边→后面连续写多个 V。
- n 个操作不一定是 n 个信号量：信号量个数 = 跨进程边数 **m**，通常 m 小于 n。
- 2011 银行叫号里「叫号→顾客上前接受服务」只有一条跨进程边，就是最小的前驱图。
- 2013 博物馆里「进门→参观→出门」全在同一个进程内，天然有序，一个信号量都不用写（它要的是限流和门锁）。
- 命名用「起点→终点」：S_AC 一眼能对上边，比 s1/s2 好改卷；题面给 cobegin/coend 就照抄，别自创进程名。
- 链式前驱不能合并：S1 → S2 → S3 是两条边、两个信号量，不能因为「都是顺序」就只写一个，否则 S3 可能在 S2 之前跑。

模板/流水线

```
/* 以 5 个操作 A-E 为例：C 需 A、B 完成后执行；E 需 C、D 完成后执行 */
semaphore a = 0;        /* A → C 这条边：A 未完成则为 0，C 的 P 会阻塞 */
```

```
semaphore b = 0;      /* B → C */
semaphore c = 0;      /* C → E */
semaphore d = 0;      /* D → E */

A() { ...; V(a); }                    /* 起点之后 V：宣告「我完成了」 */
B() { ...; V(b); }
C() { P(a); P(b); ...; V(c); }        /* 终点之前 P：等齐全部前驱；完成后 V 自己的出边 */
D() { ...; V(d); }
E() { P(c); P(d); ...; }              /* 两条入边，两个 P 连着写 */
```

机械「每条边一个信号量」会把同进程内顺序也写上，2022 就靠这点扣分 · 一条边一个信号量，两条边共用会让 P 提前通过 · V 写在「本操作全部做完」之后，P 写在「第一句要用的东西」之前 · 初值写成 1 就退化成互斥，顺序完全失效

限流容量信号量

真题年份 | 2013（博物馆限流 500 人） | 只有「进」没有「取」，写成 empty/full 一对就是错的

必背结论

- 只有占名额、还名额，没有取走资源的一方→只需要一个计数信号量 empty= 容量。
- full 用不上：没有消费者，不产生「可取数量」这个概念。
- 「出入口一次仅许一人通过」是另一个互斥量 mutex=1，进门和出门都要包。
- 顺序：P(empty) 必须在 P(mutex) 之前。反了就是满员时有人攥着门锁等名额，把出门的人也堵在门外 = 死锁。
- V(empty) 必须在出门之后，代表名额真正还回来了。
- 限流与互斥不是一回事：互斥保证「一次只有一个」，限流只保证「总数不超过 N」。500 人的馆里可以同时有 500 人在参观，这就是限流的证据。
- 哲学家变体（n 人、m 个碗、每两人一根筷子）：限流计数 S = min(m, n-1)，把同时抢筷子的人压到 n−1，破坏循环等待。

模板/流水线

```
semaphore mutex = 1;        /* 出入口互斥：一次只许一人通过 */
semaphore empty = 500;      /* 馆内剩余容量，初值 = 最大容纳人数 */

Visitor_i() { while (1) {
    P(empty);               /* 馆内没满才允许靠近门口；满了在门外等 */
    P(mutex);               /* 门口一次一人 */
    进门;
    V(mutex);               /* 通过门口即放锁，别把门锁带进参观阶段 */
    参观;
    P(mutex);               /* 出门同样要独占门口 */
    出门;
    V(mutex);
    V(empty);               /* 已离馆，腾出一个名额 */
} }
```

```
/* 哲学家变体：n 人围桌，m 个碗，每两人一根筷子 */
semaphore S = min(m, n - 1);        /* 至多 n-1 人同时抢，必有人能拿齐 */
semaphore bowl = m;                 /* m 个碗 */
semaphore chop[n];
int i;
for (i = 0; i < n; i++) chop[i] = 1;  /* 每根筷子初值 1 */

Philosopher_i() { while (1) {
    P(S);                           /* 限流：先把同时抢的人数压下来 */
    P(bowl);                        /* 要有碗才能吃 */
    P(chop[i]);                     /* 左筷子 */
    P(chop[(i + 1) % n]);           /* 右筷子 */
    eat();
    V(chop[i]); V(chop[(i + 1) % n]);
    V(bowl);
    V(S);                           /* 全部还清后才放开限流名额 */
    think();
} }
```

「最多容纳 N 人」不等于「要 empty/full 一对」，先问有没有取走的一方 · V(empty) 忘写 = 馆内人数只增不减 · 限流信号量 P 必须在最外层、V 在最后，夹在中间就等于没限流

连续 k 次才释放

真题年份 | 2014（环形缓冲区，消费者连续取 10 件） | 错误写法最多的一道真题

必背结论

- 两层锁缺一不可：mutex1 管缓冲区，mutex2 管「这一整组 10 次」。
- P(mutex2) 在 for 外面，V(mutex2) 在 for 后面——这一组才是原子的。
- P(full)、P(mutex1) 必须在 for 内部：P 一次只保证「至少 1 件」，不保证「连续 10 件」。
- 「下一个取货的必须换人」这个要求靠 mutex2 在组与组之间互斥实现，mutex1 做不到。
- 环形缓冲区：in = (in+1) % 1000、out = (out+1) % 1000，取模不能漏。
- 生产者侧完全不受约束：题目只限制消费者，别给生产者套 mutex2，那会把并发度砍掉。
- 若题目反过来要求「生产者连续放 k 件、期间别人不许放」，把 mutex2 挪到生产者侧，骨架一模一样；两边都要求就是两把外层锁。谁被要求「连续」，锁就套在谁身上。
- 自查只要问一句：缓冲区里只有 3 件产品时这段代码会不会卡住？会卡住就是 P(full) 写到循环外了。

模板/流水线

```
semaphore mutex1 = 1;        /* 缓冲区互斥：所有生产者/消费者共用 */
semaphore mutex2 = 1;        /* 消费者之间「连续取 10 件」这一组的互斥 */
semaphore empty  = 1000;     /* 环形缓冲区容量 1000，初始空 */
semaphore full   = 0;        /* 初始无产品 */
int in = 0, out = 0;

Producer() { while (1) {
    P(empty);                /* 有一个空位才放 */
    P(mutex1);               /* 抢缓冲区锁，且必须在 empty 之后 */
    放入产品; in = (in + 1) % 1000;
    V(mutex1);
    V(full);                 /* 多一件产品 */
} }

Consumer() { while (1) {
    P(mutex2);                          /* 外层锁：占住「这一组」的资格 */
    for (int i = 1; i <= 10; i++) {
        P(full);                        /* 每取一件都要单独 P：保证真有 10 件 */
        P(mutex1);                      /* 每件都单独抢缓冲区锁 */
        取出产品; out = (out + 1) % 1000;
        V(mutex1);
        V(empty);                       /* 每件都还一个空位，生产者才能继续放 */
    }
    V(mutex2);                          /* 10 件取完才放开资格 */
} }
```

只写一个 mutex：两个消费者能同时进 for，连续 10 件的要求没实现 · P(mutex1) 提到 for 外面：空缓冲区时持有缓冲区锁去等 full，生产者拿不到锁 = 死锁，而且这 10 件里禁止了生产者插入（题目并未禁止）· P(full) 提到 for 外面：只保证有 1 件，缓冲区里只有 3 件时第 4 次循环就卡死

对称同步双重信箱

真题年份 | 2015（两人信箱辩论） | 生产者—消费者的双向包装，两个方向完全镜像

必背结论

- 两个信箱 = 两组信号量，绝不能用同一个 mutex 管两个信箱。
- A 信箱：emptyA = M − x（剩余空位）、fullA = x（已有邮件数）；B 信箱：emptyB = N − y、fullB = y。
- 初值是题给的剩余量，不是容量 M/N——这题最大的坑就在这里。
- A 的代码与 B 完全镜像：把 A/B 互换，一行逻辑都不用改。

- 每个进程做两件事：先从自己信箱取（走 fullA + mutexA），再往对方信箱投（走 emptyB + mutexB），两组方向不能串。
- 「互相等对方来信」构成对称同步：两人可能同时卡在取信上，谁也没投信→必须靠题给的初始邮件 x、y 打破僵局，这就是初值写 x、y 而不是 0 的原因。
- 答题顺序：两组信号量与四个初值写在最上面（主要采分点集中在初值），再完整写 A，最后写「B 与 A 对称」——对称能省一半篇幅，但 A 的每一行必须真的对。

模板/流水线

```
semaphore mutexA = 1, mutexB = 1;          /* 两个信箱各一把锁，不能合并 */
semaphore emptyA = M - x, emptyB = N - y;  /* 剩余空位：容量减已有 */
semaphore fullA  = x,     fullB  = y;      /* 已有邮件数：题给的初始值 */

A() { while (1) {
    P(fullA);                  /* A 信箱里要有对方的信才能取 */
    P(mutexA);                 /* 取信动作互斥 */
    取出对方的问题;
    V(mutexA);
    V(emptyA);                 /* 取走一封，A 信箱多一个空位 */
    回答问题并提出新问题;
    P(emptyB);                 /* B 信箱不满才能投，满了在这里等 */
    P(mutexB);                 /* 投信动作互斥 */
    将新邮件放入 B 信箱;
    V(mutexB);
    V(fullB);                  /* B 信箱多一封信，唤醒在等信的 B */
} }

B() { /* 与 A 完全对称：P(fullB) →取→ V(emptyB)；P(emptyA) →投→ V(fullA) */ }
```

初值写 M、N 而不是 M−x、N−y 是本题零分点 · 两个信箱共用一个 mutex = 互斥范围错 · 取信用 fullA、投信用 emptyB 与 fullB，串了就是往自己信箱投

方向互斥独木桥

真题年份 | 2017（独木桥） | 读者—写者的换皮题：同方向当读者，反方向当写者

必背结论

- 同方向可同时上桥→每个方向一套「计数器 + 保护计数器的 mutex」；桥本身一个 bridge=1。
- 第一个上桥的人 P(bridge) 锁住方向，最后一个下桥的人 V(bridge) 放行对面。
- 中间的人不碰 bridge，只加减计数，所以同向能并行。
- 若题面改成「桥上一次只允许一人通过」，就退化成单个 mutex=1 包住「过桥」，不需要计数。
- 两个方向的 cnt 和 mutex 各一套，共用一个会让两个方向互相串。
- 计数器只在 mutex 内加减，if (cnt == 0) 的判断也必须在锁内。
- 题面的「桥上不能掉头」或「一次只允许同方向的人通过」是同一句话：方向锁在整段通过期间不许释放；若再加「桥最多承载 8 人」，就在外面再叠一个限流信号量。

模板/流水线

```
int cntA = 0, cntB = 0;           /* 两个方向各自在桥上的人数 */
semaphore mutexA = 1, mutexB = 1; /* 各自计数器的互斥量，不能合并 */
semaphore bridge = 1;             /* 桥的使用权：初值 1 = 桥空 */

GoRight() { while (1) {
    P(mutexA);                    /* 保护 cntA */
    if (cntA == 0) P(bridge);     /* 我是本方向第一个人：把桥从对面手里抢过来 */
    cntA++;                       /* 后续同向的人 cntA>0，不再抢 bridge，直接上桥 */
    V(mutexA);
    过桥;                          /* 同方向可并行通过 */
    P(mutexA);                    /* 下桥也要改计数，先锁 */
    cntA--;
    if (cntA == 0) V(bridge);     /* 本方向最后一人：放开桥给对面 */
```

```
    V(mutexA);
} }

GoLeft() { /* 与 GoRight 镜像：mutexB / cntB，桥锁同样是 bridge */ }
```

P(bridge) 写到 if (cntA == 0) 外面 = 同方向也只能一人过桥，违背题意的并行要求 · cntA-- 与 V(bridge) 都必须在 P(mutexA) 内 · 两个方向共用一套 cnt/mutex 会让「同向并行」失效，还可能死锁

售票与流水线变体

真题年份 | 2020/2021/2022/2023/2024/2025（连续六年） | 近六年全是老朋友换皮，识别出族属就直接默写模板

必背结论

- 三问定族属，顺序不能换：有没有「取走」的一方？有→产消族（empty/full 一对）；没有→限流族（只要 empty）。这一问定了这道题的大部分分数，答错后面全白写。
- 有没有「先后」？有→每条跨进程边一个 0 初值同步量；流水线就是「每段传送带 = 一个容量 n 的缓冲区」。
- 有没有「一次只能一个」？有→ mutex=1 包住临界动作。
- 售票是「两向握手」：顾客 V 一个「我来了」、售票员 V 一个「票好了」，两个方向各一个 0 初值同步量，外加窗口互斥量。
- 流水线 = 前驱链 + 产消：K 个工位有 K−1 段传送带，每段一对 empty=n/full=0，每个工位一个 mutex。
- 考场节奏：这道题给 8 分钟——2 分钟判族属并写声明区，5 分钟写进程体，1 分钟逐条过检查清单。族属判对，剩下是默写；族属判错，写满也拿不到位置分。

模板/流水线

年份	变体	信号量设置	破题点
2020	售票/服务窗口	窗口 mutex=1 + 两个反向同步量	顾客与售票员不互斥，只握手
2021	流水线装配	每段传送带 empty=n full=0	相邻工位一对同步量，别跨级定义
2022	缓冲区同步	产消三件套，容量按题给	先看「已有 x 个」还是「初始为空」
2023	同步 PV	两条反向同步量 + mutex=1	交替执行就是两个 0 初值互锁
2024	同步 PV	计数 + 同步混搭	先数资源个数，再定同步方向
2025	同步 PV	以同步为主，可能无互斥	通篇没有共享变量就别硬加 mutex

```
/* 售票：窗口互斥 + 两向握手（顾客与售票员之间不互斥，只同步） */
semaphore mutex = 1;      /* 窗口互斥：同一时刻只有一位顾客在窗口 */
semaphore req   = 0;      /* 顾客已到窗口，初值 0 = 还没人来 */
semaphore resp  = 0;      /* 票已办好，初值 0 = 还没出票 */

Customer_i() {
    P(mutex);                       /* 占住窗口 */
    递上钱和证件;
    V(req);                         /* 通知售票员：我来了（先发生者 V） */
    P(resp);                        /* 等票办好（后发生者 P）；此间仍持窗口锁 = 窗口只服务一人 */
    取走车票和找零;
    V(mutex);                       /* 让出窗口，下一位才能上前 */
}

Clerk() { while (1) {
    P(req);                         /* 有人来才干活，没人就等 */
    收钱、打票;
    V(resp);                        /* 票好了，通知顾客 */
} }
```

```
/* 流水线装配：3 个工位，2 段传送带，每段容量 1 */
semaphore empty1 = 1, full1 = 0;    /* 工位 1 → 工位 2 的传送带 */
semaphore empty2 = 1, full2 = 0;    /* 工位 2 → 工位 3 */
```

```
semaphore mutex1 = 1, mutex2 = 1;   /* 两段传送带各自互斥 */

Station1() { while (1) {
    加工零件;                        /* 加工自己干，不互斥 */
    P(empty1);                       /* 传送带 1 有空位才放 */
    P(mutex1); 放到传送带 1; V(mutex1);
    V(full1);                        /* 传送带 1 多一件 */
} }

Station2() { while (1) {
    P(full1); P(mutex1); 从传送带 1 取; V(mutex1); V(empty1);   /* 先取上一段 */
    加工零件;
    P(empty2); P(mutex2); 放到传送带 2; V(mutex2); V(full2);   /* 再放下一段 */
} }

Station3() { while (1) {
    P(full2); P(mutex2); 从传送带 2 取; V(mutex2); V(empty2);
    装配;
} }
```

流水线别只写前驱同步量却漏掉传送带容量，上游比下游快时零件会无限堆积 · 售票题给顾客和售票员之间加 mutex 是过度互斥，扣分 · 判族属之前不要动笔，写错族属改起来比重写还慢

分页拆位总模板

真题年份 | 2013/2015/2017/2018/2020/2024（OS 大题） | 页大小一换就能再考一次，拆位是所有分页题的第一问，5–8 分

必背结论

- 页面大小 2^k 字节→页内偏移 k 位（4KB → 12 位，1KB → 10 位，2MB → 21 位）。
- 页号位数 $m = n - k$（n 为逻辑地址位数）；进程页数 = 页表项数 = 2^m。
- 页表大小 $= 2^m \times e$（e 为页表项长）；页表占页数 $= \lceil 2^m e / 2^k \rceil$。
- 页框号位数 = 物理地址位数 $-k$；页框数 = 内存容量 / 页面大小。
- 页表项里只存页框号，不存物理地址：物理地址 = 页框号 $\times 2^k$ + 偏移。
- 页号与页框号都从 0 编号；页号隐含在「第几项」里，不占表项空间。
- 反向题（408 常考）：偏移 k 位→页有 2^k 个地址；页号 m 位→最多 2^m 页；页框号 f 位→内存最多 2^f 个页框。
- 页表占页数一律向上取整，剩 1 字节也要占整整一页；页表项个数只等于页数，与项多长无关。

题目给的条件	落笔就写的结论
页面大小 4KB	偏移 12 位，页内 2^12 个地址
页面大小 1KB	偏移 10 位
逻辑地址 32 位、页 4KB	页号 20 位，共 2^20 页
逻辑地址 20 位、页 4KB	页号 8 位，共 256 页
页表项 4B	一页放 1024 项，一级索引 10 位
物理内存 256MB	物理地址 28 位，页框号 16 位
偏移 2 位、页框号 2 位	页 4 个地址、页框 4 个、总地址 16 个

三步：拆（÷页大小取商余）→换（页号换成页框号）→拼（页框号×页大小 + 偏移）；偏移从第一步到最后一步一位都不动。

数字例子：逻辑地址 20 位（虚址空间 2^{20} = 1MB）、页 4KB。$k = 12$、$m = 8$ → 256 页→页表 $256 \times 4\text{B} = 1\text{KB}$ → 占 1 个页框。虚址 `0x1A2B4`：低 12 位 `0x2B4` 是偏移，高 8 位 `0x1A` = 26 是页号；页表第 26 项存页框号 0x05 →物理地址 `0x052B4`。

同一个进程换成 32 位虚址：页号 20 位→ 2^{20} 页→单级页表 4MB = 1024 页，这才逼出二级页表。

模板/流水线

```
/* 拆位流水线：所有分页题共用的前 6 行 */
k = log2(page_size);              /* 4KB->12, 1KB->10, 2MB->21 */
m = n - k;                        /* 页号位数 */
entries = 1 << m;                 /* 页表项数 = 进程页数 */
pt_bytes = entries * e;           /* e = 页表项长，真题多为 4B */
pt_pages = (pt_bytes + page_size - 1) / page_size;

W = addr & ((1 << k) - 1);        /* 页内偏移：低 k 位 */
P = addr >> k;                    /* 页号：其余高位 */
F = pagetable[P].frame;           /* 查表只取页框号 */
pa = (F << k) | W;                /* 拼接：页框号在高、偏移在低 */
```

页表项存的是页框号不是物理地址，忘了乘页大小就差 2^k 倍 · 十六进制里× 2^12 是加 3 个 0，但× 4 只是左移 2 位，不是加零 · 题目不说「按字节编址」时只能写 2^12 个地址，直接写 4KB 不严谨

多级页表拆位流水线

真题年份 | 2013/2015/2017/2018/2020/2024（OS 大题） | 二级页表是内存大题的绝对主力；三级大题未考，只作二级的直接推广

必背结论

- 二级地址 = P_1（页目录号） + P_2（页表索引） + W；三级再加一段 P_3，各段位数之和 = 地址总位数。
- 唯一硬约束：最低 k 位永远是偏移，其余高位按题目给的分段从左往右切。
- 每级索引位数 = $\log_2$(一页能放的项数)= $\log_2(2^k/e)$；4KB 页 + 4B 项 = 1024 项 = 10 位。
- 32 位地址 + 4KB 页 + 4B 项→ 10 | 10 | 12，两级刚好用完；要三级得 42 位（10 | 10 | 10 | 12）。
- 级数 = 查表访存次数：一级 2 次、二级 3 次、三级 4 次（末次是取数）。
- 名称对应：一级页号 = 页目录号，二级页号就叫页号；页目录项存下一级表的页框号，页表项存数据页的页框号。
- 「访问了几张二级页表」只看页目录号：两个地址页目录号相同→只访问 1 张；不同才算 2 张。
- 分段顺序不能颠倒：页目录号占高位、偏移占低位，写反了地址全错，先画一遍二进制再动笔。

级数	地址结构	查表访存
一级	页号 + 偏移	1 次
二级	页目录号 + 页号 + 偏移	2 次
三级	索引 1 + 索引 2 + 索引 3 + 偏移	3 次

数字例子（正向）：虚址 0x10801008、页 4KB、项 4B、结构 10 | 10 | 12。写成二进制 0001 0000 1000 0000 0001 0000 0000 1000，从高位数 10 位→页目录号 0x042，再 10 位→页号 0x001，末 12 位→偏移 0x008。

数字例子（反向，2018 型）：给页目录号 6、页号 6、偏移 8，各写成 10/10/12 位再拼 0000000110 0000000110 000000001000 = 0x01806008。

数字例子（三级）：42 位虚址、页 4KB、项 4B → 10 | 10 | 10 | 12；页目录 1 张、二级最多 1024 张、三级最多 2^{20} 张，实际只为用到的那一支建表，走表 3 次加取数 1 次共 4 次访存。

10 位、12 位都不是 4 的整数倍，必须先把地址写成完整二进制再切，硬凑十六进制必错。

模板/流水线

```
/* 三级页表走表：每一级都只是「基址 + 索引 * 项长」 */
uint64_t walk(uint64_t va, uint64_t dir_base) {
    int k = 12, s = 10;                 /* 偏移 12 位，每级索引 10 位 */
    uint64_t W  = va & ((1 << k) - 1);
    uint64_t i3 = (va >> k) & ((1 << s) - 1);
    uint64_t i2 = (va >> (k + s)) & ((1 << s) - 1);
```

```
    uint64_t i1 = (va >> (k + 2*s)) & ((1 << s) - 1);  /* 剩余高位全给 i1 */
    uint64_t p1 = *(uint32_t *)(dir_base + i1*4);      /* 页目录项 -> 页框号 */
    uint64_t p2 = *(uint32_t *)((p1 << k) + i2*4);     /* 先乘页大小还原首址 */
    uint64_t p3 = *(uint32_t *)((p2 << k) + i3*4);
    return (p3 << k) | W;               /* 页框号在高，偏移原样 */
}
```

页目录项里存的是页框号，取下一级表首址必须先乘页大小 · 三级最少访存 4 次，别照二级的 3 次套 · 各段位数之和写完先加一遍，不等于地址总位数就是拆错了

页表项物理地址计算

真题年份 | 2013（OS 大题） | 地址拆分之后必问的落地一步，同一套公式连问两问

必背结论

- 页表项物理地址 = 页表首地址 + 页号 × 页表项长度。
- 页目录项物理地址 = 页目录表首地址 + 页目录号 × 页目录项长度。
- 二级页表首地址 = 上级表项里存的页框号 × 页面大小，所以二级页表首址不用另存。
- 一级页表（页目录）的首地址必须存：一个进程只有一张，放页表基址寄存器 / PCB 里。
- 页表必须连续存放，否则「首址 + 页号×项长」这条公式不成立；这也是二级页表要解决的问题。
- 页号占的位置是「第几项」，像数组下标一样不占存储；但页表本身一定在内存里，要访存。
- 问「某项的地址」分三步写：算项地址→读出里存的页框号→乘页大小得下一级首址。
- 地址变换靠物理地址访存，所以「这一项在哪」必须先算出来，这一步也是 2013 那道题给分的落脚点。

题目问的是	套哪条公式
页表里第 p 项在哪	页表首址 + p ×项长
页目录表第 d 项在哪	页目录表首址 + d ×项长
二级页表在哪	页目录项里的页框号×页大小
数据页在哪	页表项里的页框号×页大小

数字例子：页目录表首址 0x00201000、页目录项 4B、页目录号 0x042 → 0x00201000 + 0x042 × 4 = 0x00201000 + 0x108 = 0x00201108。该表项存页框号 0x00301 →二级页表首址 0x00301 × 2^12 = 0x00301000；二级页号 0x001 →页表项地址 0x00301000 + 4 = 0x00301004；若该项存 0x00401，数据页首址 0x00401000，加偏移 0x008 得 0x00401008。

另一组数：页表首址 0x00100000、页号 0x123、项长 4B →项地址 0x00100000 + 0x48C = 0x0010048C，因为 0x123 × 4 在二进制里就是左移 2 位。

模板/流水线

```
/* 两级页表：从虚地址一路算出每个表项所在地址 */
pa_dir = dir_base + P1 * e;         /* 页目录项地址 */
f1 = *(uint32_t *)pa_dir;           /* 取出二级页表的页框号 */
pa_pt = (f1 << k) + P2 * e;         /* 二级页表项地址：先乘页大小再加 */
f2 = *(uint32_t *)pa_pt;            /* 取出数据页的页框号 */
pa = (f2 << k) + W;                 /* 最终物理地址 */
```

十六进制乘 4 不是加两个 0：0x042 × 4 是 0x108，要左移 2 位 · 忘了「页框号«k」直接把页框号当首址，后面全错 ·「表项的地址」与「表项里存的页框号」是两个不同的量

多级页表占页数估算

真题年份 | 2015（OS 大题） | 二级页表题必问的第二问，标准答案常是 1025 页

必背结论

- 一页能放的项数 = 页面大小 / 页表项长度；4KB / 4B = 1024 项，与进程大小无关。
- 单级页表占页数 = $\lceil 2^m e/2^k \rceil$。
- 二级占页数 = 1（页目录）+ 二级页表张数，张数 = $2^m/1024$。
- 2015 标准数：2^{20} 页→ 1024 张二级页表 + 1 张页目录 = 1025 页。
- 反直觉：全满时二级比单级（1024 页）多 1 页，多的就是页目录自己。
- 分级真正省的是「连续性」：单级要 4MB 连续，二级每张表只要 1 页连续。
- 三级同理：页目录 1 张 + 中间层张数 + 末级张数，逐层用上一层项数除以每表项数。
- 题目按「该进程所有页面都已装入」算；实际运行时二级页表只为用到的那一支建。
- 占页数正比于进程页数，只有「每页表项数」这一项与进程大小无关。
- 二级页表的每一项指向一个数据页（不是下一级表），所以张数 = 页数 ÷ 每表项数；只有三级才多出「中间层」。
- 页表项长度按题目给的算：给 4B 就是 4B，题目改成 8B 则每页只放 512 项、索引变 9 位。

页大小 / 项长	一张表页能放几项	对应索引位数
4KB / 4B	1024	10 位
4KB / 8B	512	9 位
1KB / 4B	256	8 位
2MB / 4B	524288	19 位

数字例子：页 4KB、项 4B、2^{20} 页。单级页表 $2^{20} \times 4\text{B} = 4\text{MB} = 1024$ 页；二级每张表 1024 项→ 1024 张 + 页目录 1 张 = 1025 页。若进程只用到 3 页，二级只需 2 页，单级仍要 4MB。

数字例子二：逻辑地址 22 位、页 4KB →页号 10 位→ 1024 页→单级页表 $1024 \times 4\text{B} = 4\text{KB}$ 恰好 1 页；若改成二级，要 1 张页目录 + 1 张页表 = 2 页，分级反而多占 1 页——分级只在进程很大时才划算。

模板/流水线

```
/* 多级页表内存开销：先算项数，再算页数 */
n_entries = 1 << m;                     /* 总页数 = 总表项数 */
per_page  = page_size / e;              /* 一张表页放几项 */
n_tables  = (n_entries + per_page - 1) / per_page;   /* 二级表张数 */
total     = n_tables + 1;               /* 别忘了页目录那一页 */
bytes     = total * page_size;          /* 问内存开销才乘 */
```

> 全满时二级比单级多 1 页，别背成「分级一定更省」· 页目录永远只有一张，先把「+1」写上再算张数 · 问「共占多少页」答页数，问「占多少内存」要再乘页大小

访存次数统计模板

真题年份 | 2009（OS 大题） | TLB + 缺页 + LRU 三件套那道题的计数部分

必背结论

- 查 TLB 不算访存次数（相联存储器，只另计时）；TLB 命中直接得页框号，省掉查页表。
- 查页表每级 1 次访存；取数 1 次访存，Cache 命中则省掉。
- 二级页表：TLB 命中 1 次、未命中 3 次；三级：命中 1 次、未命中 4 次。
- 缺页那一次要另算磁盘 I/O 次数，不进「访存次数」。

- 总式：访存次数 $= N \times [(1-h)L + (1-c)]$，$h$ 为 TLB 命中率，L 为页表级数，c 为 Cache 命中率。
- 命中率、缺页率都按「访问次数」统计；一条指令访存多次就按次算，不按条算。
- TLB 用页号（标记）查，在 MMU 里；Cache 用物理地址查，在 CPU 与内存之间，两者串联不是二选一。
- 无 TLB 时页表访存次数就是级数 L，与 Cache 命中率完全解耦，可以分开算。
- Cache 命中与否只影响最后取数那 1 次，一次也不会影响查页表的 L 次。
- 题目问「访问内存几次」时把查 TLB 排除在外；只有问「访问存储系统几次」才把 TLB 一起算进去。
- 缺页处理完重新执行该指令时会再查一次表，题目若把 TLB 更新算进去就按命中处理。

情形	访存次数
TLB 命中 + Cache 命中	0 次
TLB 命中 + Cache 未命中	1 次
TLB 未命中 + Cache 命中（二级页表）	2 次
TLB 未命中 + Cache 未命中（二级页表）	3 次

数字例子一：1000 次数据访问、二级页表、TLB 命中率 0.9、Cache 命中率 0.95 → $1000 \times [0.1 \times 2 + 0.05] = 250$ 次访存。

数字例子二：没有 TLB、二级页表、Cache 命中率 0.9 → $1000 \times [2 + 0.1] = 2100$ 次访存，是前者的 8 倍多。

数字例子三：1000 次访问缺页 12 次→缺页率 1.2%；每次缺页换出加换入算 2 次盘访问→ 24 次盘 I/O，这 24 次一律不进访存次数。

模板/流水线

```
/* 访问次数统计：访存与访盘两个计数器，别合并 */
long mem = 0, disk = 0;
for (i = 0; i < N; i++) {
    if (!tlb_hit(va)) {            /* TLB 未命中 */
        mem += L;                  /* 逐级查页表，每级 1 次 */
        if (!present(va)) {        /* 存在位为 0 -> 缺页 */
            disk++;                /* 换入/换出各算一次盘访问 */
            /* 处理完改页表与 TLB，再【重新执行】本条指令 */
        }
    }
    if (!cache_hit(pa)) mem += 1;  /* 取数未命中 Cache 才访存 */
}
```

查 TLB 不算访存次数但算时间，两个口径别混 · 缺页的磁盘 I/O 单列，别塞进访存次数 · TLB 命中省掉的是查页表的 L 次，取数那 1 次永远省不掉

有效访问时间 EAT

真题年份 | 2009（OS 大题） | TLB + 缺页的「访问时间」就是这条公式的代入

必背结论

- 主式 $EAT = (1-p)t_a + p\,t_f$，p 为缺页率，t_a 为不缺页的一次访问时间，t_f 为缺页处理时间。
- 有 TLB 时 $t_a = h(t_{tlb} + t) + (1-h)(t_{tlb} + Lt)$，$h$ 为 TLB 命中率。
- 无 TLB 且单级页表 $t_a = 2t$；无 TLB 且二级页表 $t_a = 3t$ ——先算 t_a，再乘 $(1-p)$。
- 缺页处理含磁盘寻道与传输，量级是毫秒，比访存大 4 个数量级。
- 缺页率 10^{-4} 就能把 EAT 拉到 7–10 倍量级，这是「虚拟存储不能滥用」的量化依据。
- 命中率 $= 1 - p$，两者是同一件事的两面；h 与 p 相互独立，不能相加也不能互推。
- 问「加速比」用不缺页时间除以 EAT，问「慢多少倍」用 EAT 除以不缺页时间。
- 单位先统一：t、t_{tlb} 用 ns，t_f 给的是 ms 就先乘 10^6 再代。

- 缺页率每降一个数量级，pt_f 就降一个数量级——减少缺页比加快访存有效得多。
- 同时给 TLB 与 Cache 时按「TLB →页表→ Cache →内存」四段串起来列，漏掉 Cache 段是最常见的错。
- 口径先统一：$p = 1-$ 命中率；题目给「平均每 100 万次访问缺一次页」就直接取 $p = 10^{-6}$。
- 卷面写法三段：先写 t_a 的式子，再写 EAT 的式子，最后代数字——只给一个数最多拿一半分。

$$EAT = (1-p)\left[h(t_{tlb}+t) + (1-h)(t_{tlb}+Lt)\right] + pt_f$$

数字例子一（有 TLB）：$t = 100\text{ns}$、$t_{tlb} = 10\text{ns}$、$h = 0.9$、$L = 2$、$p = 10^{-4}$、$t_f = 8\text{ms}$。$t_a = 0.9\times110+0.1\times210 = 120\text{ns}$；$EAT = 0.9999 \times 120 + 0.0001 \times 8\,000\,000 \approx 920\text{ns}$，比不缺页慢约 7.7 倍。

数字例子二（无 TLB、单级）：$t_a = 200\text{ns}$、$p = 10^{-5}$、$t_f = 8\text{ms} \to EAT \approx 200 + 80 = 280\text{ns}$，可见哪怕缺页率再小，$pt_f$ 也是主导项。

数字例子三（提高命中率）：$h = 0.98$ 时 $t_a = 0.98 \times 110 + 0.02 \times 210 = 112\text{ns}$，$EAT \approx 112 + 80 = 192\text{ns}$，比 $h = 0.9$ 时的 920ns 好得多——瓶颈已经转回缺页。

参数	含义	常见取值
t	一次访存时间	100 ns
t_tlb	查一次 TLB	10 ns
t_f	缺页处理（含磁盘）	8 ms
p	缺页率	10^-4 到 10^-6

模板/流水线

```
/* EAT 代入流水线：先算不缺页的时间，再按概率加权 */
double t_a = h*(t_tlb + t) + (1-h)*(t_tlb + L*t);  /* 命中/未命中两段 */
double eat = (1-p)*t_a + p*t_f;                    /* p 是缺页率 */
double ratio = eat / t_a;                          /* 缺页把性能拉低几倍 */
```

缺页率是「每次访问缺页的概率」，不是缺页次数除以页数 · t_f 带磁盘时间，先统一成 ns 再代入 · h 与 p 是两个独立参数，不能相加也不能互推

置换算法填表模板

真题年份 | 2009/2010/2012（OS 大题） | 置换表是送分块，画错一格后面全错

必背结论

- OPT：往后看，淘汰最长时间内不再被访问的；理论下界，不可实现。
- FIFO：队列，淘汰最先进来的；命中不改变队列顺序；唯一会出现 Belady 异常。
- LRU：往前看，淘汰最久没被访问的；要维护访问历史（栈或计数器）。
- CLOCK：访问位 0 就换出，1 就清 0 后移；命中只置 1，指针不动。
- 改进 CLOCK：淘汰顺序 $(0,0) \to (0,1) \to (1,0) \to (1,1)$；新调入页 $A = 1$、$M = 0$。
- 缺页次数 $\neq$ 置换次数：前几次还有空闲页框，只缺页不置换。
- Belady 异常只发生在 FIFO：同一走向 3 块缺页 9 次、4 块反而 10 次。
- 缺页率 = 缺页次数 / 访问总次数：上表 20 次访 12 次缺页即 60%，FIFO 15 次即 75%。
- 加页框只对 FIFO 可能变差；OPT 与 LRU 都不会出现 Belady 异常。

算法	淘汰谁	往哪看
OPT	未来最久才用到的	后
FIFO	最先进来的	队列
LRU	最久没用过的	前
CLOCK	访问位为 0 的	转圈
改进 CLOCK	00 先走、11 最后	访问位 + 修改位

填表格式（走向 7 0 1 2 0 3 0 4 …，3 个页框，FIFO 前 8 列）：

页面走向	7	0	1	2	0	3	0	4
块 1（最旧）	7	7	7	0	0	1	2	3
块 2	—	0	0	1	1	2	3	0
块 3（最新）	—	—	1	2	2	3	0	4
缺页	√	√	√	√	·	√	√	√

同一条 20 次走向的总数：OPT 缺页 9 次、FIFO 15 次、LRU 12 次。

缺页栏打√求和就是缺页次数；命中率 = 1 − 缺页次数 / 访问总次数；问「置换次数」要减掉还有空闲页框的那几次。

改进 CLOCK 的四步（每次淘汰都从头走一遍）：

1. 扫一圈找 $(A=0, M=0)$，找到就淘汰，本轮不改任何标志位。
2. 若失败，再扫一圈找 $(A=0, M=1)$，把扫过的页 A 清 0。
3. 若仍失败，重复第 1 步（此时所有 A 都已是 0，必能找到）。
4. 若仍失败，重复第 2 步。

数字例子：4KB 页、两个页框放页 3（$A=1, M=0$，在 60H）与页 4（$A=1, M=1$，在 80H），访问 `02A01H` →页号 `02H` 存在位 0，缺页；第 1、2 步都失败，第 3 步淘汰已变 $(0,0)$ 的页 3 →物理地址 `60H × 4KB + A01H = 60A01H`。

模板/流水线

```
/* 三种算法共用的填表骨架：只换 pick_victim 一行 */
for (i = 0; i < n; i++) {
    v = ref[i];
    if (find(v) >= 0) { fault[i] = 0; touch(v); continue; }  /* 命中 */
    fault[i] = 1;
    if (has_free_frame()) { place(v); continue; }   /* 只缺页，不置换 */
    j = pick_victim();      /* OPT/LRU/FIFO/CLOCK 的唯一差别 */
    evict(j); place(v);
}
faults = sum(fault);                              /* 缺页次数 */
repl   = faults > frames ? faults - frames : 0;  /* 置换次数 */
```

前几次缺页有空闲页框，只缺页不置换，两个数必须分开 · FIFO 命中不改变队列顺序，改了就全错 · CLOCK 命中只把访问位置 1，只有发生淘汰才移动指针

请求分页完整访存流程

真题年份 | 2024（OS 大题 · 大页分页缺页） | 流程题年年以「这一步谁做、几次访存、什么中断」出现

必背结论

- 顺序：拆虚址→查 TLB →查多级页表→缺页中断→拼物理地址→查 Cache →访主存。
- 请求分页页表项 = 页号 + 页框号 + 状态位 + 访问字段 + 修改位 + 外存地址。
- 存在位 0 →缺页；修改位 1 →换出要写回外存；访问位供置换算法用。
- 缺页中断是内中断（内部异常），发生在指令执行期间，一条指令可多次缺页。
- 缺页处理完是「重新执行」被中断的指令，不是执行下一条。
- 无空闲页框时：选牺牲页→脏页写回→调入新页→改页表与 TLB →重新执行。
- 大页（页变大）的直接后果：偏移位数变多、页表项数变少，其余流程一字不改。
- 驻留集是「页框的集合」、工作集是「页面的集合」，驻留集不能小于工作集，否则频繁缺页。
- 三种分配策略：固定分配局部置换、可变分配全局置换、可变分配局部置换。
- 缺页处理五步：看有无空闲页框→无则选牺牲页→脏页写回→调入新页→改页表与 TLB 并重新执行。
- 缺页率与缺页发生在哪一级页表无关，只与访问序列和页框数有关。

步	做什么	谁做	访存
1	拆虚址为页号与偏移	硬件	0
2	查 TLB 比标记与有效位	硬件	0
3	逐级查页表取页框号	硬件	L 次
4	判存在位，为 0 发缺页中断	硬件	0
5	调页 / 置换 / 脏页写回	操作系统	访盘
6	拼物理地址、查 Cache 取数	硬件	0 或 1 次

数字例子：32 位虚址、页 2MB（大页）→ $k = 21$，页号 11 位→ 2048 页，偏移 `0x1FFFFF`，页号 `va>>21`。页变大只改了 k，后 6 步一步没变。

模板/流水线

```
/* 一次访存的完整流水线，任何一支都可能被单独提问 */
W = va & (page_size - 1);
P = va >> k;
if (tlb_tag(P) == P && tlb_valid(P)) {
    F = tlb_frame(P);                       /* 命中：1 次访存搞定 */
} else {
    for (s = 0; s < L; s++)                 /* 逐级查表，L 次访存 */
        pte = read_mem(base[s] + idx[s]*4);
    if (!pte.present) {
        page_fault();                       /* 内中断，转内核态 */
        if (pte.modified) write_back(pte);   /* 脏页先写回外存 */
        F = alloc_or_replace(P);            /* 置换算法在这里 */
        pte.present = 1; pte.modified = 0; pte.accessed = 1;
        tlb_fill(P, F);
        retry_instruction();                /* 重新执行本条指令 */
    }
    F = pte.frame; tlb_fill(P, F);
}
pa = (F << k) | W;
if (cache_hit(pa)) return cache_read(pa);
return mem_read(pa);                        /* 未命中再访存并调入 */
```

缺页中断是内中断、在指令执行期间产生，写成外中断或「执行后」就错 · 缺页返回是重新执行本条指令，不是下一条 · TLB 与页表必须同步更新，否则会反复缺页

软硬件互斥代码模板

真题年份 | 选择常考，大题未考 | 服务于 PV 大题第（1）问「为什么必须互斥」与四准则辨析，PV 大题见总表 2009/2011/2013/2015/2017/2020 各行

必背结论

- 临界区四部：进入区、临界区、退出区、剩余区；四准则：空闲让进、忙则等待、有限等待、让权等待。
- 关中断只对单 CPU 有效，多处理机上别的 CPU 照样能进；开关中断是特权指令，用户程序不能用。
- TestAndSet 把「读旧值」与「置 1」做成一条原子指令；Swap 原子交换 lock 与局部 key，二者本质相同。
- 二者都靠 while 空转，只满足前三条，不满足让权等待。
- 只有记录型信号量的 P 操作能实现让权等待；整型信号量仍是忙等。
- 单标志法必须交替用（违空闲让进）；双标志先检查不能互斥；双标志后检查会饥饿。
- 共享变量（如信号量 S 的 value）本身就是临界资源，对它读写必须互斥。
- 判定四步：先看「检查」与「置标志」谁在前→再找「两个同时进」或「两个都进不去」的反例→再看进不去时是空转还是阻塞→最后逐条对四准则。
- 单标志法用 turn 交替、初值就定死谁先进，所以违背空闲让进；双标志后检查两人都置 true 后互相等，会饿死。
- Peterson 的 turn 是「最后写入者吃亏」，能保证至少一个进程进得去，所以不饥饿。

方法	互斥	空闲让进	有限等待	让权等待
单标志法	能	否	能	否
双标志先检查	否	能	能	否
双标志后检查	能	否	否	否
Peterson	能	能	能	否
关中断	能	能	能	否
TS 指令 / Swap	能	能	能	否
记录型信号量	能	能	能	能

模板/流水线

```
/* ① 关中断：单 CPU 最省事，多处理机无效 */
void enter_cs(void) { disable_int(); }   /* 特权指令，只能内核态执行 */
void exit_cs(void)  { enable_int();  }

/* ② TestAndSet：硬件保证「读旧值 + 置 1」不可分割 */
int TestAndSet(int *lock) {
    int old = *lock;      /* 读旧值 */
    *lock = 1;            /* 立即置 1，与上一行原子 */
    return old;
}
while (TestAndSet(&lock)) ;   /* 返回 1 表示别人占着，继续空转 */
/* 临界区 */
lock = 0;                     /* 退出区：普通写即可释放 */

/* ③ Swap：原子交换，key 是每个进程自己的局部变量 */
void Swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }
int key = 1;
do { Swap(&lock, &key); } while (key);   /* key 被换成 0 才算拿到锁 */
/* 临界区 */
lock = 0;

/* ④ 记录型信号量：唯一能实现让权等待的 */
void wait(sem *S)  { S->value--; if (S->value < 0) block(S->list); }
void signal(sem *S){ S->value++; if (S->value <= 0) wakeup(S->list); }
```

关中断只对单 CPU 有效，多处理机上另一个核照样进临界区 · TS 与 Swap 只是写法不同，判定结论完全一样，都是忙等 · 能互斥不等于四准则全满足，Peterson 就偏偏缺让权等待

三种物理结构总表

真题年份 | 2011/2014（各 7 分） | 约三年一次大题，选择几乎每年一问

必背结论

- 物理结构只回答一句话：这个文件占了哪些盘块、靠什么记下来。
- 连续分配：FCB 记〈起始块号，块数〉或〈起始块号，结束块号〉；支持随机访问；有外部碎片；插入/删除要整段挪。2011 第 (1) 问答”连续更合适”，理由是数据一次写入不再修改、连续存放寻道距离最短、随机访问效率最高。
- 隐式链接：FCB 只记起始块号；每块末尾留指针，指针占 $\lceil \log_2 N \rceil$ 位（N 为总块数）；只能顺序访问；无外部碎片，但指针吃空间、断链即全丢。
- 显式链接（FAT）：全盘一张表，行数 = 块数，每行位数 = 块号位数；开机把表读入内存，所以能随机访问；缺点是表占内存。
- 索引分配：每个文件一张索引表，索引块自身也占盘块；可随机访问；无外部碎片。
- 两个量必须分开：FAT 表的大小≠ FAT 能表示的文件最大长度。
- FCB 与索引结点分离后，目录项只剩〈文件名，索引结点号〉，所以一个盘块能放更多目录项：设目录项 16B，4KB 块就能放 256 项。
- 2011 第 (2) 问：FCB 集中存放更好——按名查找时只需读 FCB 所在的那几块，减少磁头移动和访盘次数。
- 统一数字例子：4KB 盘块、4B 块号→ $N = 2^{32}$、$n = 1024$；隐式链接单块可用 $4096 - 4 = 4092$B；FAT 表项 2B → 2^{16} 簇→最大文件 256MB、表大小 128KB。

结构	FCB 记什么	随机访问	外部碎片	最大文件长度	读第 k 块访盘（k 从 1 数）
连续	起始块号 + 块数	可以	有	受连续空闲区限制	1
隐式链接	只记起始块号	不行	无	块数×（块大小 − 指针）	k
显式链接 FAT	只记起始块号	可以（查内存表）	无	簇数×簇大小	1
索引	索引块号	可以	无	见混合索引公式	2

索引结构”读第 k 块” = 索引块 1 次 + 数据块 1 次；题目说索引结点已在内存就只算 1 次。表里的 k 从 1 数，代码里的 k 从 0 数，差 1。

模板/流水线

```
/* 三种结构：访问第 k 个逻辑块（0 起）需要读几个盘块 */
int Reads_Cont(int k) { return 1; }          /* 起始块号 + k，直接算，读 1 次 */
int Reads_Link(int k) { return k + 1; }      /* 第 0 块读 1 次，必须顺着链走到第 k 块 */
int Reads_FAT (int k) { return 1; }          /* 表已在内存，查表得簇号，直接读 1 簇 */

/* 索引：索引结点在内存 → 级别 + 1；不在内存 → 级别 + 2 */
int Reads_Index(int level, int inodeInMem) {
    return level + 1 + (inodeInMem ? 0 : 1);
}

/* 隐式链接最大长度：1KB 块、4B 指针 → 指针 32 位 → 2^32 块 → 单块可用 1020B */
double LinkMaxGB(void) {
    return 4294967296.0 * 1020 / 1073741824.0;   /* = 4080 GB，2014 第（2）问答案 */
}
```

隐式链接与显式链接名字常被叫反（FAT 才叫显式）· 连续分配”外部碎片”别答成内部碎片 · 索引结构别漏算索引块本身占的盘块 · 2011 第 (1) 问要答连续并给出”一次写入不修改”这个前提

连续分配插入删除次数

真题年份 | 2014（7 分） | 两个小问全在这一题里，模板默写即可

必背结论

- 铁律：外存不能就地修改，任何” 挪一块” = 读 1 次 + 写 1 次 = 2 次访盘。
- 插入为第 p 条（共 N 条）有两条路：把第 p 条之后整体后移（动 $N-p+1$ 条），或把前 $p-1$ 条整体前移。题目给” 前后都有空闲空间” 就是让你取小的那个。
- 2014：$p=30, N=200$ →后移 171 条、前移 29 条→取 29 → $29\times2=58$ 次。
- 新记录已在内存→只写 1 次；总数 $=58+1=59$ 次。
- 插在第 1 条或第 $N+1$ 条时挪动条数为 0，只写 1 次——公式自动覆盖这种边界。
- 删除第 p 条：前移 $p-1$ 条或后移 $N-p$ 条，取小→ $T=\min(p-1,\ N-p)\times2$；删完块数减 1。
- FCB 变化：起始块号和块数都变；若 FCB 记的是〈起始，结束〉，则只改起始块号。
- 链接分配插入：前 $p-2$ 块只读；第 $p-1$ 块读 + 写（改指针）；新块只写→合计 $p+1$（$p=30\to31$ 次）。
- 链接分配删除：顺链读到第 $p-1$ 块用 $p-1$ 次读，改指针再写 1 次→共 p 次，但一个数据块都不用挪。
- 索引分配插入最省：索引表在内存时只写新数据块 1 次；若索引块也要回写，再加 1 次读 + 1 次写。这就是” 为什么大文件用索引” 的答题落点。
- 量级感受：每次访盘按 10ms 估，59 次约 0.59s ——所以连续分配” 读得快但改得慢”，答题时要写出这个代价。
- 通用公式：连续插入 $T=\min(p-1,\ N-p+1)\times2+1$；链接插入 $T=(p-2)+2+1$。
- 官方评分说明：2014 第 (1) 问的 FCB 变化若不含” 起始地址” 和” 文件大小” 两项，该小问不给分。

模板/流水线

```
/* 连续分配：内存中的记录插入为第 p 条，共 n 条，每块 1 条 */
int InsCont(int n, int p) {
    int back  = n - p + 1;        /* 第 p 条及之后整体后移的条数 */
    int front = p - 1;            /* 前 p-1 条整体前移的条数 */
    int m = (front < back) ? front : back;   /* 挪的条数取小，最省 */
    return m * 2 + 1;             /* 每条读一次写一次；新记录已在内存只写 1 次 */
}
/* 连续分配：删除第 p 条，共 n 条 */
int DelCont(int n, int p) {
    int back  = n - p;            /* 后面 n-p 条往前填 */
    int front = p - 1;            /* 前面 p-1 条往后填 */
    int m = (front < back) ? front : back;
    return m * 2;                 /* 只挪，没有新记录要写 */
}
/* 链接分配：插为第 p 条（p >= 2） */
int InsLink(int p) {
    int i, t = 0;
    for (i = 1; i <= p - 2; i++) t += 1;  /* 顺链走到第 p-2 块：只读 */
    t += 2;                               /* 第 p-1 块：读出，再写回新指针 */
    t += 1;                               /* 新插入块：只写一次，指针指向原第 p 块 */
    return t;                             /* = p + 1，2014 答案 31 */
}
/* 链接分配最大长度：1KB 块、4B 指针 */
double LinkMax(void) { return 4294967296.0 * (1024 - 4); }  /* ≈ 4080 GB */
```

只算读不算写（挪一块必须读写各一次）· ” 已在内存” 的新记录又被算了一次读 · 把” 前移 29 条” 算成 30 条（第 p 条本身不动）· FCB 只答块数不改起始块号

混合索引最大文件长度

真题年份 | 2012/2018、2022、2026（文件大题） | 四年重复同一个式子，全书最高频计算

必背结论

- 第一步永远先算” 一个索引块能放几个地址项”：$n=\dfrac{\text{块大小}}{\text{地址项大小}}$。4KB/4B → 1024；256B/4B → 64；1KB/4B → 256。

- 最大长度 $=(a_1\times 1+a_2n+a_3n^2+a_4n^3)\times$ 块大小，最后一步必须乘数据块大小，否则算出来的只是块数。
- 4KB 块 + 4B 项时：一个直接项 = 4KB，一个一级项 = 4MB，一个二级项 = 4GB，一个三级项 = 4TB；记这四个数比记公式更快。
- 2018：8 直接 + 一/二/三级各 1 → 32KB + 4MB + 4GB + 4TB。
- 2026 结构：5 直接 + 一/二/三级各 1（4KB 块、4B 项）→ 20KB + 4MB + 4GB + 4TB。
- 2010 选择 30：4 直接 + 2 一级 + 1 二级，块 256B、项 4B → $n=64$ → $(4+128+4096)\times 256\text{B}=1082368\text{B}=1057\text{KB}$。
- 2012：容量 4TB、块 1KB →块数 $4\text{TB}/1\text{KB}=2^{32}$ →块号要 32 位 = 4B；512B 索引区→ 128 项→最大 128KB。
- 2012 第 (2) 问变形：〈起始块号 6B，块数 2B〉+ 504B 直接项（6B/项）→ $2^{16}\times 1\text{KB}+84\text{KB}=2^{26}\text{B}+84\text{KB}$；要取最长则起始块号与块数各 4B（各能寻址 2^{32} 块 = 4TB）。
- 反向题（给长度求级数）：$4\text{GB}/4\text{KB}=2^{20}$ 块，而 $n=1024$、$n^2=1048576$，$1+1024<2^{20}<1+1024+1048576$ →一级、二级都要用，三级不用。
- 2018 第 (2) 问：inode 总数 $=2^{20}$簇 $\times$ $4\text{KB}/64\text{B}=2^{26}=64\text{M}$；5600B 文件占 2 簇→数据簇够 256M 个→受 inode 限制，最多 64M 个文件。
- 2018 第 (3) 问：6KB 小于 32KB 只读直接项；40KB 超过 32KB 还要多读一个一级索引块→时间不同。问” 获取最后簇号的时间” 就是在问级数。
- 与单个文件长度无关的量只有一个：索引结点总数（2013 选择 26 选 A）。索引块与数据块题目不说就默认同大。
- 回代检验：把答案块数再乘块大小，若量级差 1024 倍就是漏乘或漏除；把 n、n^2、n^3 依次列出来乘，别跳步。
- 地址项为什么常是 4B：块号空间 2^{32} 就需要 32 位；若盘更大（2012 第 (2) 问）就得用 6B，索引区能装的项数随之变少，所以” 块号占几字节” 和” 能装几项” 是一对反比关系。

级别	每个地址项管几块	4KB 块、4B 项时
直接	1	4KB
一级	n = 1024	4MB
二级	n^2 = 1048576	4GB
三级	n^3 = 1073741824	4TB

模板/流水线

```
/* BS 块大小，PS 地址项大小；a1 直接、a2 一级、a3 二级、a4 三级 */
long long MaxFile(int BS, int PS, int a1, int a2, int a3, int a4) {
    long long n = BS / PS;                /* ① 一个索引块放几个地址项 */
    long long blk = a1;                   /* ② 直接项：每项 1 块 */
    blk += (long long)a2 * n;             /*    一级：每项 n 块 */
    blk += (long long)a3 * n * n;         /*    二级：每项 n^2 块 */
    blk += (long long)a4 * n * n * n;     /*    三级：每项 n^3 块 */
    return blk * (long long)BS;           /* ③ 别忘了再乘数据块大小 */
}
/* 反向：给定块数 m，求最少用几级（a1 直接、a2 一级、a3 二级） */
int LevelOf(long long m, int a1, int n) {
    if (m <= a1) return 1;
    if (m <= a1 + (long long)n) return 2;
    if (m <= a1 + (long long)n + (long long)n * n) return 3;
    return 4;
}
/* 2018 调法：MaxFile(4096,4,8,1,1,1) → 32KB+4MB+4GB+4TB */
/* 2012 第 (1) 问：块号位数 = log2(4TB/1KB) = 32 位 = 4B；512/4 = 128 项 → 128KB */
```

漏乘数据块大小（只报”4228 块”）· 把 n^2 算成 $2n$ · 题目问” 哪几级间接地址项” 时要排除直接项 · 2012 的” 块号最少占几字节” 要用总块数取对数，不是用索引区大小

偏移定位与读盘块数

真题年份 | 2022/2026（文件大题） | 隔年就考，四步模板

必背结论

- 逻辑块号（从 0 编号）$K=\left\lfloor \frac{\text{偏移}}{\text{块大小}} \right\rfloor$，块内偏移 = 偏移 mod 块大小。用 0 起编号，商就是第几块，不必管余数。
- 定级（0 起）：$K<a_1$ 直接；$K<a_1+n$ 一级；$K<a_1+n+n^2$ 二级；否则三级。
- 读盘块数（索引结点已在内存）= 级别 + 1：直接 1、一级 2、二级 3、三级 4；索引结点不在内存再 +1。
- 2026：4KB 块、5 个直接项、$n=1024$；偏移 21460 → $K=5$ →落在第 6 块（1 起）→一级间接→读 2 个盘块。
- 索引结点号≠盘块号：2022 的”10” 是结点号，答案要写”10 号索引结点所在的盘块 + 30 号数据块”。
- inode 所在盘块 = inode 表起始块号 + $\left\lfloor \frac{inode\text{号}}{\text{每块}inode\text{数}} \right\rfloor$，每块 inode 数 = $\frac{\text{块大小}}{inode\text{大小}}$，$4096/128=32$。
- 2026：表从 100 号块起连续 4096 块→容量 $4096\times 4\text{KB}=16\text{MB}$ → inode 总数 $16\text{MB}/128\text{B}=131072=2^{17}$ → 最多存 131072 个文件（受 inode 数限制，不受盘块数限制）。
- 2015 选择 29：10 直接 + 一/二级各 1、1KB 块、4B 项（$n=256$）→ 1234 偏移在第 2 块（直接，1 次）；307400 在第 301 块（二级，3 次）。
- 打开文件读第一个字节的完整链：目录项→ inode 块（1 次）→数据块（1 次）；2022 第 (3) 问 course 目录已在内存→就是这 2 块。
- 为什么坚持 0 起编号：1 起编号要判断” 余数是否为 0 才进位”，多一个判断就多一处错；0 起直接取商，21460 ÷ 4096 = 5 就是第 5 号块。
- 题目问” 最多需要读多少盘块” 就按最坏路径答：二级索引要读两个索引块加一个数据块，别心疼，写满 3。
- 2026 第 (1) 问三小问是三套公式：inode 块号用整除、读盘块数用级别 + 1、最多文件数用 inode 总数；分三行写，别串成一团。

级别	块序号 K 范围（0 起）	5 直接 + n=1024 时
直接	0 至 a1−1	0 至 4
一级	a1 至 a1+n−1	5 至 1028
二级	a1+n 至 a1+n+n²−1	1029 至 1049604
三级	余下	1049605 起

模板/流水线

```
/* 输入文件内偏移 off，返回索引级别 1..4（1 直接、2 一级、3 二级、4 三级） */
int Level(long long off, int BS, int a1, int a2, int a3) {
    long long n = BS / 4;            /* 4B 地址项 */
    long long K = off / BS;          /* ① 逻辑块号，0 起：商就是块号 */
    if (K < a1) return 1;            /* ② 直接 */
    K -= a1;
    if (K < (long long)a2 * n) return 2;            /* ③ 一级 */
    K -= (long long)a2 * n;
    if (K < (long long)a3 * n * n) return 3;        /* ④ 二级 */
    return 4;                                       /* ⑤ 三级 */
}
/* 读盘块数：级别 + 1，索引结点不在内存再 +1 */
int Reads(long long off, int BS, int a1, int a2, int a3, int inodeInMem) {
    return Level(off, BS, a1, a2, a3) + 1 + (inodeInMem ? 0 : 1);
}
/* inode 号 id 所在盘块：表从 start 号块起连续存放 */
long long InodeBlk(long long id, int BS, int IS, long long start) {
    return start + id / (BS / IS);    /* 4096/128 = 32 个 inode 一块 */
}
/* 2026：21460/4096 = 5 → 一级 → 2 块；inode 总数 4096*32 = 131072 */
```

把索引结点号当盘块号写（2022 的 10）· 用 1 起编号又忘了余数不为 0 要进位 · ” 最多读几个盘块” 要含索引块但不含题

目已在内存的索引结点 · 二级索引漏算其中一层索引块

FAT 表大小与路径查找

真题年份 | 2016（9 分） | 三问全是固定套路，选择也常考空闲块管理结构

必背结论

- FAT 就是一个数组：行数 = 簇（块）数，每行位数 = 簇号位数。表项 t 字节→最多 2^{8t} 个簇（16 位→ 2^{16} 簇）。
- FAT 最大长度 $= 2^{8t} \times t$ 字节；2016：$2^{16} \times 2\text{B} = 2^{17}\text{B} = 128\text{KB}$。
- 单个文件最大长度 = 簇数 × 簇大小；2016：$2^{16} \times 4\text{KB} = 2^{28}\text{B} = 256\text{MB}$。
- 表项值：−1（全 1）表示文件最后一块，−2（0）表示空闲——所以 FAT 顺便管空闲空间，一表两用。
- 目录项只存文件名 + 第一个簇号，其余簇号在 FAT 里；所以”file1 的第 3 簇” 要查两次表才能得到。
- 簇链数格子：file1 为 $100 \to 106 \to 108 \to -1$，则 106 存在 100 号表项、108 存在 106 号表项、108 号表项里是 −1；规律是” 表项号 = 前一个簇号，表项内容 = 下一个簇号”。
- 目录本身也是文件：设目录项 16B，4KB 块一块放 256 项；目录文件长大一样要顺着 FAT 找簇。
- 按名存取：查目录项得首簇→查 FAT 得后续簇→读数据簇。FAT 已在内存，查表不计访盘次数。
- 逐级查找读盘数：路径深度为 d 层目录→读 d 个目录文件所在簇 + 1 个数据簇（全无缓存时）；2016 给了”FAT 和 dir 已在内存”，所以只算 2 个。
- 2016 第 (4) 问：第 5000 个字节→ $5000/4096 = 1$ 余 904 →第 2 个簇→ FAT[100]=106 →要读 48 号簇（dir1 目录文件）和 106 号簇，共 2 个。
- FAT 与索引的分工要一句话说清：FAT 全盘一张表，表的大小只与总簇数有关；索引每个文件一张表，表的大小与文件个数成正比。
- 第 (1) 问写目录内容时只写〈文件名，首簇号〉两列，把 106、108 也写进目录项就错了，它们在 FAT 里。
- 表项数的上限由表项宽度决定：16 位→ 65536 簇，配 4KB 簇→单文件上限 256MB，同时也意味着这个 FAT 最多管 65536 个簇。

场景（路径 dir/dir1/file1，第 5000 字节）	必须读的簇	个数
FAT 与 dir 已在内存	48、106	2
全无缓存（连 dir 都要读）	1、48、106	3

模板/流水线

```
/* 表项 t 字节 → 簇数；FAT 表大小；单文件最大长度 */
long long FATClusters(int t) { return 1LL << (8 * t); }        /* t=2 → 65536 */
long long FATSize(int t)     { return FATClusters(t) * t; }    /* 128KB */
long long FATMaxFile(int t, int clusterSize) {
    return FATClusters(t) * clusterSize;                       /* 2^16*4KB = 256MB */
}
/* 第 off 个字节 (0 起) 落在第 k 个簇：k = off / 簇大小，0 起 */
int FATCluster(int fat[], int start, long long off, int CS) {
    long long k = off / CS, i;
    int c = start;
    for (i = 0; i < k; i++) c = fat[c];   /* 表在内存，查表不算访盘 */
    return c;
}
/* 逐级查找：要读的簇序列 = 每层目录文件所在簇 + 最后数据簇 */
void FATPath(int d1, int d2, int start, int fat[], long long off, int CS) {
    printf("read %d, %d, %d\n", d1, d2, FATCluster(fat, start, off, CS));
}
/* 目录项号与表项号互查：表项号 = 前驱簇号 */
```

FAT 表大小与单文件最大长度写反 · 以为目录项里存着全部簇号 · 第 5000 字节算成第 1 个簇（余数不为 0 要进一位）· 把 −2（空闲）当结束标志

删除目录的元数据操作

真题年份 | 2026（文件大题） | 元数据清单题，按” 目录项→目录块→ inode →位图→超级块” 顺序背

必背结论

- 删目录 dir1 要动五处：父目录中的目录项、dir1 目录文件占的盘块、dir1 的索引结点、磁盘位示图、索引结点位示图（再加超级块的空闲计数）。
- 顺序固定：先清空（非空则递归删内部所有文件和子目录，自底向上），再删父目录里的目录项，最后回收盘块与 inode。
- 回收盘块 = 磁盘位示图对应位清 0；回收 inode = 结点位示图对应位清 0，结点号可再分配。
- 删目录项 = 把该项标记为空（或让后续项前移覆盖），父目录的文件长度、修改时间随之改变——目录也是文件，长度和时间戳是元数据。
- 目录非空不能直接删；题目说”dir1 为空” 才可跳过递归这一步。
- 反向的” 创建目录” 要对称背：父目录加一项、结点位图置 1、磁盘位图置 1（放”.” 和”..”）、超级块计数减 1。
- 硬链接只删目录项并把计数减 1，计数为 0 才释放 inode 与盘块（2022 考过”inode 号相同即硬链接”）。
- 删除已打开的文件：目录项立刻消失，但只要有进程还打开着，inode 与盘块要等最后一个进程关闭才回收。
- 超级块要同步：空闲盘块数 + dir1 占的块数，空闲 inode 数 + 1。
- 考场就写这四句：在父目录中删除目录项、回收目录文件所占盘块并清磁盘位图、释放 inode 并清结点位图、更新超级块空闲计数。四句齐了就满分。

元数据	要做的操作	记在哪个位图
父目录的目录项	删除 dir1 那一项，目录长度改变	不涉及
dir1 目录文件占的盘块	全部回收	磁盘位示图清 0
dir1 的索引结点	回收，结点号可再用	索引结点位示图清 0
dir1 内部文件（若非空）	先递归删除，各自同上	两张位图都要改
超级块	空闲盘块数、空闲 inode 数增加	不涉及

模板/流水线

```
/* 删除目录 d（父目录 parent）的元数据操作清单，按此顺序答 */
void DeleteDir(Dir *parent, Dir *d) {
    int i;
    /* ① 非空先清空：自底向上递归删子项 */
    for (i = 0; i < d->n; i++) {
        if (d->item[i].isDir) DeleteDir(d, &d->item[i]);
        else Unlink(d, &d->item[i]);           /* 删文件：回收盘块 + inode */
    }
    /* ② 父目录：删除 dir1 的目录项（文件名 + 索引结点号） */
    RemoveEntry(parent, d->name);
    /* ③ 回收 dir1 目录文件占用的盘块：磁盘位示图对应位清 0 */
    for (i = 0; i < d->nblk; i++) DiskBitmap[d->blk[i]] = 0;
    /* ④ 回收 dir1 的索引结点：结点位示图对应位清 0，结点号可再用 */
    InodeBitmap[d->ino] = 0;
    /* ⑤ 超级块同步 */
    Super.freeBlocks += d->nblk;
    Super.freeInodes += 1;
}
/* 硬链接：Unlink 里只做 link--，link 归零才走 ③④⑤ */
```

只删目录项、不释放 inode 和位图（空间泄漏）· 漏掉磁盘位示图只改结点位示图 · 目录非空硬删 · 硬链接文件删一个目录项就把 inode 释放了（应计数减 1）

磁盘调度手算模板

真题年份 | 2010/2019，各 7 分 | 磁头移动量 + 总耗时，隔几年一次

必背结论

- 总时间 = 寻道时间 + 请求数×旋转延迟 + 请求数×传输时间。
- 一圈时间 $T = \dfrac{60}{r}$ 秒（r 为 rpm）；6000rpm → $T = 10$ms。给 rpm 先换算再动手。
- 旋转延迟 $= \dfrac{T}{2}$（随机扇区平均找半圈）；传输时间 $= \dfrac{\text{读的扇区数}}{\text{每道扇区数}} \times T$。
- 移动量按相邻磁道逐段累加；CSCAN 的回程（一端→另一端）必须算进去。
- 2010：100 → 120 → 30 → 50 → 90 = 20+90+20+40 = 170 磁道→ $170 + 4 \times 5 + 4 \times 0.1 = 190.4$ms。
- 2019 SSTF：簇号 ÷ 1000 = 柱面号→ 100、60、101、110；磁头在 85 → 100(15) → 101(1) → 110(9) → 60(50) = 75 磁道。该题只问次序不问时间，所以就只算柱面距离。
- SSTF 会饥饿；SCAN 是电梯、走到端点（或最大请求）才掉头；CSCAN 方向不变、到端立刻回另一端；LOOK 到最大请求就掉头。
- 按老师口径，真题多半按 LOOK 算（到最大请求就掉头）；题目明确给出磁道号范围（如 0 到 199）时老老实实走到端点。
- 换 Flash/SSD：无寻道无旋转，按到达顺序的 FCFS 最优、最公平（2010 第 (3) 问的答案）。
- 数格子就画五列：步、当前、下一、移动、累计；每一步都要写，最后一行就是答案。
- 时间占比要有感觉：2010 里寻道 170ms、旋转 20ms、传输 0.4ms ——机械盘瓶颈全在寻道，所以调度算法只优化寻道，这句话常作为” 为什么这样调度” 的得分点。
- 若题目给” 相邻磁道移动时间 2ms”，就把移动磁道数乘 2；给” 每读一个扇区 0.1ms” 就直接乘扇区数，不必绕圈数。
- 算 CSCAN 的顺序技巧：先把请求排序，把大于当前磁头的按升序走完，再从最小请求按升序继续走，中间那一段跳跃就是回程。
- 饥饿风险排序：SSTF 最重（远端请求可能永远等），SCAN 次之，CSCAN 与 FCFS 基本没有——选择题常问” 哪个会饥饿”。

算法	服务顺序（磁头 100，向大）	移动磁道数
FCFS	50、90、30、120	240
SSTF	90、120、50、30	130
SCAN（走到 199 端点）	120、199、90、50、30	268
SCAN（真题 LOOK 口径）	120、90、50、30	110
CSCAN	120、30、50、90	170

模板/流水线

```
/* 数格子表：步 | 当前 | 下一 | 移动 | 累计（2010 CSCAN 填法）
    1 | 100 | 120 |  20 |  20
    2 | 120 |  30 |  90 | 110   <- 回程，CSCAN 一端直接跳另一端
    3 |  30 |  50 |  20 | 130
    4 |  50 |  90 |  40 | 170          */

/* 算出移动磁道数 d 后统一结算时间（单位 ms） */
double TotalTime(int d, int n, int rpm, int secPerReq, int secPerTrack) {
    double T    = 60000.0 / rpm;                        /* 一圈，6000rpm → 10 */
    double seek = d;                                    /* 相邻磁道 1ms，d 就是 ms */
    double rot  = T / 2.0;                              /* 旋转延迟：半圈 → 5 */
    double xfer = (double)secPerReq / secPerTrack * T;  /* 传输：1/100 圈 → 0.1 */
    return seek + n * (rot + xfer);                     /* 170+4*(5+0.1)=190.4 */
}
```

CSCAN 漏掉 120 → 30 这 90 个磁道的回程 · 旋转延迟按一整圈算（应半圈）· 转速 rpm 不换成” 每秒圈数” · 4 个请求只乘一次旋转延迟（每个请求都要转半圈）

磁盘地址与簇号换算

真题年份 | 2019（7 分） | 地址三元组 + 簇号换算，一题三问一次拿满

必背结论

- 地址 = 〈柱面号，盘面号（磁头号），扇区号〉；柱面号就是磁道号，各盘面上同号磁道组成一个柱面。
- 容量 = 柱面数×盘面数×每道扇区数×扇区大小；2019：$300 \times 10 \times 200 \times 512\text{B} = 307200000\text{B}$。
- 每柱面扇区数 = 盘面数×每道扇区数 = 2000；每柱面簇数 = $2000 \div 2 = 1000$；每磁道簇数 = $200 \div 2 = 100$。
- 簇号→地址：柱面号 = 簇号÷每柱面簇数（整除）；余数 r 再÷每磁道簇数得盘面号；扇区号 =（余数 mod 每磁道簇数）×每簇扇区数。
- 2019 第 (3) 问：100530 →柱面 100，$r = 530$ →盘面 5，$530 \bmod 100 = 30$ →扇区 $30 \times 2 = 60$。答〈100，5，60〉。
- 反向：簇号 = 柱面号×每柱面簇数 + 盘面号×每磁道簇数 + 扇区号÷每簇扇区数。
- 边界校验：簇号 99 与 100 在同柱面不同盘面（一个是〈c,0,198〉，一个是〈c,1,0〉），算完回代一遍最稳。
- 簇号→物理地址由 I/O 系统的磁盘驱动程序（设备驱动程序）完成，不是文件系统也不是用户程序。
- 2019 SSTF 只用到柱面号：簇号整除 1000 → 100、60、101、110，调度算法只看柱面。
- 若每簇只有 1 个扇区，簇号就退化成扇区的线性编号，公式照样成立。
- 单位看清：307200000B 约合 293MiB、307MB；题目要 MB 时按题面给的是 10^6 还是 2^{20} 换算。
- 扇区号一律从 0 编号，一个簇占连续 k 个扇区，所以簇在磁道内的起始扇区 = 簇内序号 $\times k$。
- 三步不要合成一条长式子：先整除求柱面、再整除求盘面、最后取模乘 k 求扇区，分步才不会被中间量带错。

量	2019 的算法
容量	$300 \times 10 \times 200 \times 512\text{B} = 307200000\text{B}$
每柱面扇区	$10 \times 200 = 2000$
每柱面簇	$2000 \div 2 = 1000$
每磁道簇	$200 \div 2 = 100$
100530 的柱面	$100530 \div 1000 = 100$ 余 530

模板/流水线

```
/* 容量：C 柱面、H 盘面、S 每道扇区、sz 扇区大小 */
long long Capacity(int C, int H, int S, int sz) {
    return (long long)C * H * S * sz;              /* 2019: 307200000 */
}
/* 簇号 → 柱面/盘面/扇区，k = 每簇扇区数 */
void Cluster2Addr(long long c, int H, int S, int k,
                  int *cyl, int *head, int *sec) {
    long long perCyl = (long long)H * S / k;       /* 每柱面簇数 1000 */
    long long perTrk = S / k;                      /* 每磁道簇数 100 */
    long long r;
    *cyl  = (int)(c / perCyl);                     /* 100530/1000 = 100 */
    r     = c % perCyl;                            /* r = 530 */
    *head = (int)(r / perTrk);                     /* 530/100 = 5 */
    *sec  = (int)((r % perTrk) * k);               /* 30*2 = 60 */
}
/* 地址 → 簇号（反向核对用） */
long long Addr2Cluster(int cyl, int head, int sec, int H, int S, int k) {
    return (long long)cyl * (H * S / k) + (long long)head * (S / k) + sec / k;
}
```

把柱面号答成盘面号（簇号整除得到的是柱面）· 扇区号忘乘每簇扇区数（应 60 不是 30）· 算每柱面簇数时忘了先乘盘面

数 · 把换算程序答成文件系统或用户程序

位示图大小计算

真题年份 | 2010 | 其余以选择出现（2023 选择 25 页框位图、2024 选择 26）

必背结论

- 一位管一个盘块（或页框、簇）：位数 = 总块数；字节数 = $\lceil$块数/8$\rceil$。
- 2010：16384 块→ 16384 位 = $16384/8 = 2048\text{B} = 2\text{KB}$，正好塞进题目给的 2KB 内存。
- 2010 官方写法：$16384/32 = 512$ 字，$512 \times 4\text{B} = 2\text{KB}$（按 32 位字组织）。
- 2023 选择 25：页 4KB、物理内存 16GB →页框 $2^{34}/2^{12} = 2^{22}$ 个→位图 2^{22} 位 $= 2^{19}\text{B} = 512\text{KB}$。
- 2024 选择 26：位图法占用的外存空间只与总块数有关，与当前空闲块数无关（位图大小固定）。
- 换算数字：16384 个 4KB 块→容量 $16384 \times 4\text{KB} = 64\text{MB}$；131072 个 inode 的位图 $= 131072/8 = 16\text{KB}$；2KB 位图占 1 个 4KB 盘块。
- 按字组织（32 位/字）：(i, j) 从 0 起→块号 $= 32i + j$；反向 $i = \lfloor$块号$/32\rfloor$，$j =$ 块号 mod 32。
- 分配 = 找 0 置 1 并记块号；回收 = 置 0。0 表示空闲、1 表示已用（按题目定义，2026 题面就是这么写的）。
- 找连续多个空闲块（连续分配要的）就顺着位图扫连续 0，扫到就切成一段。
- 与别的空闲管理对比：位图空间固定但整张要占内存/外存；空闲表随碎片变长；成组链接法用栈分批读入。
- 两种单位别混：按位算” 位数 = 块数”，按字算” 字数 = 块数/32”（字长 32 位）；两种算法的结果必须相等，不等就是字长用错。
- 判断题套路：题目说” 用 X KB 内存记录 N 个块”，先用 $X \times 8$ 与 N 比，对得上就答位示图，顺手写出位数公式。
- 位图还能反推容量：位数 × 块大小 = 可管理的总容量，16384 位配 4KB 块刚好 64MB。
- 磁盘位示图与索引结点位示图是两张独立的表（2026 题面明确给了两张），别合成一张算。

场景	位数（= 块数）	位图大小
2010：16384 个盘块	16384	2KB
2023：16GB 内存、4KB 页	2^22	512KB
2026：2^17 个 inode	131072	16KB
4TB 磁盘、4KB 块	2^30	128MB

模板/流水线

```
/* 位示图字节数：N 个块（页/簇） */
long long BitmapBytes(long long N) { return (N + 7) / 8; }
/* 按 32 位字组织：块号 b 与行列 (i,j)，均从 0 起 */
void Block2Pos(long long b, int *i, int *j) { *i = (int)(b / 32); *j = (int)(b % 32); }
long long Pos2Block(int i, int j)          { return (long long)i * 32 + j; }
/* 分配一个空闲块：扫位图，0 为空闲 */
long long Alloc(unsigned int *map, long long N) {
    long long b;
    for (b = 0; b < N; b++)
        if (((map[b / 32] >> (b % 32)) & 1) == 0) {
            map[b / 32] |= (1u << (b % 32));      /* 置 1 = 已用 */
            return b;
        }
    return -1;                                    /* 无空闲块 */
}
/* 回收：map[b/32] &= ~(1u << (b%32)); */
```

位图大小按” 当前空闲块数” 算（应始终按总块数）· 字节数忘向上取整 · 把 16384 位算成 16384 字节 · 行列编号从 1 起却仍用 32i+j

第四编

计算机网络

综合应用题第 47 题，共 9 分 | 性价比最低，放最后做，先拿第 1 小问

分章分值与大题分布

真题年份 | 2010/2011/2015/2017/2019/2021/2022/2025 | 链路层是前三章里唯一上过大题的章节，其余年份的 47 题都落在网络层与传输层

必背结论

- 计网共 25 分：8 道单选 16 分 + 第 47 题约 9 分；四科稳定比约 45/45/35/25。
- 第 1 章只出选择题（分层模型、性能指标），大题从不单独考，但 47 题的第 (1) 问几乎永远是一道时延题。
- 第 2 章只出选择题（奈氏与香农、编码、中继器与集线器），从 2009 到 2026 没有一道物理层大题。
- 第 3 章是前三章的重心：帧格式、CSMA/CD、滑动窗口、交换机，这四块每一个都能直接变成 47 题的一问。
- 47 题按问给分：第 (1) 问基本送分，算不出结果也要把公式、代入、单位三行摆上去。
- 链路层卷面占 4 到 6 分选择；一旦出现在 47 题里，单题就能顶到 9 到 11 分，是性价比最高的一章。
- 第 3 章选择题的高频点只有五个：CSMA/CD 三问、最短帧长、交换机转发、GBN 窗口、802.11 地址。
- 前三章适合 40 分钟一轮复习：公式背完直接做 2010、2017 两道大题，再做 8 道选择，不要通读教材。
- 第 1 章唯一必须记住的计算是「三种交换方式的总时间」：电路交换是建链时间加 x/b 加传播时延，中间结点不存储转发；报文交换是 $(m+1)T$；分组交换见第 2 节。
- 前三章只求不丢分：25 分里它们占 8 分左右，剩下 17 分在 IP 与 TCP，别在前三章深挖偏题。

章节	选择分值	出现过大题的年份	在 47 题里的角色
第 1 章体系结构	2 到 4 分	未单独考	时延、单位换算、封装层次
第 2 章物理层	2 到 4 分	未单独考	带宽与速率、编码、集线器
第 3 章数据链路层	4 到 6 分	2010 2011 2015 2017 2019 2021 2022 2025	帧格式、CSMA/CD、GBN、交换机
第 4 到 6 章网络传输应用	8 到 12 分	2009 2012 2013 2014 2016 2018 2020 2023 2024 2026	子网、路由、TCP

前三章合计约 8 分：第 1、2 章各 1 道选择，第 3 章 2 到 3 道选择，再加 47 题里链路层的那一两问。

模板/流水线

```
/* 第 47 题 12 分钟流水线：按顺序写，能给分的地方一个不漏 */
void solve() {
    /* 1. 单位先统一：速率与频率按 10 的幂，存储容量按 2 的幂；
          题面写明「1 兆 = 10^6」就以题面为准（每年都有人在这里丢分） */
    /* 2. 画链：主机--路由器--主机。数清 m 个存储转发结点，
          每个结点都要「收完再发」，所以每过一站多一个 T */
    /* 3. 把 T = L*8/b 和 tau = d/v 先算成 ms，写在草稿最上面，
          后面每一问都会复用到这两个数 */
    /* 4. 认题型：本文件第 2 到第 10 节，一类对一个模板，不要现场推 */
    /* 5. 每问写四行：公式、代入、结果、单位。只写答案的题，过程分一分没有 */
    /* 6. 复查三个陷阱：
          a) 「有效」数据速率：分子扣掉 18 B 首尾，分母的 T1 仍按整帧
          b) tau 是单程，争用期是 2tau，别把两者当成一个数
          c) 窗口 K 与序号比特 n 是两回事：GBN 要求 K <= 2^n - 1 */
}
```

第 1、2 章的选择题必须全对，那里没有计算量只有记忆 · 第 47 题不要留空，公式写对就有分 · 单位写成 ms 还是 s 必须在题末标清楚

时延四兄弟 · 计算模板

真题年份 | 2010/2011/2015/2017/2019/2021 | 不单独命题，但几乎是每道 47 题的第 (1) 问

必背结论

- 发送时延 $T = L/b$：把 L bit 推上链路要多久，只与帧长和速率有关，与距离无关。
- 传播时延 $\tau = d/v$：比特在介质里跑 d 米要多久，只与距离和介质有关，与帧长无关；铜缆与光纤 $v \approx 2 \times 10^8$ m/s，无线 3×10^8 m/s。
- 处理时延与排队时延：题目不提就当 0；一旦写「路由器同时只能处理一个」，就必须画时间线，套不出公式。
- 时延带宽积 $= \tau \cdot b$，单位是 bit：链路管道里能装下的比特数；给时延带宽积和带宽可以反解 τ。
- 总时延 = 发送 + 传播 + 处理 + 排队；一段链路上有 k 个结点就叠 k 份。
- 分组交换总时间：n 个分组、每个传输时延 T、经过 m 个存储转发结点且不计传播时延，为 $nT + mT$。
- 若要计传播时延且路径上 1 个路由器：$nT + \tau + T + \tau$；报文交换经 m 个路由器：$(m+1)T$。
- 计算顺序固定：先全部换成 bit 与 s，再算 T 与 τ，最后才相加；中途换单位必错。
- 计算题里的 RTT 一律当 2τ（往返传播）用，不要搬书上「含发送与确认」的定义来做题。
- 数字例子：$n = 1000$、$T = 0.08$ ms、$m = 2$ 时总时间 80.16 ms；同样数据量改用报文交换则只有 $(m+1)T$，但那个 T 要按整个报文算，通常远大于分组。

传输时延问「帧多长、口多快」，传播时延问「线多长、跑多快」，两件事没有任何关系。

模板/流水线

```
/* 输入：帧长 L 字节、速率 b bit/s、距离 d 米、结点数 m、分组数 n
   输出：T、tau、时延带宽积、总时间，单位统一 ms */
double v = 2e8;                    /* 铜缆/光纤传播速度 */
double T   = L * 8.0 / b * 1e3;    /* 发送时延 ms */
double tau = d / v * 1e3;          /* 传播时延 ms，单程 */
double pipe = d / v * b;           /* 时延带宽积 bit */

double total(int mode) {
    if (mode == 0) return n * T + m * T;              /* 只算存储转发 */
    if (mode == 1) return n * T + tau + T + tau;      /* 1 个路由器加传播 */
    if (mode == 2) return (m + 1) * T;                /* 报文交换 */
    return -1;                                        /* 有处理时延：画图 */
}

/* 例：L=1518B, b=10Mb/s -> T=1.2144ms；d=2000m -> tau=0.01ms
   两者差 121 倍：同一条线路上，帧的发送时间可以远大于传播时间
   例：980000B 文件、分组 1000B（其中头 20B）、2 个路由器
       n = 980000/980 = 1000，T = 0.08ms，总时间 = 1000T + 2T = 80.16ms */
```

争用期是 2τ，不是 τ · 速率换算用 10 的幂、容量用 2 的幂，别搞反 · 题目说「忽略传播时延」时公式里一个 τ 都不许出现

信道容量 · 奈氏与香农

真题年份 | 选择常考，大题未考 | 物理层只出单选，服务于 47 题里「链路带宽、传输速率」的参数设定

必背结论

- 奈奎斯特（理想无噪声低通）：波特率上限 $2W$ Baud，比特率上限 $C = 2W \log_2 V$，V 是码元种类数即波形数。
- 香农（有噪声）：$C = W \log_2(1 + S/N)$，与波形数无关，是理论上限。
- 信噪比换算：$10 \lg(S/N)$ dB；10 dB 对应 $S/N = 10$，20 dB 对应 100，30 dB 对应 1000，40 dB 对应 10000。
- 一个码元携带 $\log_2 V$ bit，故 $R = B \cdot \log_2 V$，B 是波特率。
- 两个定理同时给：取小的那个，$C = \min(C_{奈}, C_{香})$。
- 香农式里当 $S/N \gg 1$ 时可近似 $\log_2(S/N)$，30 dB 就是每赫兹约 9.97 bit，口算按 10 bit/Hz。
- 求波形数的题：先由香农定上限，再由 $2W \log_2 V$ 不小于目标速率反解最小的 V，取 2 的整数次幂。
- dB 速记：3 dB 约 2 倍、6 dB 约 4 倍、10 dB 是 10 倍，考试只给 10 的整数倍，直接用表背四个数最快。
- 香农给的是「超不过」的上限，题目问实际速率还要同时满足调制方式的 $\log_2 V$，两个条件并列写出来。

奈氏管「无噪声时码元能画几种」，香农管「有噪声时最多能传多少」，两者都要满足时取小。

模板/流水线

```c
/* 已知带宽 W(Hz)、信噪比 db、波形数 V，求最大数据率 */
#include <math.h>
double cap(double W, double db, int V) {
    double snr = pow(10.0, db / 10.0);      /* dB 转倍数，用 10lg 不是 20lg */
    double cn  = 2 * W * log2((double)V);   /* 奈氏：无噪声上限 */
    double cs  = W * log2(1 + snr);         /* 香农：有噪声上限 */
    return cn < cs ? cn : cs;               /* 取小者 */
}

/* 反解波形数：要求速率 R，最小 V 满足 2W*log2(V) >= R */
int minV(double W, double R) {
    int V = 2;
    while (2 * W * log2((double)V) < R) V *= 2;   /* V 取 2 的幂 */
    return V;
}

/* 例：W=3kHz、V=4、30dB
   奈氏 2*3000*2 = 12 kb/s；香农 3000*log2(1001) 约 29.9 kb/s
   答案 12 kb/s，取小者
   例：W=4kHz、30dB，要求 20 kb/s，则 log2(V) >= 20000/8000 = 2.5，
       V 取 8（3 bit 每码元，可达 24 kb/s） */
```

信噪比用 10 lg，别写成 20 lg · 波特率上限是 $2W$，比特率才乘 $\log_2 V$ · 两式都给时一定取 min，相加或相乘都是错

编码与调制速查

真题年份 | 选择常考，大题未考 | 曼彻斯特与 4B/5B 只出单选，但 2010/2011 的以太网大题默认「以太网用曼彻斯特编码」

必背结论

- 曼彻斯特编码：每个比特中间必有一次跳变，高到低记 1、低到高记 0（也可约定相反），1 bit 用 2 个码元，编码效率 50%，自带位同步，10BASE-T 用它。
- 差分曼彻斯特：看比特开始处有无跳变，有跳变记 0、无跳变记 1，抗干扰强于曼彻斯特，效率仍是 50%。
- 4B/5B：每 4 bit 映射成 5 bit，效率 80%，保证不出现长串 0，100BASE-TX 用它再配 MLT-3。
- NRZ：一个码元一个比特，效率 100%，但没有同步能力；NRZI 用「遇 1 跳变」携带时钟，USB 用它。
- 编码效率 = 数据率 / 波特率；反过来求波特率：曼彻斯特乘 2，4B/5B 乘 1.25。
- 基带传输把数字信号直接送上线路，宽带传输要先用 ASK、FSK、PSK 把数字调成模拟。
- 调制效率：ASK/FSK/PSK 各 1 bit 每码元，QAM-16 是 4 bit，QAM-64 是 6 bit。
- 曼彻斯特每个比特有两种可能的波形，接收端靠中间的跳变提取时钟，因此不需要额外的同步信号。
- 4B/5B 的 5 bit 码里 16 个用于数据、其余作控制码，任何组合都不会出现三个以上连 0，跳变足够密。
- 数字例子：给比特串 1011 画曼彻斯特，每位中间画一次跳变，画完数出 4 次跳变、8 个码元，波特率就是数据率的 2 倍。

曼彻斯特的波特率是数据率的 2 倍，4B/5B 是 1.25 倍；给数据率求带宽先换波特率，再用 $2W$ 反解。

模板/流水线

```c
/* 已知数据率 R(bit/s) 和编码方式，求波特率 B 与所需基带带宽 W */
double baud(double R, int type) {
    switch (type) {
        case 0: return R;          /* NRZ：1 bit 每码元 */
        case 1: return R * 2;      /* 曼彻斯特与差分曼彻斯特 */
        case 2: return R * 1.25;   /* 4B/5B */
        case 3: return R / 4;      /* QAM-16，4 bit 每码元 */
        case 4: return R / 6;      /* QAM-64，6 bit 每码元 */
    }
    return R;
}
double needW(double R, int type) {
    return baud(R, type) / 2.0;    /* 奈氏：2W 是波特率上限 */
}

/* 例：10 Mb/s 用曼彻斯特 -> 20 MBaud -> 需 10 MHz
```

```
    100 Mb/s 用 4B/5B -> 125 MBaud -> 需 62.5 MHz
  这就是「提速必须换编码，否则带宽撑不住」的原因 */
```

曼彻斯特效率是 50%，不是 100% · 4B/5B 是 4 变 5，波特率比数据率大不是小 · 码元种类与比特数用 $\log_2 V$ 换算，别按倍数硬记

停等 GBN SR 利用率

真题年份 | 2010/2017/2025 | 2010 停等算有效速率、2017 GBN 加捎带确认算利用率、2025 又考 GBN 窗口，三年一轮

必背结论

- 停止等待：$W_T = 1$，$W_R = 1$，$U = \dfrac{T_1}{T_1 + 2\tau + T_2}$。
- 后退 N 帧 GBN：$W_T \le 2^n - 1$，$W_R = 1$；一帧出错，该帧及其后所有帧全部重发；确认是累计的。
- 选择重传 SR：$W_T + W_R \le 2^n$，通常取 $W_T = W_R = 2^{n-1}$；只重发出错那一帧，接收方要有缓冲。
- 通用利用率：$U = \dfrac{K\,T_1}{T_1 + 2\tau + T_2}$，$K$ 是发送窗口；忽略确认帧取 $T_2 = 0$，两帧等长取 $T_1 = T_2$，给 RTT 就取 $2\tau = \text{RTT}$。
- 实际数据率 $= U \cdot b$，也等于「有效数据量除以发送周期」；算「有效」时分子扣掉 18 B 首尾，分母里的 T_1 仍按整帧算。
- 反解：$U = 100\%$ 时 $K = (T_1 + 2\tau + T_2)/T_1$，再取最小 n 满足 $K \le 2^n - 1$（GBN）或 $K \le 2^{n-1}$（SR）。
- $U = 100\%$ 的含义：发完第 K 帧的瞬间第 1 帧的确认刚好回来，信道一秒都没停，此时实际速率等于带宽。
- 发送周期 $= T_1 + 2\tau + T_2$，从发出第一帧算到收到它的确认，分子分母都按这个口径，别只算单程。
- 数字例子：帧 1000 B、1 Mb/s、RTT 20 ms、两帧等长，$T_1 = 8$ ms，$K = 1$ 时 $U = 8/36 \approx 22\%$；要 100% 需 $K = 4.5$ 取 5，则 $2^n - 1 \ge 5$，$n = 3$。
- 停等就是 $K = 1$ 的特例，两个公式不必分开背；GBN 把窗口取满 $2^n - 1$ 时效率最高。

窗口与序号的关系只有两条：GBN 是 $K \le 2^n - 1$，SR 是 $K \le 2^{n-1}$，背错一个整题就崩。

模板/流水线

```
/* 已知帧长 L(B)、速率 b、单程 tau、窗口 K、确认帧长 L2，求利用率 */
double util(double L, double b, double tau, int K, double L2) {
    double T1 = L * 8.0 / b;                    /* 数据帧传输时延 */
    double T2 = (L2 < 0) ? 0 : L2 * 8.0 / b;    /* 忽略确认帧时传 -1 */
    return K * T1 / (T1 + 2 * tau + T2);
}

/* U=100% 反解窗口，再求最小序号比特 n */
int minBits(double L, double b, double tau, double L2, int isSR) {
    double T1 = L * 8.0 / b, T2 = L2 * 8.0 / b;
    int K = (int)ceil((T1 + 2 * tau + T2) / T1);      /* 需要的窗口 */
    int n = 1;
    while (isSR ? (K > (1 << (n - 1))) : (K > (1 << n) - 1)) n++;
    return n;
}

/* 例（2017 型）：n=3 -> K=7；T1=T2=0.08ms，RTT=0.96ms
   U = 7*0.08/(0.08+0.96+0.08) = 0.56/1.12 = 50%
   例（2010 型）：K=1，T1=1.2144ms，T2=0.0512ms，2tau=0.02ms
   U = 1.2144/1.2856 约 94.5%，有效速率 = 1500*8/1.2856ms 约 9.33 Mb/s */
```

「有效数据率」分子用 1500 B、分母用 1518 B，同一题两处不同数字 · GBN 是累计确认，确认号 3 表示 0、1、2 都收到了 · SR 的窗口上限是 2^{n-1} 不是 $2^n - 1$

CSMA/CD 争用期与帧长

真题年份 | 2010（冲突检测与以太网效率）· 2022（冲突域） | 唯一以争用期当第 (1) 问的年份，之后只作小问出现

必背结论

- 十六字：先听后发、边听边发、冲突停发、随机重发。
- 传播时延 $\tau = d/v$；争用期 $= 2\tau$；最短帧长 $L_{\min} = 2\tau \cdot b$。
- 检测冲突的时间：最短 τ（双方同时发，各等一个单程），最长 2τ（一方快发完时对方才发）——记「最短 τ、最长 2τ」。
- 以太网最短帧长 $L_{\min} = 64$ B；10 Mb/s 以太网争用期 $= 51.2\ \mu s$，按 $2\tau \cdot v$ 反推最大跨距约 5.12 km。
- 帧必须发满争用期才敢认为「没冲突」：帧太短，发完了冲突才传回来，发送方会误判成功。
- 二进制指数退避：第 k 次重传从 $\{0, 1, \ldots, 2^k - 1\}$ 中随机取 r，推迟 $r \cdot 2\tau$；$k = \min(\text{重传次数}, 10)$；重传到 16 次仍失败就丢弃。
- 收到小于 64 B 的帧一律当冲突碎片丢弃，不算有效帧。
- 以太网不重传也不确认：冲突后由上层超时重发，链路层只负责「发不出去就退避」。
- 100 Mb/s 与 1000 Mb/s 以太网因跨距缩短，争用期仍按 512 bit 的发送时间设计，所以最短帧长还是 64 B。
- 冲突检测靠边发边听：发完还没听到冲突才算成功，所以「发送时间不小于争用期」是安全线。

最短帧长的本质是「发送时间不小于争用期」：$L/b \geq 2\tau$，两边乘 b 就是 $L \geq 2\tau b$。

模板/流水线

```
/* 已知电缆长度 d、传播速度 v、数据率 b，求争用期、最短帧长与退避时间 */
double tau  = d / v;              /* 单程传播时延 */
double win  = 2 * tau;            /* 争用期 */
double Lmin = win * b;            /* 最短帧长 bit，再除 8 变字节 */

/* 给定标准最短帧长反推最大跨距 */
double maxD(double Lmin, double b, double v) {
    return (Lmin * 8 / b) / 2 * v; /* 2tau = L/b，故 d = tau*v */
}

/* 第 k 次重传的平均推迟时间：r 平均取 (2^k - 1)/2 */
double backoff(int k, double win) {
    if (k > 10) k = 10;           /* 超过 10 次不再增大 */
    return ((1 << k) - 1) / 2.0 * win;
}

/* 例（2010 型）：d=2000m、v=2e8 -> tau=10us、2tau=20us、b=10Mb/s
   Lmin = 20us * 10Mb/s = 200 bit = 25 B，小于 64 B
   标准取 64 B 更保守，等效于更长的争用期
   第 4 次重传：r 属于 0..15，平均 7.5*20us = 150us */
```

争用期是往返的 2τ，用 τ 乘速率会算小一半 · 「最短 τ、最长 2τ」方向别反 · k 到 10 封顶、16 次丢弃，这两条每年都有人漏

以太网帧逐字段解析

真题年份 | 2011（帧解析加 ARP 与 IP 转发）· 2010（1518 B 与 64 B） | 给一段十六进制就要能切出全部字段

必背结论

- 前导码 7 B（10101010）加 SFD 1 B（10101011）：物理层加的，用于位同步，不计入帧长。
- 目的 MAC 6 B、源 MAC 6 B、类型 2 B、数据 46 到 1500 B、FCS 4 B（CRC-32）。
- 首部加尾部共 18 B；最短帧 $46 + 18 = 64$ B，最长帧 $1500 + 18 = 1518$ B；加 802.1Q 标签 4 B 后最长 1522 B。
- 数据段最短 46 B 是由 64 B 反推的：$64 - 6 - 6 - 2 - 4 = 46$，不是独立规定。
- 类型字段：0x0800 是 IP，0x0806 是 ARP，0x8100 是 802.1Q，0x86DD 是 IPv6。
- 目的 MAC 首字节最低位为 1 表示组播或广播，FF-FF-FF-FF-FF-FF 全 1 是广播地址。
- 抓包给的十六进制一般从目的 MAC 开始（不含前导码），此时目的 MAC 偏移 0 到 5，类型字段偏移 12 到 13。
- MAC 地址 48 位即 6 字节，写成 12 个十六进制数；前三字节是厂商号 OUI，后三字节由厂商分配。
- 解析顺序固定：先看前 6 字节定目的，再看类型定上层，最后用总长减 18 得数据长度。

帧长 = 数据长度 + 18；只要题里出现 1518、64、1522 这三个数，考的就是这节。

模板/流水线

```
/* f[] 是从目的 MAC 开始的一帧十六进制字节流，len 为总字节数 */
void parse(unsigned char *f, int len) {
    int i;
    printf(" 目的 MAC ");  for (i = 0; i < 6; i++)  printf("%02X-", f[i]);
    printf("\n 源 MAC  "); for (i = 6; i < 12; i++) printf("%02X-", f[i]);

    int type = (f[12] << 8) | f[13];      /* 类型字段，偏移 12 */
    if (type == 0x0800) printf("\n 上送 IP\n");
    if (type == 0x0806) printf("\n 上送 ARP\n");
    if (type == 0x8100) {                 /* VLAN 帧，数据整体后移 4 B */
        printf("\nVLAN 标签，VID = %d\n", ((f[14] & 0x0F) << 8) | f[15]);
        type = (f[16] << 8) | f[17];
    }

    int dataLen = len - 18;               /* 减掉 6+6+2+4 */
    if (len < 64)         printf("\n 小于 64 B：冲突碎片，丢弃\n");
    if (dataLen > 1500)   printf("\n 超长帧，非法\n");
    printf(" 数据 %d B，FCS %02X%02X%02X%02X\n",
           dataLen, f[len-4], f[len-3], f[len-2], f[len-1]);
}

/* 例：收到 64 B 帧 -> 数据 46 B；收到 1518 B -> 数据 1500 B
      带 VLAN 的 1522 B -> 去掉 4 B 标签后按 1518 B 判最长帧 */
```

前导码 8 B 不计入帧长，别加到 64 或 1518 里 · VLAN 帧要把类型字段偏移加 4 再读 · 数据段 46 B 是最小值不是固定值

交换机自学习与作用域

真题年份 | 2019/2021/2022 | 2019 交换机与路由器、2021 交换表学习、2022 冲突域判定，连续三年都碰

必背结论

- 自学习三步：进来就记「源 MAC 加入口端口」；再查目的 MAC，查到只从该口转发，查不到从除入口外的所有口泛洪；回程时表里已有对方，单口转发。
- 转发表项有老化时间，默认 300 s，老化后重新泛洪。
- 直通式交换：只读到目的 MAC 那 6 B 就开始转发，延迟 $= 48/b$；存储转发：收完整帧校验后再发，时间按最短帧 64 B 算 $= 512/b$。
- 冲突域：集线器不隔离，整个集线器连同其主机算一个冲突域；交换机每个端口一个冲突域；路由器隔离冲突域。
- 广播域：交换机默认不隔离，同一 VLAN 是一个广播域；路由器接口与 VLAN 隔离广播域。
- 数冲突域就是数交换机端口数，把每个集线器并到它所连的那个端口里；数广播域就是数路由器接口数加各 VLAN 数。
- 交换机每个端口是独立冲突域，也是独立带宽：全双工下同一端口可同时收发，不再共享。
- 广播帧在同一广播域内泛洪，路由器不转发广播；所以「能不能收到广播」是判广播域最快的标准。
- 转发只看目的 MAC，与 IP 无关；同一交换机下跨子网也要先把帧发给网关的 MAC。

集线器是物理层设备，只放大不判断；交换机是链路层设备，按 MAC 表转发——这一句决定所有作用域题。

模板/流水线

```
/* 转发表：表项 = MAC + 端口 + 老化计时 */
struct item { unsigned char mac[6]; int port; int ttl; } tab[64];

void arrive(unsigned char *src, unsigned char *dst, int in) {
    int i, hit = -1;
    for (i = 0; i < n; i++)                    /* ① 进来先记源 */
        if (same(tab[i].mac, src)) { tab[i].port = in; tab[i].ttl = 300; hit = i; }
    if (hit < 0) add(tab, src, in);
```

```
    for (i = 0; i < n; i++)                    /* ② 再查目的 */
        if (same(tab[i].mac, dst)) {           /* 查到：单播 */
            forward(tab[i].port);
            return;
        }
    flood_except(in);                          /* 查不到：除入口外泛洪 */
}

/* 直通与存储转发的时间，b 为端口速率 */
double cutThrough(double b)  { return 6 * 8.0 / b; }    /* 48 bit */
double storeFwd(double b)    { return 64 * 8.0 / b; }   /* 512 bit */

/* 例：100 Mb/s 下直通 0.48us、存储转发 5.12us、整帧 1518 B 则 121.44us
   例：8 口交换机，3 个口各接一个集线器（各带 2 台主机），5 个口各接 1 台主机
       冲突域 8 个、广播域 1 个；把其中一个口换成路由器，广播域变 2 */
```

直通按 6 B 算时间、存储转发按 64 B，两者不能混 · 集线器不隔离冲突域，交换机默认不隔离广播域 · 回程方向表里已有对方，只走一个口，不要再泛洪

802.11 地址四字段

真题年份 | 2022（802.11 地址含义） | 大题里只作为一个小问出现，选择题偶见

必背结论

- 802.11 帧最多四个地址，各 6 B，408 只考前三个，第四个只在无线桥接 WDS 时出现。
- 铁律：地址 1 永远是当前接收方 RA，地址 2 永远是当前发送方 TA，地址 3 是最终对端（源或目的，看方向）。
- 去往 AP（主机 A 发给 AP）：地址 1 是 AP 的 BSSID，地址 2 是主机 A，地址 3 是目的主机 B。
- 来自 AP（AP 发给主机 B）：地址 1 是主机 B，地址 2 是 AP，地址 3 是源主机 A。
- 无线主机之间不能直接发，A 到 B 必须两跳：A 发给 AP 一帧，AP 再发给 B 一帧，两帧的地址组合完全不同。
- 帧头里地址 1 从偏移 4 开始（帧控制 2 B 加持续时间 2 B 之后），地址 2 偏移 10，地址 3 偏移 16。
- 802.11 用 CSMA/CA 而不是 CSMA/CD：无线不能边发边听，改用 ACK 确认；可选 RTS/CTS 解决隐蔽站。
- 帧间间隔固定大小顺序：SIFS 最短，DIFS 次之，退避只在 DIFS 之后进行，这决定了谁先抢到信道。
- 地址 4 出现即表示 WDS 桥接，考卷只问前三个；看到四个地址不必慌，按 1 收 2 发 3 对端照样填。

方向	地址 1 接收方	地址 2 发送方	地址 3 对端
A 经 AP 发往 B	AP 的 BSSID	A 的 MAC	B 的 MAC
AP 转发给 B	B 的 MAC	AP 的 BSSID	A 的 MAC
有线侧进入 AP	目的主机 B 的 MAC	AP 的 BSSID	有线源主机 MAC

口诀「1 收 2 发 3 对端」：先判这一帧是谁在发给谁，再照表填，从不去猜。

模板/流水线

```
/* 按方向填地址：dir=0 主机 A 到 AP，dir=1 AP 到主机 B，dir=2 有线主机 C 经 AP 到 B */
void fillAddr(int dir, char *A, char *B, char *AP, char *C, char *out[3]) {
    if (dir == 0) {              /* 去往 AP：收方是 AP */
        out[0] = AP;             /* 地址 1 = BSSID */
        out[1] = A;              /* 地址 2 = 源主机 */
        out[2] = B;              /* 地址 3 = 目的主机 */
    } else if (dir == 1) {       /* 来自 AP：收方是主机 B */
        out[0] = B;              /* 地址 1 = 目的主机 */
        out[1] = AP;             /* 地址 2 = AP 自己 */
        out[2] = A;              /* 地址 3 = 源主机 */
    } else {                     /* 有线侧经 AP 发给 B */
        out[0] = B;              /* 地址 1 = 无线目的主机 */
        out[1] = AP;             /* 地址 2 = AP 自己 */
        out[2] = C;              /* 地址 3 = 有线源主机 */
    }
```

```
}

/* 例：A=00-AA、AP=00-11、B=00-BB
   A 发 AP 帧：地址 1=00-11、地址 2=00-AA、地址 3=00-BB
   AP 发 B 帧：地址 1=00-BB、地址 2=00-11、地址 3=00-AA
   两帧的地址 1、2 恰好互换，考的就是这个「换位」 */
```

地址 1 是接收方不是目的方，两者在 A 发给 AP 时重合，最容易记混 · 地址 3 去 AP 时是目的、来 AP 时是源 · 不要漏掉「A 到 B 要两帧」这件事

PPP 与 HDLC 及 VLAN

真题年份 | 选择常考，大题未考 | 服务 47 题里「链路层封装、帧长变化」这一类小问

必背结论

- PPP 帧：标志 0x7E、地址 0xFF、控制 0x03、协议 2 B、信息、FCS 2 B（可协商 4 B）。
- PPP 面向字节：异步传输用 0x7D 做字节填充（转义 0x7E 和 0x7D 本身），同步传输用零比特填充。
- PPP 只检错不纠错：无序号、无确认、不保证可靠交付；只支持点对点全双工；用 LCP 建链、NCP 配网络层。
- HDLC 帧：标志 0x7E、地址 1 B、控制 1 或 2 B、信息、FCS 2 或 4 B；面向比特，用零比特填充（连发 5 个 1 插一个 0）。
- PPP 与 HDLC 都以 0x7E 为标志，区别是 PPP 面向字节且不可靠，HDLC 面向比特且可靠（有编号有确认）。
- VLAN：802.1Q 标签 4 B，等于 TPID 0x8100（2 B）加 TCI（2 B，PRI 3 bit、DEI 1 bit、VID 12 bit），插在源 MAC 之后。
- VID 取值 0 到 4095，可用范围 1 到 4094；带标签后最长帧 $1518 + 4 = 1522$ B。
- 交换机端口：Access 口收发都不带标签，Trunk 口跨交换机时带标签。
- 不同 VLAN 之间不能直接通信，必须经路由器或三层交换机，这是 VLAN 隔离广播域的代价。
- PPP 与 HDLC 都是点对点协议，不能用于总线型广播网络；以太网才是广播型链路的代表。
- 零比特填充与字节填充都是透明传输手段，目的是让数据里出现 0x7E 时不被误认成帧标志。

看到类型字段 0x8100 就是 VLAN 帧，后面所有偏移整体加 4，这一条能救回一整问。

模板/流水线

```
/* 判断是否 VLAN 帧并给出真实类型，返回数据段起始偏移 */
int vlanCheck(unsigned char *f, int len, int *type) {
    *type = (f[12] << 8) | f[13];
    if (*type != 0x8100) return 14;          /* 普通帧，数据从 14 起 */
    int tci = (f[14] << 8) | f[15];
    int vid = tci & 0x0FFF;                  /* 低 12 位是 VID */
    *type = (f[16] << 8) | f[17];            /* 真实类型后移 4 B */
    printf("VLAN %d, PRI %d\n", vid, tci >> 13);
    return 18;                               /* 数据从 18 起 */
}

/* 零比特填充：连 5 个 1 后插 0（HDLC 与同步 PPP 用） */
void bitStuff(char *in, char *out) {
    int i, cnt = 0, j = 0;
    for (i = 0; in[i]; i++) {
        out[j++] = in[i];
        if (in[i] == '1') { if (++cnt == 5) { out[j++] = '0'; cnt = 0; } }
        else cnt = 0;
    }
    out[j] = 0;
}

/* 例：原串 01111111110（10 个 1）填成 01111101111100
      带 VLAN 的 1522 B 帧，去掉 4 B 标签后按 1518 B 判最长帧 */
```

802.1Q 标签插在源 MAC 之后，不是帧首 · 零比特填充是插 0，接收端对应删 0 · PPP 不可靠，别写成「有确认重传机制」

网络层大题题型地图

真题年份 | 2009/2012/2013/2014（42、43 题）/2015/2018（42、47 题）/2019/2020/2021/2022/2024/2025 | 计网历年真题（2009 到 2026）里网络层占大半，是唯一每年必出计算题的一章

必背结论

- 计网每年 1 题（第 47 题，9~11 分），个别年份 2 题（2014、2018）；其中网络层相关的至少占一半。
- 全部问法只有四个底层动作：算地址（子网/聚合）、查表（最长前缀/路由表）、改字段（TTL/校验和/分片）、改地址（ARP/NAT）。
- 拿到题先做三件事：画拓扑并把每条链路两端接口 IP 标上→把每个网段写成 a.b.c.d/N →看所有斜线长度是否一致（一致是定长，不一致是变长）。
- 第 47 题只有 9 分却要连算好几层，限时 12 到 15 分钟，先做第 1 小问保证有分。

题型	年份	拿到手先算什么
子网划分、写路由表、聚合	2009	先按” 每子网不少于 N 个地址” 定借位
IP/TCP 首部解析、TTL、填充	2012	先由首部长度字段乘 4 定出 IP 首部真长
自治系统、最长匹配、聚合	2013	先把每个目的网络写成前缀并按长度排序
OSPF 链路状态、Dijkstra	2014（42 题）	先把链路代价抄成邻接矩阵
路由表、IP 转发、TTL、LSI	2014（43 题）	先算目的 IP 与每条掩码的按位与
DHCP、ARP、掩码、默认网关	2015	先比网络号判同网，再定网关
最小生成树组网、TTL 可达性	2018（42 题）	先按 TTL 剩余值算还能过几跳
子网划分、广播地址、分片	2018（47 题）	先判目的地址是不是广播地址
交换机与路由器、NAT、定向广播	2019	先填 NAT 表（内网 IP 端口对公网 IP 端口）
局域网互联、路由转发、NAT	2020	先判同网，再逐跳写源与目的地址
DNS/HTTP 封装、ARP、交换表	2021	先写五层封装顺序与每层字节数
冲突域、DHCP、802.11 地址	2022	先数冲突域（交换机每个口一个）
RIP/OSPF/BGP、TTL、路径选择	2024	先按管理距离排出优先级
卫星链路、GBN、VLSM	2025	先按需求量降序排，再逐块对齐

模板/流水线

```
// 拿到第 47 题，先看这四步
① 数网段：图里几个网络、几台路由器、几个接口；
② 标前缀：每个网段写成 a.b.c.d/N，直接标在图上；
③ 拆小问：算地址（子网）/ 写表（路由）/ 改字段（TTL，分片）/ 改地址（ARP,NAT）；
④ 查同网：每台设备的掩码与网关配得对不对；
```

计网大题的分不在难，在” 每问写全”：目的网络、掩码、下一跳、接口四样齐了才给满分。

别把 2016/2017/2023/2026 的 TCP 与 GBN 题算进网络层 · 2014 与 2018 是两道大题，两问都要答 · 只写出” 网络号等于 IP 与掩码” 也有步骤分，空着是 0 分

IPv4 首部逐字段

真题年份 | 2012 | IP/TCP 首部联考，首部长度、总长度、TTL、填充各一小问

必背结论

- 固定部分 20 B，恰好 5 行、每行 4 B；带选项时最长 60 B，不足用填充补齐到 4 B 的倍数。
- 首部长度 4 位，单位 4 B：最小 5（即 20 B），最大 15（即 60 B）；总长度 16 位，单位 1 B，含首部加数据，上限 65535。
- 数据部分长度 = 总长度 − 首部长度（不是一律减 20，有选项时要减真首部）。
- 标识 16 位：同一原始数据报拆出的所有分片取值相同，用于重组。
- 标志 3 位：最高位保留为 0，DF 为 1 表示禁止分片，MF 为 1 表示后面还有分片。
- 片偏移 13 位，单位 8 B，指该片数据在原始数据部分里的偏移。
- TTL 8 位：每经一个路由器减 1，减到 0 就丢弃并发 ICMP 超时，作用是防环路。
- 协议 8 位：ICMP 为 1、IGMP 为 2、TCP 为 6、UDP 为 17、OSPF 为 89。
- 首部校验和 16 位，只校验首部；每经一个路由器必须重算，因为 TTL 变了。
- 源与目的地址各 32 位，点分十进制；MAC 是 6 B、十六进制、两个一组用连字符。

字段	位数	单位	一句话
版本	4	无	4 表示 IPv4
首部长度	4	4 B	存 5 就是 20 B
区分服务	8	无	一般不管
总长度	16	1 B	含首部，最大 65535
标识	16	无	同一数据报的各分片相同
标志	3	无	保留位、DF、MF
片偏移	13	8 B	起始字节号除 8
生存时间	8	跳	每跳减 1
协议	8	无	上层是 TCP 还是 UDP
首部校验和	16	无	只查首部，每跳重算
源地址	32	无	全程不变
目的地址	32	无	全程不变
选项与填充	0 到 40 B	1 B	补齐到 4 B 倍数

模板/流水线

```
// 解析一个 IP 数据报，只需 6 步
p     = 首部起始地址;
ver   = p[0] >> 4;                      // 版本：高 4 位
ihl   = (p[0] & 0x0F) * 4;              // 首部长度：单位 4B，得字节数
total = (p[2] << 8) | p[3];             // 总长度：单位 1B
data  = total - ihl;                    // 数据部分长度
ident = (p[4] << 8) | p[5];             // 标识：16 位
df    = (p[6] >> 6) & 1;                // 禁止分片
mf    = (p[6] >> 5) & 1;                // 后面还有分片
off   = (((p[6] & 0x1F) << 8) | p[7]) * 8;   // 片偏移乘 8 得字节数
ttl   = p[8];  proto = p[9];            // 生存时间、协议
// 校验和：只对首部 ihl 字节按 16 位反码求和
// 题目问"每跳哪些字段变"：TTL、首部校验和、目的 MAC、源 MAC
```

首部长度单位是 4 B 不是 1 B · 总长度单位 1 B、片偏移单位 8 B，三个单位别串 · TTL 只在路由器上减，交换机与集线器不减

IP 分片计算模板

真题年份 | 2012/2018（47 题） | 约两年一次，固定问” 第二片的总长度、片偏移、MF”

必背结论

- 每片可装数据 = MTU − 20，且必须是 8 的整数倍；不是就向下取到 8 的倍数。
- MTU 含首部，别只减数据；有选项时减真首部长度。
- 数据部分长度 = 总长度 − 20；片数 = 向上取整（数据长度除每片数据）。
- 片偏移 = 该片数据起始字节号除 8；MF 最后一片为 0，其余为 1；标识全部相同。
- 每片总长度 = 每片数据 + 20（每片都自带 20 B 首部）。
- 校验：各片数据长度之和等于原数据长度。
- 不是 8 的倍数时，每片数据向下取整到 8 的倍数，步长随之变小：MTU 为 1482 时每片只能放 1456 B（182 乘 8）。
- 练手例：总长度 3000、MTU 800，数据 2980，每片 780 不是 8 的倍数，向下取 776（97 乘 8），四片为 776、776、776、652，片偏移依次 0、97、194、291。
- DF 为 1 时禁止分片，超过 MTU 就丢弃并回 ICMP 不可达（差错类型是” 需要分片但 DF 置位”）。
- 重组只在目的主机做：按标识把同组分片归到一起，按片偏移排序，遇到 MF 为 0 的片就是末片，缺任何一片整包丢弃。
- 各分片只有总长度、片偏移、MF、首部校验和不同；标识、协议、源与目的地址全部相同。

片号	数据区间	数据长度	片偏移	MF	本片总长度
1	0 到 1479	1480	0	1	1500
2	1480 到 2959	1480	185	1	1500
3	2960 到 3979	1020	370	0	1040

（总长度 4000、首部 20、MTU 1500：数据 3980，每片 1480，三片之和 3980 对上）

模板/流水线

```
// 总长度 T、MTU M、首部 H = 20
d   = T - H;                          // 数据部分
cap = ((M - H) / 8) * 8;              // 每片数据，向下取 8 的倍数
start = 0; k = 1;
while (start < d) {
    len = (d - start < cap) ? (d - start) : cap;   // 末片可能不满
    off = start / 8;                               // 片偏移除 8
    mf  = (start + len < d) ? 1 : 0;
    printf(" 第%d 片 数据%d 片偏移%d MF%d 总长度%d\n", k, len, off, mf, len + H);
    start += len; k++;
}
```

把” 每片长度是 8 的倍数” 当成整片总长度，那说的只是数据部分 · 忘了 MTU 含首部，写成 MTU 就是每片数据长度 · 最后一片数据长度 = 数据长度 − 起始号（减完要加 1 才是个数）

子网划分三步法与 VLSM

真题年份 | 2009/2015/2018（47 题）/2025 | 几乎每年一问：掩码、可用主机数、广播地址

必背结论

- 三步：先由需求定主机号位数 n →借位得到新前缀→写掩码并算块大小。

- 可用主机数 = 2 的（32 减前缀）次方再减 2；子网数 = 2 的借位数次方（子网号全 0 全 1 可用，不扣）。
- 块大小 = 256 − 末字节掩码值；各子网基址 = 块大小乘 0、1、2、3。
- 网络地址 = IP 与掩码按位与；广播地址 = 网络地址 + 块大小 − 1；可用范围是两端各去一个。
- 末字节速算：248 = 255 − 7 得 5 个 1，即 /29；记牢 /25 为.128、/26 为.192、/27 为.224、/28 为.240、/29 为.248、/30 为.252、/32 为.255。
- VLSM：按需求量从大到小排，每块起始地址必须是自己的块大小的整数倍（对齐）。
- 只要求分 N 个子网、不限规模：子网号按前缀码排 0、10、110、1110、1111，最后一个写成全 1，第一个最大、末两个最小。
- 分配顺序必须从大到小：先给小需求会把地址切碎，后面的大块就再也对不齐。
- 点对点链路要 2 个地址时用 /30 正好；/31 只剩全 0 与全 1 两个值，常规不用于主机。
- 2025 把 VLSM 与 GBN 窗口、卫星链路放在同一题，先算完子网再算传输时间，别混在一起算。

192.168.1.0/24 划 4 个等长子网（借 2 位得 /26，块大小 64）：

子网	网络地址	广播地址	可用范围	可用数
1	192.168.1.0	192.168.1.63	.1 到.62	62
2	192.168.1.64	192.168.1.127	.65 到.126	62
3	192.168.1.128	192.168.1.191	.129 到.190	62
4	192.168.1.192	192.168.1.255	.193 到.254	62

VLSM：同一网段按 100、50、20、2 台的需求分四块：

需求	主机号位数	新前缀	块大小	分配结果
100	7（126 可用）	/25	128	192.168.1.0/25
50	6（62 可用）	/26	64	192.168.1.128/26
20	5（30 可用）	/27	32	192.168.1.192/27
2	2（2 可用）	/30	4	192.168.1.224/30

模板/流水线

```
// 定长：要求每个子网至少 H 台
n = 0; while ((1 << n) - 2 < H) n++;    // 主机号位数
p = 32 - n;                             // 新前缀
block = 1 << n;                         // 块大小
// 基址 = block 的整数倍；广播 = 基址 + block - 1
// 变长：需求降序，每块取刚够用的 n，基址向上对齐到自己的 block
// 校验：所有块不重叠、都在原网段内
```

减 2 只扣主机号，子网号不扣，两个”减不减”最常错 · 广播地址不是最后一个可用地址，还要再减 1（.63 是广播，.62 才是可用）· VLSM 分完必须检查起始地址是否对齐到自己块大小

路由聚合与最长前缀匹配

真题年份 | 2009/2013/2024 | 聚合是”只准增加一条路由”的唯一解法，最长前缀是查表题命门

必背结论

- 聚合三步：把地址写成二进制→从左找第一次出现不一样的位→从此位起全部置 0；相同位数就是聚合后的前缀长度。
- 聚合后必须验算：原来的每个地址都落在聚合结果里，否则就是聚合错了。

- 匹配五步：取目的 IP →用每项掩码长度截前 N 位→与该项目的网络比较→多个匹配取 N 最大→ 0.0.0.0/0 截 0 位恒匹配。
- 长前缀能匹配，短前缀一定也能匹配，所以必须显式取最长，不能” 第一个匹配就走”。
- 特定主机路由写 /32（32 位全 1），必须 32 位完全相等；默认路由写 /0，等价于” 互联网”。
- 聚合与子网划分互逆：把聚合出的 /19 按定长再切 4 块就是 /21。
- 2024 的 BGP 路径选择是同一套” 多命中取最优”：先比属性优先级，再逐条比下来。
- ” 只准加一条路由” 的写法：把必须走同一出口的两个网络聚合，写成一条带掩码的表项，绝不写两条。
- 聚合后出现” 空洞”（覆盖了并不存在的网段）可以接受，路由表只认前缀；但若把别的真实网络也盖进来，就必须回退一位重算。

目的地址 202.118.1.130 查表：

表项	截取结果	是否匹配
202.118.1.0/25	末字节 128，与 0 不等	否
202.118.1.128/25	末字节 128，相等	是（最长）
202.118.1.0/24	前三字节相等	是
0.0.0.0/0	截 0 位恒相等	是

取 /25 那条转发。聚合例：202.118.1.0/25 与 202.118.1.128/25 前 24 位相同、第 25 位不同，得 202.118.1.0/24；末字节 32、40、48、56 的前 19 位相同，聚合成 x.x.32.0/19。

模板/流水线

```
// 聚合：求一组网络的最长公共前缀
p = 0;
while (p < 32 && 对所有 i 都有 bit(net[i], p) == bit(net[0], p)) p++;
// 相同 p 位 → 结果 = net[0] 前 p 位 + 后面全 0，写成 /p
// 最长匹配：
best = -1;
for (每条表项 r)
    if ((D & mask(r.len)) == r.net && r.len > best) { best = r.len; out = r; }
if (best < 0) { 丢弃；回 ICMP 不可达；}     // 有默认路由时一定命中
// 输出 out 的下一跳与接口
```

> 聚合只保留到” 不一样的前一位” · 多命中随便挑一条 · 忘了 0.0.0.0/0 一定能匹配，就会答成” 丢弃”

路由表转发决策流程

真题年份 | 2014（43 题）/2019/2020 | 写路由表加判断出口，几乎每年一问

必背结论

- 四列：目的网络、子网掩码、下一跳、接口；没有掩码列时目的网络必须写成 a.b.c.d/N。
- 直连网络下一跳写”—” 或 Direct；非直连写离自己最近的那个路由器接口 IP。
- 决策顺序：命中本机某接口就交付本机→否则最长前缀匹配→同前缀再比管理距离与度量→最后才用默认路由。
- 优先级从高到低：特定主机路由 /32、具体网络路由、默认路由 /0。” 直连优先” 其实是它前缀更长，不是单独一条规则。
- 管理距离：直连 0、静态 1、eBGP 20、OSPF 110、RIP 120，越小越可信。
- 转发时改三样：TTL 减 1、首部校验和重算、源与目的 MAC 重写；IP 地址不动（NAT 例外）。
- 定向广播（主机号全 1 的网段广播）只在本子网内广播，路由器不转发；255.255.255.255 是受限广播。
- 收到一条新路由时：先比前缀长度，再比管理距离，最后比度量，三项都相同才做负载分担。

- 2014（43 题）的 LSI 更新只重跑一遍 Dijkstra，把受影响的目的网络与下一跳改掉，别把整张表推翻重写。
- 路由表里”接口”与”下一跳”必须成对；同一台路由器的不同端口属于不同网络。

2009 那题 R1 的路由表（四列写法：第三行是特定主机路由，第四行是默认路由）：

目的网络	子网掩码	下一跳	接口
202.118.1.0	255.255.255.128	—	L0
202.118.1.128	255.255.255.128	—	L0
202.118.3.2	255.255.255.255	202.118.2.2	L0
0.0.0.0	0.0.0.0	202.118.2.2	L0

模板/流水线

```
// 一台路由器收到目的地址为 D 的分组
best = -1;
for (每条表项 r)                                    // ① 最长前缀
    if ((D & mask(r.len)) == r.net && r.len > best) { best = r.len; hit = r; }
if (best < 0 && 有默认路由) hit = 默认路由;          // ② 兜底
if (best < 0) { 丢弃; 回 ICMP 目的不可达; return; }
if (hit. 下一跳 == "—") next = D; else next = hit. 下一跳;   // ③ 定下一跳
if (--TTL == 0) { 丢弃; 回 ICMP 超时; return; }     // ④ 改字段
重算首部校验和;
用 ARP 求 next 的 MAC，重封装后从 hit. 接口 发出;
```

下一条写成最终目的主机地址（应为最近的路由器接口）· 没有掩码列不加 /N 直接扣分 · TTL 减到 0 是丢弃，不是继续转发

ARP 全过程与帧的变化

真题年份 | 2011/2015/2021 | ARP 是跨层题的固定配角，问”请求帧目的 MAC”与”哪层地址在变”

必背结论

- 作用：已知 IP 求 MAC。同一网段才直接解析目的主机，跨网段解析的是默认网关的 MAC。
- 请求广播（目的 MAC 全 F），响应单播；两者都封装在以太网帧里，帧类型 0x0806，ARP 报文 28 B。
- 同网段 A 发 B：帧源 MAC 为 A、目的 MAC 为 B，IP 源为 A、目的为 B。
- 跨网段 A 发 B：帧源 MAC 为 A、目的 MAC 为网关，但 IP 源仍是 A、目的仍是 B（IP 全程不变）。
- 每经一个路由器：MAC 换一次、TTL 减 1、首部校验和重算，IP 地址不变。
- ARP 缓存有老化时间（约 20 分钟），避免对方换网卡后还发旧帧。
- 以太网帧头尾共 18 B：目的 MAC 6、源 MAC 6、类型 2、FCS 4；数据 46 到 1500 B。
- ARP 报文 28 B：硬件类型、协议类型、硬件地址长度、协议地址长度、操作码（1 请求、2 响应）、源 MAC、源 IP、目的 MAC、目的 IP。
- 广播帧进交换机后被泛洪到同一 VLAN 的所有端口，交换机不隔离广播域，这是 2021 交换表题的前提。
- 以太网帧最短 64 B、最长 1518 B（不含前导码），不够要填充；截断发生在末尾 FCS 之前。

场景	帧目的 MAC	帧源 MAC	IP 源	IP 目的
同网段 A 到 B	B	A	A	B
跨网段 A 到 B	网关	A	A	B
网关再转发	下一跳路由器	网关	A	B
最后一跳	B	路由器	A	B

模板/流水线

```
// A 给 B 发 IP 分组，掩码 M
if ((A_ip & M) == (B_ip & M)) {          // ① 同网段
    查 ARP 缓存 B；未命中则广播请求（目的 MAC 全 F）；
    B 单播响应；写入缓存；
    帧：目的 MAC = B，源 MAC = A；
} else {                                 // ② 跨网段
    G = 默认网关 IP；                     // 下一跳，不是最终目的
    广播 ARP 请求 G；网关单播响应；
    帧：目的 MAC = G，源 MAC = A，IP 源 = A，IP 目的 = B；
}
// ③ 每一跳：解帧 → 查表 → ARP 下一跳 → 重封装 → TTL-1 → 校验和重算
```

> 以为跨网段 ARP 解析的是目的主机，其实解析网关 · 以为 IP 每跳都变，只有 MAC 变（NAT 才改 IP）· ARP 响应也写成广播

DHCP 四报文与租期

真题年份 | 2015/2022 | 常与”源 IP 是什么、目的地址是什么”一起考，报文错一个就丢分

必背结论

- DHCP 是应用层协议，跑在 UDP 上：客户端 68、服务器 67。
- 四报文顺序：Discover（发现）、Offer（提供）、Request（请求）、ACK（确认）。
- 客户端发的两个报文源 IP 都是 0.0.0.0（它还没有地址），目的都是 255.255.255.255。
- 服务器发的两个报文源 IP 是服务器地址，目的通常仍是广播。
- 报文靠事务 ID 配对，不靠 MAC 地址。
- 租期：T1 为 50% 时单播向原服务器续租，T2 为 87.5% 时广播向任意服务器续租，到期未续则释放并重新 Discover。
- 中继代理把本网段广播转成单播发给另一个网段的服务器，源地址是代理自己的接口地址。
- 可能收到多台服务器的 Offer，客户端只对选中的那台发 Request，其余服务器收到广播 Request 后撤回自己的 Offer。
- 四个报文之外还有 Release（释放）、Decline（检测到地址冲突）、Inform（只要配置不要地址），知道名字即可。
- 租期的意义是可回收：到期不续就释放给别人用，避免一台机器长期占着地址又不出现。

报文	方向	源 IP	目的 IP
Discover	客户端到服务器	0.0.0.0	255.255.255.255
Offer	服务器到客户端	服务器地址	255.255.255.255
Request	客户端到服务器	0.0.0.0	255.255.255.255
ACK	服务器到客户端	服务器地址	255.255.255.255

模板/流水线

1. 判本网段有无服务器：没有就由中继代理转发。
2. 客户端广播 Discover，源 0.0.0.0，目的全 1，带事务 ID。
3. 服务器回 Offer，携带候选 IP、掩码、网关、DNS、租期。
4. 客户端广播 Request，声明选中哪台服务器的哪个地址。
5. 服务器回 ACK，客户端用配置好的掩码与网关开始通信。
6. 到 T1 单播续租，到 T2 广播续租，都失败则租期到点重新 Discover。

> 以为 Request 是发给选定服务器的单播，它是广播 · 把 DHCP 当成网络层协议 · 续租 T1 还写 255.255.255.255

NAT 地址转换表填写

真题年份 | 2019/2020 | NAT 必考” 填表” 加” 说清四个阶段源与目的怎么变”

必背结论

- 私有地址三段：10.0.0.0/8、172.16.0.0/12、192.168.0.0/16，不能出现在互联网上，可以在不同内网重复用。
- 私网能重复的原因是公网路由表里根本没有这三段，出网必须换成公网地址。
- 静态 NAT 一对一固定映射，NAPT 一对多靠端口；题目说” 多台内网主机共用一个公网 IP” 就是 NAPT。
- 代价是破坏端到端：外网不能主动连内网，要靠静态端口映射；公网端口用尽就再也转不出去。
- NAT 表一行四项：内网 IP 比端口、公网 IP 比端口；靠端口区分一对多，这就是 NAPT。
- 出内网：改源 IP 加源端口（私有换成公网）；进内网：改目的 IP 加目的端口（公网换回私有）。
- 普通路由器转发时 IP 地址不变，只有 NAT 路由器必须改一个 IP。
- 改了 IP 要重算 IP 首部校验和，改了端口还要重算 TCP 或 UDP 校验和。
- 2020 四段式：H2 发出（源为私有、目的为服务器公有）、NAT 转发（源变成 NAT 外口公网、目的不变）、普通路由器（都不变）、最后一跳（目的变成内网服务器的私有、源不变）。

内网地址比端口	公网地址比端口	外部看到的
192.168.1.2 比 3300	203.1.1.1 比 4001	203.1.1.1 比 4001
192.168.1.3 比 3300	203.1.1.1 比 4002	203.1.1.1 比 4002

（内网端口允许相同，公网端口必须唯一，否则回不来）

模板/流水线

```
// 填 NAT 表
出方向：查表（内网 IP，内网端口）
        命中 → 直接复用该行；未命中 → 分配一个未用的公网端口 p，记一行
        改写 源 IP = 公网 IP，源端口 = p;       // 目的一个字节不动
进方向：查表（目的 IP，目的端口）
        命中 → 改写 目的 IP = 内网 IP，目的端口 = 原内网端口
        未命中 → 丢弃
每次改写后：重算 IP 首部校验和；改过端口还要重算 TCP/UDP 校验和
```

进内网时去改源地址，进内网只改目的 · 以为 NAT 只改 IP 不改端口，一对多就写不出来 · 忘写重算校验和这一分

RIP / OSPF / BGP 对比

真题年份 | 2013/2014（42 题）/2024 | 只考小问：谁是内部、度量是什么、封装在哪

必背结论

- RIP：内部网关协议，距离矢量，度量是跳数（最大 15，16 表示不可达），每 30 s 换整表，封装在 UDP（端口 520），管理距离 120。
- OSPF：内部网关协议，链路状态，用 Dijkstra 算最短路径，度量是代价（10 的 8 次方除带宽），触发式洪泛，直接封装在 IP（协议号 89），管理距离 110。
- BGP：外部网关协议，路径矢量，用于 AS 之间，先建 TCP 连接（端口 179），管理距离 eBGP 20、iBGP 200。
- 一句话必背：RIP 在 UDP 里、OSPF 在 IP 里、BGP 在 TCP 里。
- 距离矢量是” 和邻居换整张表”，链路状态是” 洪泛链路信息、各自建树算路”。
- 防环手段：RIP 用水平分割与毒性逆转，OSPF 用区域划分，BGP 用 AS 路径里出现自己就丢弃。
- AS 是单一技术管理下的一组网络；AS 内部用 IGP，AS 之间用 EGP，2013 考的就是这条边界。
- OSPF 五种分组：问候、数据库描述、链路状态请求、链路状态更新、链路状态确认。

- BGP 四种报文：打开、更新、保活、通知，全部跑在 TCP 179 的连接上。
- 距离矢量的病是坏消息传得慢：链路断了要靠计数到无穷才收敛，所以跳数上限取 15。

项目	RIP	OSPF	BGP
类型	内部 IGP	内部 IGP	外部 EGP
算法	距离矢量	链路状态	路径矢量
度量	跳数，16 不可达	代价，与带宽反比	AS 路径等属性
封装	UDP 520	IP 协议号 89	TCP 179
管理距离	120	110	20 或 200
更新	30 秒整表	触发洪泛 LSA	TCP 上增量

模板/流水线

```
// 判断该走哪条路由协议的路由
if（两条路由前缀长度不同） 取最长前缀；          // 先比前缀
else if（管理距离不同）     取管理距离小的；      // 直连 0 < 静态 1 < eBGP 20 < OSPF 110 < RIP 120
else                        比度量，同协议内才有意义；
// 封装速记：RIP → UDP 520 | OSPF → IP 89 | BGP → TCP 179
```

> 把 OSPF 说成跑在 UDP 上 · 把 BGP 当内部网关协议 · RIP 的 16 不是”还能用 16 跳”，16 就是不可达

从 A 到 B 的完整答题模板

真题年份 | 2015/2019/2020 | 所有网络层大题的通用收口，按八步写不丢分

必背结论

- 八步：判同网、定下一跳、ARP 求 MAC、逐跳转发、TTL 递减、按出口 MTU 分片、经 NAT 改地址、目的主机重组。
- 判同网只比网络号：A 的 IP 与 A 的掩码、B 的 IP 与同一掩码，两个结果相等就同网。
- 同网下一跳就是 B 本身；不同网下一跳是 A 的默认网关。
- 中间每一步：TTL 减 1、为 0 就丢弃并回 ICMP 超时、重算首部校验和、ARP 下一跳、换 MAC 重新封装。
- 分片只在该跳出口 MTU 小于分组长度时发生，重组只在目的主机做，中间路由器不重组。
- 只有 NAT 路由器改 IP，其余路由器 IP 地址一个字都不改。
- 卷面写成一张逐跳表最稳：跳次、设备、源 MAC、目的 MAC、源 IP、目的 IP、TTL，一行一跳，阅卷按行给分。
- TTL 可达性判断：剩余 TTL 等于初始值减经过的路由器数，减到 0 的那台丢弃分组，所以”能到达”要求经过的路由器数小于初始 TTL。
- 经过交换机时什么字段都不改，只按交换表转发帧，TTL 与 IP 一个字节不动。

模板/流水线

```
// ① 判同网
if ((A & M) == (B & M)) next = B; else next = A 的默认网关;
// ② 求 next 的 MAC（同网 ARP B，跨网 ARP 网关），封装成帧从 A 的接口发出
// ③ 每个路由器 r:
//    TTL--; 若为 0 → 丢弃 + 回 ICMP 超时（源地址为 r 的出口 IP）
//    重算首部校验和;
//    若 r 是 NAT 路由器：出内网改源 IP 与端口，进内网改目的 IP 与端口
//    最长前缀查表定 下一跳 与 出口接口;
//    若 分组长度 > 出口 MTU → 按分片模板切片，各片独立转发
//    用 ARP 求下一跳 MAC，重封装，从出口发出
// ④ 到达目的网段的最后一台路由器：ARP 目的主机，帧目的 MAC = B
// ⑤ 目的主机按标识加片偏移重组，交给上层（看协议字段）
// ⑥ 回程：源与目的互换；若经 NAT 出口，回包目的地址是 NAT 表的公网那一侧
```

忘了 TTL 会在某台路由器归零就答”能到达” · 在中间路由器写”重组” · 把分片写在最后一跳，而不是出口 MTU 不够的那一跳

UDP 与 TCP 对比表

真题年份 | 选择常考，大题未考 | 传输层大题的送分入口，服务 2016/2023/2026 的开头（先定协议再算数）

必背结论

- UDP 首部固定 8 B，四个字段各 2 B：源端口、目的端口、长度、校验和。
- 长度字段 = 首部 + 数据，$L_{总} = 8 + L_{数据}$，所以最小值为 8，最大 65535，数据上限 65507 B。
- TCP 首部最小 20 B，带选项最大 60 B（数据偏移 4 位 × 4 B）。
- UDP 无连接、不可靠、保留报文边界；TCP 面向连接、可靠、面向字节流、全双工。
- 选 UDP 的判据是「一句请求一句应答就够」：重传和建连的时延比丢一个报文的代价更贵，所以 DNS、DHCP 这类一次问答的业务宁可自己重发。
- 端口 16 位：0–1023 熟知端口（DNS 53、HTTP 80、FTP 21 与 20）；1024–49151 登记端口；49152–65535 客户端临时端口。
- 复用是按源端口把多进程数据合到传输层，分用是按目的端口把数据交给进程；套接字 = IP + 端口。
- 首部里的「窗口」字段是 TCP 才有的，含义是接收窗口。

对比项	UDP	TCP
连接	无连接，直接发	三次握手建立，四次挥手释放
可靠性	不可靠，出错丢弃不重传	确认加重传，可靠有序不重复
数据单位	用户数据报，保留报文边界	字节流，不保留边界
首部开销	8 B	20 B，带选项最多 60 B
流量控制	没有	滑动窗口，rwnd 由接收方通告
拥塞控制	没有	慢开始、拥塞避免、快重传、快恢复
通信方式	一对一、一对多、多对多	只能一对一，全双工
典型应用	DNS、DHCP、TFTP、实时音视频	HTTP、FTP、SMTP、Telnet

模板/流水线

```
/* UDP 首部 8 B = 四个 2 B 字段，按字节定位 */
字节 0-1 : 源端口
字节 2-3 : 目的端口
字节 4-5 : 长度（首部 + 数据，最小 8，最大 65535）
字节 6-7 : 校验和（填 0 表示不校验）

/* 三个必算数 */
长度字段 = 数据长度 + 8;
数据部分 = 长度字段 - 8;
校验和覆盖 = 12 B 伪首部（源 IP 4 + 目的 IP 4 + 0 + 协议 17 + UDP 长度 2）+ 首部 + 数据

/* 与 TCP 的分工速判 */
要可靠 / 有序 / 面向字节流  -> TCP（首部 20 B，端口 80、21）
要低延迟 / 小报文 / 自己重传 -> UDP（首部 8 B，端口 53、67、69）
```

把 UDP 的「长度」当成数据长度——它是首部加数据的全长，最小 8 · UDP 也算校验和，只是校验和可填 0 表示不校验，别答成 UDP 没有校验和 · UDP 8 B、TCP 20 B 这两个数考试不给，必须背

TCP 首部逐字段

真题年份 | 2012 | 逐字节解析一套十六进制首部是那道 5 分组题的骨架，2016/2023/2026 的握手题也要靠字段定位

必背结论

- 序号 32 位：本报文段数据第一个字节的编号，是字节编号，不是第几个报文段。
- 确认号 32 位：期望收到的下一个字节编号 = 已正确收到的最后一个字节 + 1；只有 ACK = 1 时才有效。
- 数据偏移 4 位：首部长度，单位 4 B，首部字节数 = 数据偏移值 $\times 4$；值 5 即 20 B，值 15 即 60 B。
- 六个标志位各 1 位：URG、ACK、PSH、RST、SYN、FIN；握手挥手只动 SYN、ACK、FIN。
- 窗口 16 位：接收窗口 rwnd，单位字节，16 位最大 65535 B，要更大得用窗口扩大选项。
- 校验和 16 位：覆盖首部 + 数据 + 12 B 伪首部（源 IP 4 + 目的 IP 4 + 0 + 协议 1 + TCP 长度 2）。
- 其余字段：源端口 16、目的端口 16、紧急指针 16、选项（MSS、窗口扩大、SACK、时间戳）。
- MSS 选项只在 SYN 里出现，声明「我能收的报文段最大多少字节」；它同时也是拥塞窗口的单位，看到 MSS 就该想到 TCP。
- 数字例子：seq = 100、长度 100 B ⇒ 占用 100–199，对方回 ack = 200。
- 首部里只有三处值得逐位看：标志位判阶段、序号与确认号算字节、窗口判还能发多少；其余字段扫一眼带过即可。

模板/流水线

```
/* 前 40 B = IP 首部 20 B + TCP 首部 20 B，每 4 B 一行共 10 行 */
行 1-5 : IP 首部（版本 + 首部长度 | 总长度 | 标识 | 标志 + 片偏移 | TTL | 协议 = 6 | 校验和 | 源 IP | 目的 IP）
行 6   : 源端口 2 B + 目的端口 2 B
行 7   : 序号 seq 4 B
行 8   : 确认号 ack 4 B
行 9   : 数据偏移 4 bit + 保留 6 bit + 六个标志 6 bit 共 2 B，紧跟窗口 2 B
行 10  : 校验和 2 B + 紧急指针 2 B

/* 三步判定 */
/* 1. 定方向：源 IP 等于主机 IP，就是主机发的 */
/* 2. 定阶段：SYN=1 且 ACK=0 -> 第 1 次握手；SYN=1 且 ACK=1 -> 第 2 次；
 *           只有 ACK=1 且 seq 等于上次的 ack -> 第 3 次握手或数据段 */
/* 3. 定字节：序号与确认号都是字节编号，直接相减或加 1 */
```

数据偏移单位是 4 B，不是 1 B，算出 20 别再乘一次 · 序号是字节编号，seq = 1000 与「第 1000 个报文段」毫无关系 · 大写 ACK 置位小写 ack 才生效，题目给 ack 先看标志位

TCP 三次握手流水线

真题年份 | 2012/2016/2023/2026 | 传输层大题第一小问的固定开局，四年都从这里起算

必背结论

- 第 1 步：SYN = 1，seq = x，不带数据，消耗 1 个序号。
- 第 2 步：SYN = 1 且 ACK = 1，seq = y，ack = x + 1，不带数据，消耗 1 个序号。
- 第 3 步：ACK = 1，seq = x + 1，ack = y + 1，可以携带数据。
- x 与 y 各自随机独立，两个方向两套编号，互不相干。
- 第 1 个数据字节的序号 $= x + 1$：2016 给 x = 100，数据从 101 起；2023 同样 100，数据是 101–18100。
- 第三次握手能捎带数据，所以建立连接只花 1 个 RTT，不是 1.5 个 RTT。
- 客户端发出第 3 步就认为连接已建立，服务器收到第 3 步才进入 ESTABLISHED；这一步既能带数据，也能在 SYN 洪泛里被用作半连接队列的清理点。
- 为什么必须三次：防止已失效的连接请求报文段又传到服务器，使服务器建立无用连接、白占缓存；两次不够（服务器无法确认客户端收到了自己的 SYN 与确认），四次多余（第三次已把双向确认补齐）。

模板/流水线

```
/* 通用流水线，把 x、y 换成题面给的数 */
第 1 步 A->B : SYN=1,            seq=x,   ack 无效    /* 消耗 x，数据从 x+1 起 */
第 2 步 B->A : SYN=1, ACK=1,     seq=y,   ack=x+1     /* 消耗 y，1 个序号 */
第 3 步 A->B : ACK=1,            seq=x+1, ack=y+1     /* 可带数据，不带也消耗 x+1 */

/* 2016：x = 100，MSS = 1 KB，缓存 20 KB，RTT = 200 ms，ssthresh = 32 KB
 * 第 2 步 ack = 101；第 3 步 seq = 101、ack = y+1
 * 数据 101 ~ 20580（20 x 1024 B），下一个待发 = 20581
 */
/* 2023：x = 100，文件 18000 B，MSS = 1000 B
 * 第 1 个字节 = 101，最后一个字节 = 18100
 * 客户端 FIN 再消耗 18101 -> 服务器第 2 次挥手 ack = 18102
 */
```

ack 写成 x，忘了 SYN 消耗 1 个序号 · 第 3 步 seq 写成 y+1——seq 用自己的编号，只有 ack 用对方的 · 答「为什么三次」只写「可靠」两字，要写「防失效的连接请求」

四次挥手与 TIME_WAIT

真题年份 | 2016/2023/2026 | 2016 考被动方释放 1.5 RTT，2023 考第二次挥手的 ack，都是纯算数

必背结论

- 第 1 步 A → B：FIN = 1，seq = u，FIN 消耗 1 个序号。
- 第 2 步 B → A：ACK = 1，seq = v，ack = u + 1；此后 B 仍可继续给 A 发数据。
- 第 3 步 B → A：FIN = 1 且 ACK = 1，seq = w（B 一直在发数据，从 v 变到 w），ack 仍是 u + 1。
- 第 4 步 A → B：ACK = 1，seq = u + 1，ack = w + 1；A 进入 TIME-WAIT。
- 状态：主动方 FIN-WAIT-1 → FIN-WAIT-2 → TIME-WAIT → CLOSED；被动方 CLOSE-WAIT → LAST-ACK → CLOSED。
- 第 2 步之后连接处于半关闭：A 不再发数据但还能收，B 可以继续把剩下的数据发完，这就是四步不能压成三步的原因。
- 等 2 MSL 两个理由：最后那个 ACK 丢了还能重发；让本连接的残留报文在网中消亡，不污染新连接。
- 谁先发 FIN 谁就是主动关闭方，服务器先关也照样成立，只是两边的时间结论互换；TIME-WAIT 期间这个四元组不能被新连接复用。
- 时间三件套：建立连接 1 RTT；被动方释放 1.5 RTT（第 2、3 步可合并）；主动方释放 1 RTT + 2 MSL。

$$T_{\text{主动}} = RTT + 2MSL, \qquad T_{\text{被动}} = 1.5\,RTT$$

- 数字：RTT = 200 ms 时被动方 300 ms；RTT = 50 ms、MSL = 800 ms 时主动方 50 + 1600 = 1650 ms、被动方 75 ms。

模板/流水线

```
/* 挥手流水线：u 是 A 的数据末字节，v 是 B 的数据末字节 */
第 1 步 A->B : FIN=1,            seq=u,   ack 可带    /* FIN 占 1 个序号 */
第 2 步 B->A : ACK=1,            seq=v,   ack=u+1     /* 半关闭，B 仍可发 */
第 3 步 B->A : FIN=1, ACK=1,     seq=w,   ack=u+1     /* ack 不变，A 已不发数据 */
第 4 步 A->B : ACK=1,            seq=u+1, ack=w+1     /* A 进 TIME-WAIT 等 2MSL */

/* 2023 上传场景：数据 101 ~ 18100，FIN 消耗 18101
 * 第 2 步 ack = 18101 + 1 = 18102
 */
/* 时间模板 */
主动方关闭时刻 = 开始挥手 + 1 RTT + 2 MSL
被动方关闭时刻 = 开始挥手 + 1.5 RTT
```

第 3 步挥手的 ack 误写成 w+1——A 早已不发数据，ack 恒为 u+1 · 主动方与被动方时间记反：主动方 1 RTT + 2 MSL，被动方 1.5 RTT · 把 MSL 当成 2 MSL 直接乘 2

序号与确认号计算模板

真题年份 | 2023/2026 | 2023 考第 1 个字节序号与挥手 ack，2026 考还能再发几个字节，两年同一套加减

必背结论

- 序号 = 本报文段第一个字节的编号；确认号 = 期望收到的下一个字节编号。
- 已正确收到多少字节：确认号相减，不加 1（ack 从 101 到 1101 正好 1000 B）。
- 某一段有多少字节：末 − 首 + 1，要加 1（1001–5000 是 4000 B）。
- 下一步该发哪个字节：上一次已发出的最后字节 + 1，按序号走，不按确认号走。
- 消耗序号四处：第 1 次握手 SYN、第 2 次握手 SYN、第 3 次握手（不带数据也消耗）、FIN。
- 于是一个方向传 N 字节数据，编号一共用掉 N + 2 个（SYN 一个、FIN 一个），这正是 2023 那把 18000 B 算成 18102 的原因。
- 发送窗口还能发多少 = 窗口值 − 已发出但未确认的字节数。

$$\text{可用窗口} = rwnd - (\text{已发出未字节} - \text{已确认未字节}), \qquad \text{下次序号} = \text{已发出未字节} + 1$$

模板/流水线

```
/* 三步走：定起点 -> 定范围 -> 定确认号与可用窗口 */
/* 第 1 步 定起点 */
起点 = 初始序号 + 1;                    /* 初始序号被 SYN 吃掉 */
/* 第 2 步 定范围 */
数据字节 = [起点, 起点 + 长度 - 1];
该段字节数 = 末 - 首 + 1;               /* 要加 1 */
/* 第 3 步 定确认号与可用窗口 */
ack = 已正确收到的最后字节 + 1;          /* 收到 100~199 就回 200 */
已收到字节数 = ack - 起点;               /* 不加 1 */
可用窗口 = 窗口值 - (已发未字节 - 已确认未字节);
下次发送起点 = 已发未字节 + 1;           /* 看序号，不看 ack */

/* 例 1（2023）：x = 100、文件 18000 B、MSS = 1000 B
 * 起点 101，末 18100，共 18000 B = 18 个 MSS
 * FIN 再占 18101 -> 对端 ack = 18102
 */
/* 例 2（窗口）：已发 501~700、收到 ack = 501、窗口 500
 * 未确认 200，可用 300，从 701 发到 1000
 * 若 ack 改成 701 就是全部确认：未确认 0，可用 500，从 701 发到 1200
 */
```

「下一次序号 = 上一次确认号」只在三次握手时巧合成立，其余一律按序号接 · 确认号相减不加 1，范围计数才加 1，两个场景别混 · 拿到 ack 先看它有没有追平已发数据，没追平的那部分继续占窗口

流量控制与滑动窗口

真题年份 | 选择常考，大题未单独考 | 服务 2016 第 2、3 问与 2026 的发送窗口问，每题都要先算 rwnd 再取 min

必背结论

- rwnd 由接收方决定，写在 TCP 首部的窗口字段，含义是「我还能收多少字节」；窗口通告的是字节数不是段数，题目给 MSS 时先换成段数再比。
- $rwnd$ = 接收缓存 − (已收到 − 已交付给应用)；题目说「只存入不取走」时它只减不增。
- 发送窗口 = $\min(cwnd, rwnd)$，所以发送窗口永远不大于 rwnd。
- 窗口字段 16 位最大 65535 B，长肥管道要靠窗口扩大选项把 rwnd 撑大。
- 收到零窗口通告就启动坚持计时器，周期性发 1 B 探测，窗口一开继续发；零窗口不等于连接断开，已发出的数据仍可重传。
- 2016 数字：缓存 20 KB、MSS 1 KB、收到第 8 个确认 ⇒ rwnd = 20 − 8 = 12 KB，cwnd = 9 KB，发送窗口 = 9 KB。

- MSS = 1000 B、cwnd = 2000 B、rwnd = 4000 B ⇒ 发送窗口 = 2000 B = 2 个段；已发 2 段其中 1 段已确认 ⇒ 还能再发 1000 B。

模板/流水线

```
/* 一行 cwnd、一行 rwnd，逐轮往下推 */
/* 初始 cwnd = 1 MSS，ssthresh = 16 MSS，rwnd = 64 MSS，数据只存不取 */
轮次   cwnd     本轮发出 MSS    rwnd 剩余
 0      1           -            64
 1      2           1            63
 2      4           2            61
 3      8           4            57
 4     16           8            49

/* 两条公式 */
rwnd = rwnd0 - 累计已收到的 MSS 数;      /* 任何阶段都能用 */
发送窗口 = min(cwnd, rwnd);
/* 收到 15 个 MSS -> cwnd = 1 + 15 = 16，rwnd = 64 - 15 = 49
 * 收到 31 个 MSS -> cwnd 必须画图（16 之后转拥塞避免，17），rwnd = 33
 */
```

把 TCP 首部的窗口当成发送窗口——它是接收窗口 · rwnd 与 cwnd 算完忘了取 min · 题目说「数据只存入不取走」时 rwnd 不回升，别写成固定值

拥塞控制四阶段

真题年份 | 2016/2023 | 两年各有一问推 cwnd，2023 还拿它算平均速率；选择题里窗口题几乎年年有

必背结论

- cwnd 单位 MSS，横轴是传输轮次 RTT，纵轴是 cwnd。
- 慢开始：每收到一个确认 cwnd + 1，效果是每轮翻倍，但增长是逐步的（2 → 3 → 4，不是 2 直跳 4）；横轴一格是一个 RTT，翻倍体现在一轮结束时，不是每个确认就跳一格。
- 拥塞避免：cwnd 到 ssthresh 后每轮只 + 1，线性增长。
- 超时：ssthresh ←当前 cwnd ÷ 2，cwnd ← 1，重回慢开始。
- 3 个冗余 ACK：快重传立即重发丢失的段；快恢复 ssthresh ← cwnd ÷ 2，cwnd ←新 ssthresh，之后线性增。
- 快重传比超时快，是因为三个冗余 ACK 说明报文还能送到、只是某一段丢了；超时说明连确认都回不来，只能从 1 重来。
- 一步到位法只在慢开始成立：cwnd = 1 + 累计已确认的 MSS 数；跨过 ssthresh 后只能画图。
- 曲线数字：1 → 2 → 4 → 8 → 16（ssthresh = 16）→ 17 →⋯→ 24 超时 ⇒ ssthresh = 12、cwnd = 1 → 1、2、4、8、12（不跳到 16）→ 13、14。
- 2016：阈值 32 KB 未到，收到第 8 个确认 cwnd = 9 KB。2023：阈值 4 MSS，ack = 2101 ⇒ 2 个确认 ⇒ cwnd = 3；ack = 7101 ⇒ 7 个确认 ⇒ 1 → 2 → 4 → 5 MSS。

模板/流水线

```
/* 输入事件序列，输出每轮 cwnd；事件：NORMAL / TIMEOUT / DUPACK */
int cwnd = 1, ssthresh = 16, r;
for (r = 1; r <= n; r++) {
    if (ev[r] == TIMEOUT) {
        ssthresh = cwnd / 2;      /* 减为【当前】cwnd 的一半，不是原门限 */
        cwnd = 1;                 /* 回到慢开始 */
    } else if (ev[r] == DUPACK) {
        ssthresh = cwnd / 2;
        cwnd = ssthresh;          /* 快恢复：直接降到新门限 */
    } else if (cwnd < ssthresh) {
        cwnd = cwnd * 2 > ssthresh ? ssthresh : cwnd * 2;
                                  /* 慢开始：本轮翻倍，越界就停在门限 */
    } else {
        cwnd += 1;                /* 拥塞避免：本轮 +1 */
    }
```

```
    printf("RTT %d: cwnd=%d ssthresh=%d\n", r, cwnd, ssthresh);
}
/* ssthresh = 16 的典型输出
 * 1 2 4 8 16 | 17 18 ... 24 | 超时 -> ssthresh = 12、cwnd = 1
 * 1 2 4 8 12（卡在门限 12，不写 16）| 13 14 ...
 */
/* 2023：ssthresh = 4，7 个确认 -> 1，2，4，5（第 4 个确认起转拥塞避免） */

/* cwnd 曲线（横轴 RTT，纵轴 MSS）
 * 24 |                                *
 * 20 |                   *
 * 16 |*        <- ssthresh = 16
 * 12 |                     \  超时后新门限 12
 *  8 |      *
 *  4 |   *
 *  2 | *
 *  1 |*
 *    +---1---2---3---4---5---6---7---8--> RTT
 */
```

阈值减半减的是「当前 cwnd」而不是「原 ssthresh」· 用 cwnd = 1 + 累计确认数跨阶段硬算（31 个确认应是 17 不是 32）· 慢开始当成一步跳 2 倍，被问「收到第 3 个确认时 cwnd 多少」就露馅

吞吐率与传输时间

真题年份 | 2025 | 卫星链路三问：单向传播时延、最大吞吐量、传 4000 B 文件的最短时间；2023 也考平均速率 2.4 Mb/s

必背结论

- 单向传播时延 = 距离 ÷ 光速：36000 km ÷ 300000 km/s = 0.12 s = 120 ms，RTT = 240 ms。
- 最大吞吐量取瓶颈：卫星链路每方向 200 kb/s 就是瓶颈，主机接千兆以太网也跑不过 200 kb/s。
- 带宽时延积 BDP = C × RTT = 200 kb/s × 0.24 s = 48 kbit = 6 KB，这是长肥管道要的最小窗口。
- 窗口不够时链路空转，吞吐率被窗口与 RTT 卡住，上限就是 窗口$/RTT$；卫星链路正是典型的长肥管道。
- 文件传输时间 = 传输时延 L × 8/C + 单向传播时延；要等确认再加 1 个 RTT。
- 卫星链路那 240 ms 往返是固定的「管道长度」，文件再小也要等满它，所以小文件在这种链路上的有效吞吐率远低于 200 kb/s。
- 4000 B 走 200 kb/s：4000 × 8 ÷ 200000 = 0.16 s，加 120 ms 传播 ⇒ 280 ms。
- 有效吞吐率 = 应用层数据量× 8 ÷总时间；1 B = 8 bit，k = 1000、M = 1000000，只有存储容量用 1024。
- 单位三步走：先全部换成 bit 与秒，除完再换成 kb/s 或 Mb/s；163840 bit/s 写成 163.84 kb/s 即可，换算是细节、列式才是分。
- 2016：20480 B ÷ 1 s = 163840 bit/s = 163.84 kb/s。2023：18000 × 8 ÷ 0.06 s = 2.4 Mb/s。

$$T = \frac{L \times 8}{C} + \frac{d}{v}, \qquad \eta = \frac{L \times 8}{T}, \qquad BDP = C \times RTT$$

模板/流水线

```
tp = d / v;                  /* 传播：36000 km / 300000 km/s = 120 ms */
tt = L * 8 / C;              /* 传输：4000 B * 8 / 200 kb/s = 160 ms */
/* 排队与处理：题目不说就按 0 */
T  = tt + tp;                /* 只要数据到达 = 280 ms */
                             /* 若还要收到确认：再加 1 RTT = 520 ms */
rate = L * 8 / T;            /* bit/s、换 kb/s 除 1000 */
/* 长肥管道：BDP = C * RTT = 200 kb/s * 0.24 s = 48 kbit = 6 KB
 * 窗口 >= BDP 才能跑满 200 kb/s；窗口只有 1 MSS 时吞吐率约 MSS / RTT
 */
/* 2023 复盘：建连 1 RTT + 传 18 个 MSS 用 5 RTT = 6 RTT = 60 ms
 * 18000 * 8 / 0.06 = 2.4 Mb/s
 */
```

传播时延用带宽去除，把 d/v 写成 L/C · 传文件只算传输时延忘了加传播时延，卫星链路上这是 120 ms 的大头 · 2023 那种题漏算建连的 1 RTT，6 RTT 写成 5 RTT

DNS 递归与迭代查询

真题年份 | 2021 | 第 1 问「除 HTTP 外还运行了哪个协议」加逐层封装；查询次数是选择题常客

必背结论

- DNS 用 UDP，服务器端口 53，作用是域名到 IP 的映射；响应超过 512 B 或做区域传送时改走 TCP 53。
- 主机→本地域名服务器：递归查询，本地服务器替主机跑完全程，只回最终答案。
- 本地域名服务器→根 / 顶级 / 权威：迭代查询，每次只告诉你下一个该问谁。
- 层级顺序：根→顶级域（.com）→权威域名服务器，权威可以有多台。
- 主机和本地域名服务器都有高速缓存，命中就不发查询，所以「最少几次」往往是 0 次。
- 封装链（2021 第 1 问答案）：DNS 报文→ UDP → IP →以太网帧。
- 查询次数：n 层域名最多 $n+1$ 次（本地递归 1 次，再迭代问根与各级），全部命中缓存则 0 次。
- 时间例子：每次查询 10 ms、建连与请求响应共 2 个 RTT ⇒ 最多 30 + 20 = 50 ms、最少 20 ms。

模板/流水线

```
/* 递归 + 迭代全流程（括号是封装层） */
/* 0. 浏览器缓存 -> hosts 文件 -> 本地 DNS 缓存，命中直接返回，0 次查询 */
/* 1. 递归: H --(DNS 查询 / UDP 53)--> 本地域名服务器 */
/* 2. 迭代: 本地 --问根----> 只回 .com 顶级服务器地址
 *          本地 --问.com--> 只回 abc.com 权威服务器地址
 *          本地 --问权威--> 回 www.abc.com 的 IP
 */
/* 3. 回程: 本地 --(DNS 应答)--> H，拿到 IP 才能发 TCP SYN */
/* 封装: DNS 报文 | UDP 首部 8 B | IP 首部 20 B | 帧头 14 B + 帧尾 4 B */
/* 次数: 最多 = 1 + 层级数；命中缓存最少 = 0 */
```

把「主机问本地」说成迭代，或把「本地问根」说成递归，方向一反全错 · 忘了先有 IP 才能建连，算时间时漏掉整个 DNS 段 · 默认权威只有一台、忘记缓存，查询次数一定算多

HTTP 请求响应与连接时间

真题年份 | 2011/2021 | 2011 第 3 问全是 RTT 记账（持续非流水线 5 张图），2021 考 HTTP 报文在协议栈里的位置

必背结论

- HTTP 用 TCP，服务器端口 80；HTTP/1.0 默认非持续连接，HTTP/1.1 默认持续连接。
- 请求报文 = 请求行（GET /index.html HTTP/1.1）+ 首部行（Host、Connection、User-Agent）+ 空行 + 实体。
- 响应报文 = 状态行（HTTP/1.1 200 OK）+ 首部行 + 空行 + 实体。
- 格式细节：每个首部占一行，冒号后一个空格；最后一个空行之后才是实体，GET 没有实体，POST 才有。
- 非持续：每个对象一条 TCP 连接，每对象 2 RTT（建连 1 + 请求响应 1）。
- 非持续的代价是每个对象都重新建连、重新慢开始；持续连接把这一笔开销摊到所有对象上，所以对象越多越划算。
- 持续非流水线：共用一条连接，建连 1 RTT，之后每个对象 1 RTT。
- 持续流水线：请求一次全发出，建连 1 RTT + 请求 1 RTT + 响应 1 RTT，约 3 RTT。
- 2011 第 3 问：请求已经发出，页面 + 5 张图共 6 个对象，持续非流水线 ⇒ 6 RTT。
- 数字：RTT = 100 ms、6 个对象 ⇒ 非持续 12 RTT = 1200 ms、持续非流水线 7 RTT = 700 ms、流水线 3 RTT = 300 ms。
- Cookie 存在客户端，补 HTTP 的无状态；条件 GET 用 If-Modified-Since 换 304。

$$RTT_{\text{非持续}} = 2N, \qquad RTT_{\text{持续非流水线}} = 1 + N, \qquad RTT_{\text{流水线}} = 3$$

模板/流水线

```
/* 先数对象 N = 1 个 HTML + (N-1) 张图，再选连接方式 */
rtt = 2 * N;                /* 非持续 */
rtt = 1 + N;                /* 持续非流水线 */
rtt = 1 + 2;                /* 持续流水线 */
/* 题目说「请求已经发出」，建连那 1 个 RTT 不再算
 * 2011 第 (3) 问：rtt = N = 6
 * 题目说「从点击链接开始」，先加 DNS 时间，再加建连 1 RTT
 */
/* 例：N = 6、RTT = 100 ms
 * 非持续 12 RTT = 1200 ms；持续非流水线 7 RTT = 700 ms；流水线 3 RTT = 300 ms
 */
```

2011 那道从「Web 请求已发出」起算，多算 1 个建连 RTT 就错 · 流水线也按 1+N 算，流水线的请求是并行发出的（1+2）· HTTP/1.1 出现 Connection: close 就退回非持续连接

从输入网址到页面显示

真题年份 | 2021 | 那道 9 分题就是这条链路：DNS 封装、ARP 与交换表泛洪、帧数统计，一问一段

必背结论

- 顺序：查缓存与 hosts → DNS 解析得 IP → ARP 求网关 MAC → TCP 三次握手→发 HTTP 请求→服务器响应→渲染页面。
- 应用层加 HTTP 首部；传输层加 TCP 首部 20 B（端口、序号、窗口）；网络层加 IP 首部 20 B（IP、TTL、协议）；链路层加帧头 14 B + 帧尾 4 B。
- 以太网帧总长 $= L + 58$（20 + 20 + 18），不足 64 B 要填充（2012 第 3 问就考这个）。
- MAC 地址逐跳改变，IP 地址端到端不变；TTL 每经一个路由器减 1（2012：64 − 49 = 15 个路由器）。
- 交换机对广播帧和未知目的 MAC 的单播帧一律泛洪，所以同网段其它主机也会收到 ARP 请求，目的 MAC 是 FF-FF-FF-FF-FF-FF。
- ARP 请求是广播、ARP 应答是单播；应答里带着对方 MAC，主机把它存进 ARP 表，过几分钟失效就要重问。
- IP 首部协议字段：TCP 填 6，UDP 填 17，由它决定交给谁。
- 端口定位进程：HTTP 80、DNS 53、FTP 21。
- 这七步里有三处一定被单独拿出来问：DNS 的逐层封装、ARP 请求的目的 MAC、帧长不足 64 B 时的填充。

模板/流水线

```
/* 封装清单，照写就拿封装分；L 为应用层报文长度 */
/* 应用层 */  "GET /index.html HTTP/1.1" + Host + 空行            长 L
/* 传输层 */  + TCP 首部 20 B（源端口随机、目的 80、seq、ack、窗口）L + 20
/* 网络层 */  + IP 首部 20 B（源 IP、目的 IP、TTL=64、协议 =6、总长）L + 40
/* 链路层 */  + 帧头 14 B（目的 MAC、源 MAC、类型 0x0800）+ FCS 4 B   L + 58
/*            若 L + 58 < 64 则填充到 64 B */
/* 时间   */  DNS 时间 + 1 RTT 建连 + 1 RTT 请求响应 + 图片等后续对象
/* 地址   */  MAC 每一跳都换；IP 全程不变；TTL 每跳减 1
```

封装顺序写反（先 IP 后 TCP），或忘掉帧尾 4 B，帧长少 4 · 以为 MAC 全程不变，跨路由器必须换 · 把 DNS 放在建立连接之后，时间和帧数两问一起错